

SC1007: Data Structures and Algorithms

Lecture Notes

Contents

1	Introduction & Memory Management in Python	2
1.1	Introduction to Memory Management	2
1.2	Names, References, and Objects	2
1.3	Python Memory Layout	3
1.4	Immutable vs. Mutable Object Memory Characteristics	3
1.4.1	Immutable Objects (Integers and Strings)	3
1.4.2	Mutable Objects (Lists, Dicts, Sets)	4
1.5	Reference Counting	4
1.5.1	Advantages of Reference Counting	5
1.5.2	Reference Counting in Practice	5
1.5.3	Disadvantages of Reference Counting	5
1.5.4	Circular References	5
1.5.5	Threading and the Mutex Solution	6
1.6	Garbage Collection	6
1.6.1	Generational Collection	6
1.6.2	Advantages and Disadvantages	7
1.6.3	Forcing Garbage Collection	7
1.7	Memory Pooling	7
1.8	Memory Interning	8
1.8.1	Automatic Interning	8
1.8.2	Manual Interning with <code>sys.intern()</code>	8
1.9	Manual Memory Management	9
1.9.1	The <code>del</code> Keyword	9
1.9.2	Weak References	9
1.9.3	The <code>gc</code> Module for Manual Control	10
2	Linked Lists and Object-Oriented Programming in Python	11
2.1	Referential Arrays	11
2.2	Creating Custom Data Types with Classes	12
2.2.1	Object-Oriented Programming	12
2.2.2	Defining a Class	12
2.3	Encapsulation	13
2.3.1	Getter Methods and Private Attributes	13
2.3.2	A Containerised Student List	14
2.4	Linked Lists	14
2.4.1	Singly Linked Lists	15
2.5	Implementation in Python	15
2.5.1	The Node Class	15

2.5.2	The LinkedList Class	15
2.6	Abstract Data Type (ADT) for a Singly Linked List	15
2.6.1	Display	16
2.6.2	Search (findAt)	16
2.6.3	Insertion	16
2.6.4	Deletion	17
2.6.5	Size	18
2.7	Comparison: Lists vs. Linked Lists	18
3	Linked List Variations	19
3.1	Singly Linked List: Refined Operations	19
3.1.1	Maintaining a size Counter	19
3.1.2	General Insert Using findNode	19
3.2	Doubly Linked List	20
3.2.1	Node Structure	20
3.2.2	Display	20
3.2.3	Search	21
3.2.4	Size	21
3.2.5	Insertion	21
3.2.6	Deletion by Value	23
3.2.7	Deletion by Index	23
3.2.8	Advantages and Disadvantages	24
3.2.9	Practical Applications	24
3.3	Circular Linked List	24
3.3.1	Adapting Standard Operations	25
3.3.2	Advantages and Disadvantages	25
3.3.3	Practical Applications	25
3.4	Comparison: SLL, DLL, and Circular	26
4	Stacks	27
4.1	What is a Stack?	27
4.2	Core Stack Operations	27
4.3	Implementation via Linked List (Version 1)	27
4.3.1	Node and Stack Classes	28
4.3.2	peek()	28
4.3.3	push()	28
4.3.4	pop()	29
4.3.5	is_empty() and get_size()	29
4.3.6	Working Example 01 — Full Stack via Linked List	29
4.4	Implementation via Linked List (Version 2)	30
4.4.1	Class Structure	30
4.5	Applications of Stacks	31
4.5.1	Operating Systems	31
4.5.2	Programming and Algorithms	32
5	Queues	33
5.1	The Queue Abstract Data Type	33
5.2	Core Queue Operations	33
5.3	Implementation via Linked List (Version 1)	33
5.3.1	Node and Queue Classes	33
5.3.2	getFront()	34
5.3.3	enqueue()	34

5.3.4	<code>dequeue()</code>	35
5.3.5	<code>isEmpty()</code> and <code>getSize()</code>	35
5.3.6	Working Example 01 — Full Queue via Direct Linked List	35
5.4	Implementation via Linked List (Version 2)	36
5.4.1	Class Structure	36
5.4.2	Adapted Core Operations	36
5.5	Combined Stack–Queue Example: Reversing a Queue	37
5.6	Applications of Queues	38
5.6.1	Operating Systems	38
5.6.2	Graph Algorithms	38
5.6.3	Programming and Simulation	38
6	Priority Queues and Stack/Queue Applications	39
6.1	Priority Queue	39
6.1.1	Types of Priority Queue	39
6.1.2	Characteristics of a Priority Queue	39
6.1.3	Implementation Strategies	39
6.2	Implementing a Priority Queue via Linked List	40
6.2.1	Node and Queue Classes	40
6.2.2	Core Operations	40
6.3	Applications of Priority Queues	42
6.4	Stack Applications	42
6.4.1	Balanced Parentheses Checking	42
6.5	Algebraic Expression Notation	43
6.5.1	Infix Notation	43
6.5.2	Postfix (Reverse Polish) Notation	43
6.5.3	Prefix (Polish) Notation	44
6.6	Converting Between Notations Using a Stack	44
6.6.1	Infix to Postfix Conversion	44
6.6.2	Evaluating a Postfix Expression	45
6.6.3	Prefix to Infix Conversion	46
6.7	Queue Applications: Palindromes and Scheduling	47
6.7.1	Palindrome Detection	47
6.7.2	Scheduling Applications	47
7	Binary Trees	48
7.1	From Linear to Non-Linear Data Structures	48
7.2	Tree Terminology	48
7.3	Binary Trees	49
7.3.1	Operations on a Binary Tree	49
7.4	Binary Tree Traversal	49
7.4.1	Pre-order Traversal	50
7.4.2	In-order Traversal	50
7.4.3	Post-order Traversal	50
7.4.4	Summary: Depth-First Traversal Orders	51
7.4.5	Breadth-First (Level-Order) Traversal	51
7.5	Applications of Tree Traversal	52
7.5.1	Counting Nodes (Size)	52
7.5.2	Finding k -th Level Descendants	52
7.5.3	Calculating the Height of a Node	52
7.6	Pre-order Traversal with an Explicit Stack	53

8	Binary Search Trees	54
8.1	The BST Property	54
8.2	BST Insertion	54
8.3	BST Searching	55
8.4	BST In-Order Traversal and Sorted Output	56
8.5	BST Deletion	56
8.5.1	Case 0: Leaf Node (No Children)	56
8.5.2	Case 1: One Child	56
8.5.3	Case 2: Two Children	57
9	AVL Trees	58
9.1	Tree Terminology Review	58
9.2	Tree Balancing and Motivation	58
9.3	AVL Trees and the Balance Factor	59
9.4	AVL Insertion: Cases	60
9.4.1	Case 1 — All Ancestors Have Balance Factor Zero	60
9.4.2	Case 2 — Insertion into the Shorter Subtree of a Non-Zero BF Ancestor	60
9.4.3	Case 3 — Insertion into the Taller Subtree (Rotation Required)	60
9.5	Rotation Operations	61
9.5.1	Right Rotation (LL Case)	61
9.5.2	Left Rotation (RR Case)	61
9.5.3	Left-Right Double Rotation (LR Case)	62
9.5.4	Right-Left Double Rotation (RL Case)	62
9.6	AVL Insertion: Symmetric Structure of Rotations	63
9.7	Other Self-Balancing BSTs	63
10	Introduction to Algorithms	65
10.1	What Is an Algorithm?	65
10.1.1	Properties of a Valid Algorithm	65
10.2	Algorithm vs. Program	65
10.3	Example 1: Arithmetic Series	66
10.4	Example 2: Fibonacci Sequence	67
10.5	Computing Problem Types	68
10.5.1	Searching	68
10.5.2	Sorting	69
10.5.3	Graph Problems	69
10.5.4	String Processing	69
10.5.5	Computational Geometric Problems	69
10.5.6	Numerical Problems and Optimisation	70
10.5.7	Combinatorial Problems	70
10.6	Selecting Data Structures	70
10.7	Algorithm Design Strategies	70
10.8	Summary	72
11	Analysis of Algorithms	73
11.1	Time and Space Complexity	73
11.1.1	Repetition Structures (Loops)	73
11.1.2	Selection Structures (Conditionals)	74
11.1.3	Recursive Functions	75
11.1.4	Useful Series Formulae	76
11.2	Order of Growth	77
11.3	Asymptotic Notation	77

11.3.1	Big-Oh Notation ($\mathcal{O}$)	78
11.3.2	Big-Omega Notation (Ω)	78
11.3.3	Big-Theta Notation (Θ)	79
11.3.4	Summary: Limit-Based Classification	79
11.3.5	Simplification Rules	79
11.3.6	Asymptotic Notation in Equations	80
11.4	Common Complexity Classes	80
11.5	Space Complexity	80
11.6	Summary	81
12	Searching	82
12.1	Sequential Search	82
12.1.1	Time Complexity of Sequential Search	82
12.2	Binary Search	83
12.2.1	Recursive Implementation	83
12.2.2	Iterative Implementation	83
12.2.3	Connection to Binary Search Trees	83
12.2.4	Tree Height and Node Count	84
12.2.5	Worst-Case Time Complexity	84
12.2.6	Average-Case Time Complexity	85
12.3	Jump Search	86
12.3.1	Time Complexity of Jump Search	86
12.4	Summary	87
13	Hash Tables	88
13.1	Direct-Address Tables	88
13.2	Hashing	88
13.3	Hash Functions	88
13.3.1	Division Method (Modulo Arithmetic)	88
13.3.2	Folding Method	89
13.3.3	Mid-Square Method	90
13.3.4	Multiplicative Congruential Method	90
13.4	Collision Resolution	90
13.4.1	Closed Addressing: Separate Chaining	90
13.4.2	Open Addressing	92
13.4.3	Time Complexity of Open Addressing	93
13.5	Deletion Under Open Addressing	94
13.6	Rehashing	94
13.7	Summary	94
14	Tries	95
14.1	Applications of Tries	95
14.2	Trie Implementation Strategies	95
14.2.1	Hash Table Children	96
14.2.2	Array Children	96
14.2.3	Linked List Children (Left-Child Right-Sibling)	97
14.3	Core Operations	97
14.3.1	Helper: Finding a Child	97
14.3.2	Searching for a Word	98
14.3.3	Inserting a Word	98
14.4	Trie Traversal	98
14.4.1	Depth-First Search (Preorder)	99

14.4.2 Breadth-First Search (Level-by-Level)	99
14.5 Application: Printing All Words	99
14.5.1 DFS Approach	99
14.5.2 BFS Approach	100
14.6 Application: Autocomplete	100
14.7 Application: Spell Checking	100
14.8 Summary	101
15 Heaps	102
15.1 Definition and Properties	102
15.2 Array Representation	102
15.3 Core Operations	103
15.3.1 Heapify (Sift Down)	103
15.3.2 Build Heap	104
15.3.3 Insertion	105
15.3.4 Deletion	105
15.4 Complexity Summary	105
15.5 Applications	106
15.6 Summary	106

Chapter 1

Introduction & Memory Management in Python

1.1 Introduction to Memory Management

Memory management refers to the critical process of allocating, deallocating, and coordinating computer memory effectively. The primary objective is to ensure that all processes execute smoothly and efficiently utilise system resources. In Python, this is achieved through a combination of dynamic memory allocation, automatic garbage collection, and reference counting mechanisms.

Definition 1.1.1 (Memory Management). Memory management is the process of controlling and coordinating a program's access to memory, encompassing allocation of memory to running processes, tracking which parts of memory are in use, and reclaiming memory when it is no longer needed.

Python's memory management subsystem provides several key features:

- **Automatic Memory Management:** Python handles memory allocation and deallocation without developer intervention.
- **Memory Pooling:** Previously allocated memory blocks are reused to reduce overhead.
- **Memory Fragmentation Prevention:** Efficient organisation of memory blocks reduces wasted gaps.

The benefits of Python's approach include prevention of memory leaks, optimised memory usage, reduced development complexity, and improved application performance. Poor memory management leads to program crashes and degraded performance.

Core Memory Management Mechanisms

Python employs five main mechanisms working in concert:

1. Reference Counting
2. Garbage Collection
3. Memory Pooling
4. Memory Interning
5. Manual Memory Management

1.2 Names, References, and Objects

Understanding Python's memory model requires distinguishing three distinct concepts:

Definition 1.2.1 (Name (Variable)). A name is a label or identifier that *points to* an object in memory. A Python variable does not directly hold a value; it holds a reference to where the value lives in the heap.

Definition 1.2.2 (Reference). A reference is the link between a name and an object. It stores the memory address of the object, allowing Python to locate and access it without embedding the value in the variable itself.

Definition 1.2.3 (Object). An object is the actual entity stored in memory — an integer, string, list, or user-defined instance. Every object has a unique memory address, a type, a value, and a reference count.

Example 1.2.4 (Name, Reference, and Object). When we write `x = 42`, Python creates an integer object 42 in the heap, then binds the name `x` to that object via a reference. Calling `id(x)` returns the memory address of the object.

```
1 x = 42
2 print(id(x)) # e.g., 140712834927920
```

The name `x` lives on the stack; the integer object 42 lives on the heap.

1.3 Python Memory Layout

Python programs use four distinct memory regions, each with a specific role.

Definition 1.3.1 (Code Segment). Stores the compiled Python bytecode and program instructions. This region is read-only during execution.

Definition 1.3.2 (Static/Global Segment). Stores global variables and static data that persist throughout the entire program's execution (e.g., module-level variables).

Definition 1.3.3 (Stack). Used for function call frames, local variables, and references. Each function call pushes a new frame; when the function returns, its frame is popped and all local names are removed.

Definition 1.3.4 (Heap (Dynamic Memory)). Used to store dynamically allocated objects: lists, dictionaries, class instances, and all other runtime objects. Managed automatically by Python's garbage collector.

Memory Layout Summary

Segment	Contents
Code	Python bytecode, program instructions
Static/Global	Global variables, static data (live for program lifetime)
Stack	Function call frames, local variables, name-to-address bindings
Heap	All objects (integers, lists, dicts, user-defined); GC-managed

1.4 Immutable vs. Mutable Object Memory Characteristics

1.4.1 Immutable Objects (Integers and Strings)

Integers and strings are *immutable*: their value cannot be changed after creation. Any apparent modification creates an entirely new object in the heap.

Example 1.4.1 (Integer Reassignment). When `y = x` is executed, both names point to the same object. After `x += 1`, `x` is rebound to a newly created integer object; `y` still points to the original.

```
1 x = 42
2 y = x      # x and y both reference the same object (0x1234)
3 x += 1     # creates new int 43 at a new address; y unchanged
4 # After: x -> 0x5678 (value 43), y -> 0x1234 (value 42)
```

Example 1.4.2 (String Immutability and Object Reuse). Because strings are immutable, Python can safely let two names reference the same string object (this is called *interning*). Reassigning one name creates a new object and leaves the other name unaffected.

```
1 s1 = "hello"
2 s2 = "hello" # s1 and s2 share the same heap object (refcount: 2)
3 s1 = "hello!" # new object created for s1; s2 still points to "hello"
```

Warning: Immutability Does Not Mean Unrebindable

Immutability means the *object's value* cannot change. The *name* can always be rebound to a different object. `x += 1` for an integer does *not* modify the integer in place; it creates a new integer and rebinds `x`.

1.4.2 Mutable Objects (Lists, Dicts, Sets)

Mutable objects can be modified in place. When two names reference the same mutable object, a mutation through either name is visible through the other.

Example 1.4.3 (Shared Mutable List). After `list2 = list1`, both names point to the same `PyList` object. Appending via `list1` changes the object that `list2` also sees.

```
1 list1 = [1, 2, 3] # PyList at 0x1234; refcount: 1
2 list2 = list1     # refcount: 2; list1 and list2 -> 0x1234
3 list1.append(4)    # modifies the object in place; list2 now also [1,2,3,4]
```

Exam Tip: `is` vs. `==` for Mutable Objects

Use `==` to compare *values*. Use `is` to check whether two names point to the *same object* (same `id()`). For mutable objects, `a is b` is `True` only when both names reference the same heap location, not merely equal contents.

1.5 Reference Counting

Reference counting is Python's *primary* memory management mechanism. Every object on the heap maintains an integer counter tracking the number of names (or containers) currently pointing to it. When the counter drops to zero the object is immediately deallocated.

Definition 1.5.1 (Reference Count). The reference count of an object is the number of active references pointing to it. It increments when a new name is bound to the object or the object is stored in a container, and decrements when a name is deleted or rebound.

1.5.1 Advantages of Reference Counting

- **Predictable Performance:** Memory is freed immediately when no longer needed, avoiding unpredictable pause times.
- **Deterministic Resource Management:** Resources (e.g., file handles) are released promptly, making reference counting well-suited to real-time systems.
- **Simplicity:** The mechanism is straightforward to reason about and transparent to developers.
- **Memory Efficiency:** Reduced memory footprint and better cache utilisation because dead objects are reclaimed instantly.

1.5.2 Reference Counting in Practice

The `sys.getrefcount()` function returns the reference count of an object. Note that calling the function itself creates a temporary reference, so the reported count is one higher than the actual live count; subtract 1 to get the true value.

Example 1.5.2 (Incrementing and Decrementing the Reference Count).

```
1 import sys
2
3 x = [1, 2, 3]
4 print(sys.getrefcount(x) - 1) # 1 (only x references the list)
5
6 y = x
7 print(sys.getrefcount(x) - 1) # 2 (x and y both reference it)
8
9 z = x
10 print(sys.getrefcount(z) - 1) # 3 (x, y, z)
11
12 del x # refcount -> 2
13 del y # refcount -> 1
14 del z # refcount -> 0 => object eligible for garbage collection
```

When the final reference is removed (`del z`), the list's reference count reaches zero and Python's allocator immediately reclaims the heap memory.

1.5.3 Disadvantages of Reference Counting

- **Overhead Costs:** Every assignment and deletion must update the counter, consuming CPU cycles and extra memory.
- **Circular References:** If two objects reference each other, neither count ever reaches zero even when both become unreachable from the program. This requires a supplementary cyclic garbage collector.
- **Threading Complications:** Reference count updates must be atomic to be thread-safe. Python's Global Interpreter Lock (GIL) ensures this but limits multi-threaded parallelism.

1.5.4 Circular References

A circular reference occurs when two or more objects reference each other, forming a cycle. Because each object in the cycle holds a non-zero count (due to the internal references), the cycle persists in memory even after all external names are deleted.

Example 1.5.3 (Circular Reference Between Two Lists).

```

1 list1 = []
2 list2 = []
3 list1.append(list2) # list1 contains a reference to list2
4 list2.append(list1) # list2 contains a reference to list1
5
6 del list1 # still referenced by list2[0]; refcount does not reach 0
7 del list2 # still referenced by list1[0]; refcount does not reach 0
8 # Both objects are unreachable but NOT yet collected by reference counting.
9 # The cyclic garbage collector detects the cycle and frees both.

```

Warning: Circular References Cause Memory Leaks Without GC

Pure reference counting cannot reclaim circular garbage. Python supplements reference counting with a *cyclic garbage collector* (the `gc` module) specifically to handle cycles. Always be mindful of circular structures in long-running programs.

1.5.5 Threading and the Mutex Solution

In a multi-threaded environment, two threads could simultaneously modify the same reference count, producing an incorrect value (a *race condition*). Python's solution at the interpreter level is the GIL. At the application level, mutual-exclusion locks (mutexes) ensure that only one thread accesses a shared resource at a time: Thread 1 acquires the lock, completes its read-modify-write, then releases the lock before Thread 2 may proceed.

1.6 Garbage Collection

Python's cyclic garbage collector (the `gc` module) supplements reference counting to handle objects involved in cycles.

Definition 1.6.1 (Garbage Collection). Garbage collection is the automatic process of detecting objects that are no longer reachable from the program and reclaiming the memory they occupy.

1.6.1 Generational Collection

Python's garbage collector uses a *three-generation* model based on the empirical observation that most objects die young (*generational hypothesis*):

- **Generation 0 (Young):** Newly created objects. Collected most frequently (triggered after every 700 new object allocations). Objects that survive are promoted to Generation 1.
- **Generation 1 (Middle-Aged):** Objects that survived at least one Generation 0 sweep. Collected after every 10 Generation 0 collections. Survivors are promoted to Generation 2.
- **Generation 2 (Old):** Long-lived objects (e.g., global configuration dictionaries). Collected least frequently — after every 10 Generation 1 collections.

Key Result: Generational GC Thresholds (Default)

Generation	Typical Contents	Collection Trigger
0 (Young)	Temporary loop vars, intermediate results	Every ~700 allocations
1 (Middle)	Session-scoped objects (e.g., a shopping cart)	After 10 Gen-0 sweeps
2 (Old)	Module-level singletons, global config	After 10 Gen-1 sweeps

1.6.2 Advantages and Disadvantages

- **Advantages:** Handles circular references that reference counting alone cannot collect; automatic with no developer effort; safer memory management.
- **Disadvantages:** Collection pauses introduce non-deterministic latency; higher memory usage (objects linger until the next sweep); resource-intensive for large object graphs.

1.6.3 Forcing Garbage Collection

Developers can take manual control of the garbage collector via the `gc` module.

Example 1.6.2 (Manual Garbage Collection to Clear Circular References). The following pattern disables automatic collection during a critical section, then forces a collection sweep to resolve a known circular reference.

```
1 import gc
2
3 gc.disable() # suppress automatic GC during sensitive operations
4
5 list1 = [1, 2, 3]
6 list2 = [4, 5, 6]
7 list1.append(list2) # circular reference
8 list2.append(list1)
9
10 del list1 # refcount still non-zero due to cycle
11 del list2
12
13 gc.collect() # cyclic GC detects and frees both objects
14 gc.enable() # restore automatic GC
```

After `gc.collect()`, both list objects are freed and the heap memory is reclaimed.

1.7 Memory Pooling

Definition 1.7.1 (Memory Pooling). Memory pooling is an optimisation technique in which Python pre-allocates fixed-size blocks of memory and reuses them for small, frequently created objects, avoiding the overhead of repeated system-level allocation and deallocation calls.

Key characteristics:

- **Pre-allocation:** Blocks are reserved ahead of time to reduce allocation cost.
- **Fixed-size Pools:** Separate pools are maintained for different object sizes.
- **Efficient Reuse:** When a pooled object is deleted its slot is returned to the pool rather than released to the OS.
- **Automatic:** Python manages pooling internally via its *pymalloc* allocator; no developer action is required.

Example 1.7.2 (Small Integer and String Pooling). Small integers (typically -5 to 256) and short identifier-like strings are pooled, so two names holding the same value share the same heap object:

```
1 a = 100
2 b = 100
3 print(a is b)    # True -- same object from the integer pool
4
```

```

5 str1 = "hello"
6 str2 = "hello"
7 print(str1 is str2) # True -- same object from the string pool

```

Warning: Pool Sharing is an Implementation Detail

Integer pooling for values outside $[-5, 256]$ and string interning behaviour are *CPython* implementation details that may differ between Python implementations (e.g., PyPy, Jython). Never rely on `is` for value equality of integers or strings in production code; always use `==`.

1.8 Memory Interring

Definition 1.8.1 (Memory Interring). Interring is an optimisation in which Python caches a single canonical copy of an immutable object (typically a string or small integer) and reuses it for all bindings with the same value, saving memory and enabling fast identity comparisons.

- Short strings and small integers are *automatically* interned.
- Interned objects with identical values share the same memory address, so `is` returns `True`.
- Longer or dynamically constructed strings are *not* automatically interned; `sys.intern()` enables manual interning.

1.8.1 Automatic Interring

Example 1.8.2 (Short Strings Share an Address).

```

1 a = "hello"
2 b = "hello"
3 print(id(a) == id(b)) # True -- same cached object
4 print(a is b)         # True
5
6 c = "hello world!"
7 d = "hello world!"
8 print(c is d)         # False -- longer string; not automatically interned

```

1.8.2 Manual Interring with `sys.intern()`

For longer strings that appear frequently (e.g., dictionary keys loaded from a large dataset), manual interning can reduce memory usage and speed up comparisons.

Example 1.8.3 (Manual Interring).

```

1 import sys
2
3 e = sys.intern("hello world!")
4 f = sys.intern("hello world!")
5 print(e is f) # True -- sys.intern() stores one canonical copy

```

Exam Tip: Interring Advantages and Limitations

Advantage: Faster equality checks using `is` (pointer comparison) instead of `==` (character-by-character comparison). **Limitation:** Only effective for immutable objects; manually interning mutable types has no effect.

1.9 Manual Memory Management

Although Python's automatic mechanisms handle most cases, developers occasionally need explicit control, particularly for memory-intensive applications or resource-constrained systems.

1.9.1 The `del` Keyword

`del` explicitly removes a name-to-object binding from the current scope, decrementing the object's reference count. If the count reaches zero, the object is immediately deallocated.

Example 1.9.1 (Explicit Deletion with `del`).

```
1 x = 42
2 y = x           # refcount of int-42 is now 2
3 list1 = [1, 2, 3, 4, 5]
4 list2 = list1    # refcount of the list is now 2
5
6 del x, y         # refcount of int-42 drops to 0 -> freed
7 del list1, list2 # refcount of list drops to 0 -> freed
8 # Both objects are now eligible for garbage collection.
```

Warning: `del` Removes the Name, Not Necessarily the Object

`del x` only removes the binding `x`. If other names still reference the same object its reference count does not reach zero and the object is *not* freed. The object is reclaimed only when *all* references to it are removed.

1.9.2 Weak References

A *weak reference* points to an object without incrementing its reference count. This allows the referenced object to be garbage collected when only weak references remain.

Definition 1.9.2 (Weak Reference). A weak reference is a reference that does not contribute to an object's reference count. When the object is collected, accessing the weak reference returns `None`.

Example 1.9.3 (Using `weakref.ref()`).

```
1 import weakref
2 import sys
3
4 def my_function():
5     return "Hello"
6
7 print(sys.getrefcount(my_function) - 1) # 1 (normal ref count)
8
9 weak_ref = weakref.ref(my_function)
10 # refcount is STILL 1 -- weak reference does not increment it
11
12 print(weak_ref()) # calls the function through the weak ref -> "Hello"
13
14 del my_function    # refcount drops to 0; object collected
15 print(weak_ref())  # None -- the object no longer exists
```

Key Result: Strong vs. Weak References

Property	Strong Reference	Weak Reference
Increments refcount	Yes	No
Prevents GC	Yes	No
Returns after deletion	Object (still alive)	None
Typical use	Normal variable binding	Caches, observer patterns

1.9.3 The gc Module for Manual Control

The `gc` module exposes three key functions:

- `gc.disable()`: Turns off automatic garbage collection. Useful when precise timing is required (e.g., latency-sensitive sections of code).
- `gc.collect()`: Triggers an immediate collection sweep. Returns the number of objects freed.
- `gc.enable()`: Re-enables automatic garbage collection after a manual control section.

Exam Tip: When to Use Manual GC Control

Disable automatic GC only during short, bounded, performance-critical sections where you can guarantee that circular-reference garbage will not accumulate. Always call `gc.enable()` afterwards; leaving GC disabled permanently risks unbounded memory growth from cyclic structures.

Chapter 2

Linked Lists and Object-Oriented Programming in Python

This chapter introduces referential arrays, object-oriented programming (OOP) fundamentals, and the singly linked list — a foundational dynamic data structure that underpins stacks, queues, and many other higher-level abstractions.

2.1 Referential Arrays

Definition 2.1.1 (Referential Array). A *referential array* is an array whose elements store *references* (pointers) to objects rather than the object values themselves. In Python, every variable is a reference to an object on the heap, so all Python lists are referential arrays.

Because Python variables hold memory addresses rather than raw values, multiple names can point to the same object simultaneously. This has two important practical consequences.

Slicing creates a shallow copy. When a slice `temp = my_list[2:5]` is created, Python allocates a new array of references but the individual elements still point to the same objects as in the original list. No object data is duplicated.

Element reassignment redirects the reference. Executing `my_list[2] = 6` does *not* mutate the existing object at index 2; it replaces the reference stored at slot 2 with a reference to the integer object 6. The original object (if no other names point to it) becomes eligible for garbage collection.

Example 2.1.2 (Referential Behaviour of Python Lists). The built-in `id()` function returns the memory address of an object, making the referential nature visible:

```
1 my_list = [1, 2, 3, 4, 5, 6, 7]
2
3 # Slicing: temp and my_list share the same element objects
4 temp = my_list[2:5] # temp holds refs to objects 3, 4, 5
5
6 # Reassignment: slot 2 now points to the integer 6
7 before = id(my_list[2]) # address of object 3
8 my_list[2] = 6
9 after = id(my_list[2]) # address of object 6
10 print(before == after) # False -- different objects
```

Key Result: Python Lists Are Referential

Python's `list` stores *references*, not values. Assignment and slicing copy references; they do not copy object data. Use `copy.deepcopy()` when independent copies of nested mutable objects are required.

2.2 Creating Custom Data Types with Classes

Python provides a small set of built-in primitive types (`bool`, `int`, `float`, `complex`, etc.). Real-world data is almost always more structured than a single primitive — a student record, for instance, combines a name (string), age (int), and CGPA (float). Object-oriented programming addresses this by allowing programmers to define their own composite types.

2.2.1 Object-Oriented Programming

Definition 2.2.1 (Object-Oriented Programming (OOP)). OOP is a programming paradigm in which code is organised around *objects* — entities that bundle state (attributes) and behaviour (methods) — rather than around procedures alone.

Every value in Python is an object. An object has:

1. **A type** — defines what kind of data the object holds (e.g., `int`, `str`, `list`).
2. **An internal data representation** — either primitive (compact storage for `int`/`float`) or composite (multiple fields for user-defined classes).
3. **Methods** — functions bound to the object that expose its behaviour (e.g., `"hello".upper()`).

An object is a specific *instance* of a *class*: `1234` is an instance of `int`; `"hello"` is an instance of `str`.

2.2.2 Defining a Class

Definition 2.2.2 (Class). A *class* is a blueprint for creating objects. It specifies the attributes (data) and methods (behaviour) that every instance of the class will possess.

The key syntactic elements are:

- **Class attribute** — defined at class scope, shared by all instances (e.g., `institution_name = "NTU"`).
- **`__init__` constructor** — called automatically when a new instance is created; initialises instance attributes.
- **`self`** — the conventional first parameter of every instance method; Python passes the calling object to it automatically.
- **Instance attributes** — defined as `self.attr = value` inside `__init__`; stored separately in each object.

Example 2.2.3 (Student Class). The following class models a student record with three instance attributes and a shared class attribute:

```
1 class Student:
2     institution_name = "NTU" # class attribute: shared by all instances
3
4     def __init__(self, name, age, cgpa):
5         self.name = name      # instance attributes
6         self.age = age
7         self.cgpa = cgpa
8
9 # Creating instances
10 student1 = Student("Newton", 20, 3.9)
11 student2 = Student("Fernando", 22, 3.7)
```

```

12
13 print(student1.name)          # Newton
14 print(student2.cgpa)          # 3.7
15 print(Student.institution_name) # NTU

```

The three main benefits of custom data types are **encapsulation** (bundle data and methods into one unit), **reusability** (define once, instantiate many times), and **abstraction** (hide implementation details behind a clean interface).

2.3 Encapsulation

Definition 2.3.1 (Encapsulation). Encapsulation is the OOP principle of bundling data (attributes) and methods that operate on that data within a single class, while controlling external access to internal state.

Python provides three access levels for attributes and methods:

Level	Naming convention	Behaviour
Public	<code>attr</code>	Accessible from anywhere.
Protected	<code>_attr</code>	Convention only — not enforced by Python; signals “do not use outside the class or its subclasses”.
Private	<code>__attr</code>	Name-mangled to <code>_ClassName__attr</code> ; prevents accidental access from outside.

2.3.1 Getter Methods and Private Attributes

When attributes are private, external code must use *getter* (and optionally *setter*) methods to read (or modify) them. This allows the class to enforce invariants and hide the internal representation.

Example 2.3.2 (Private Attributes with Getters).

```

1 class Student:
2     institution_name = "NTU"
3
4     def __init__(self, name, age, cgpa):
5         self.__name = name # private: mangled to _Student__name
6         self.__age = age
7         self.__cgpa = cgpa
8
9     def get_name(self): return self.__name
10    def get_age(self): return self.__age
11    def get_cgpa(self): return self.__cgpa
12
13    def display_details(self):
14        print(f"Name: {self.get_name()}")
15        print(f"Age: {self.get_age()}")
16        print(f"CGPA: {self.get_cgpa()}")
17        print(f"Institution: {Student.institution_name}")

```

Because no setter methods are provided, a `Student` object becomes *effectively immutable* after construction — a deliberate design choice for data-protection scenarios.

Exam Tip: Private vs. Protected in Python

Python *does not enforce* protected attributes (`_attr`) — they are accessible from outside the class; the underscore is a convention signalling “internal use only”. Double-underscore (`__attr`) applies name mangling and provides a weak form of enforcement, but is not true encapsulation in the C++/Java sense.

2.3.2 A Containerised Student List

Combining a private `Student` class with a `StudentList` class that manages a Python list of students illustrates encapsulation in practice:

Example 2.3.3 (`StudentList` with Encapsulated Operations).

```
1 class StudentList:
2     def __init__(self):
3         self.__SL = [] # private list of Student objects
4
5     def insert(self, name, age, cgpa):
6         self.__SL.append(Student(name, age, cgpa))
7
8     def delete(self):
9         self.__SL.pop() # removes the last element
10
11    def print(self):
12        for x in self.__SL:
13            print("name:", x.get_name(),
14                  "\t age:", x.get_age(),
15                  "\t cgpa:", x.get_cgpa())
16
17    sl = StudentList()
18    sl.insert("Newton1", 10, 4.00)
19    sl.insert("Newton2", 30, 4.11)
20    sl.insert("Newton3", 20, 4.22)
21    sl.print()
22    sl.delete() # removes Newton3
23    sl.print()
```

Common encapsulated operations include **Set** (update via method), **Get / Display** (retrieve a value), **Insert** (add a new object), **Delete** (remove an object), and **Size** (count of objects).

2.4 Linked Lists

Definition 2.4.1 (Linked List). A *linked list* is a linear data structure consisting of *nodes* arranged in sequence, where each node stores a data element and a reference (pointer) to the next node in the sequence. The list is terminated by a `None` reference.

Linked lists come in three principal variants:

- **Singly Linked List** — each node has one pointer, to the *next* node.
- **Doubly Linked List** — each node has two pointers, to the *next* and *previous* nodes.
- **Circular Linked List** — the last node’s pointer refers back to the first node.

Key Result: Advantages of Linked Lists

- **Dynamic sizing** — no fixed capacity; nodes are allocated on demand.
- **$O(1)$ front insertion/deletion** — no shifting of elements.
- **No wasted capacity** — memory is used exactly as needed.

Use cases include implementing stacks, queues, music playlists, and browser history navigation.

2.4.1 Singly Linked Lists

In a singly linked list each node holds:

1. A reference to the *data* object (the element of the sequence).
2. A reference to the *next* node (or `None` at the tail).

Because each node is only reachable from its predecessor, traversal proceeds strictly head-to-tail. The list is anchored by a `head` pointer stored in the `LinkedList` object.

2.5 Implementation in Python

2.5.1 The Node Class

Example 2.5.1 (Node Class). Each element of the list is represented by a `Node` instance:

```
1 class Node:
2     def __init__(self, data, next=None):
3         self.data = data # the stored value
4         self.next = next # reference to next Node (None by default)
```

`self.data` holds the element value; `self.next` is the link to the successor. An isolated node has `self.next = None`.

2.5.2 The LinkedList Class

The `LinkedList` class manages the collection of nodes and maintains a `head` pointer to the first node. An optional `size` counter provides $O(1)$ size queries.

Example 2.5.2 (LinkedList Skeleton).

```
1 class LinkedList:
2     def __init__(self):
3         self.head = None # empty list
4         self.size = 0     # track node count for O(1) size queries
```

2.6 Abstract Data Type (ADT) for a Singly Linked List

Definition 2.6.1 (Abstract Data Type (ADT)). An ADT is a high-level specification of a data type that describes *what* operations can be performed, without specifying *how* they are implemented. It encapsulates data and its operations while hiding internal details.

The ADT for a simple singly linked list defines five core operations: display, search, insert, delete, and size. Each is implemented as a method of the `LinkedList` class.

2.6.1 Display

Traversal starts at **head** and follows **next** pointers until **None**, printing each element's data along the way.

Example 2.6.2 (Display All Elements).

```
1 def display(self):
2     current = self.head
3     while current:           # stops when current is None
4         print(current.data, end=" -> ")
5         current = current.next
6     print("None")
```

2.6.2 Search (findAt)

To locate the node at a given index, decrement the index counter while advancing the current pointer. Return **None** on an empty list, an out-of-range index, or if the tail is reached prematurely.

Example 2.6.3 (Find Node at Index i).

```
1 def findAt(self, index):
2     # Bounds check using stored size (O(1))
3     if self.head is None or index < 0 or index >= self.size:
4         return None
5     current = self.head
6     while index > 0:
7         current = current.next
8         index -= 1
9     return current
```

Exam Tip: Storing size Avoids Repeated Traversal

Maintaining a **size** counter lets **findAt** reject out-of-range indices immediately in $O(1)$ instead of traversing to discover them. Always increment **size** on insertion and decrement on deletion.

2.6.3 Insertion

Insertion has three distinct cases depending on position.

2.6.3.1 Insert at the Front

Inserting at index 0 (or into an empty list) requires updating the **head** pointer. The two-step pattern is:

1. Point the new node's **next** to the current **head**.
2. Update **head** to the new node.

Example 2.6.4 (Insert at Front).

```
1 def insert_at_front(self, data):
2     new_node = Node(data)
3     new_node.next = self.head # step 1: link forward
4     self.head = new_node    # step 2: update head
5     self.size += 1
```

2.6.3.2 Insert at the Back

Traverse to the last node (where `last_node.next` is `None`), then link the new node after it. Handle the empty-list case separately.

Example 2.6.5 (Insert at Back).

```
1 def insert_at_back(self, data):
2     new_node = Node(data)
3     if not self.head:
4         self.head = new_node
5         self.size += 1
6         return
7     last = self.head
8     while last.next:
9         last = last.next
10    last.next = new_node
11    self.size += 1
```

2.6.3.3 Insert at an Arbitrary Index

To insert at index i , navigate to the node at index $i - 1$ (the *predecessor*), then splice the new node in between.

Warning: Order of Pointer Updates Matters

Always set `new_node.next = pre.next` *before* `pre.next = new_node`. Reversing the order loses the reference to the rest of the list.

Example 2.6.6 (General Insert at Index).

```
1 def insert(self, data, index):
2     new_node = Node(data)
3
4     # Case 1: empty list or front insertion
5     if self.head is None or index == 0:
6         new_node.next = self.head
7         self.head = new_node
8         self.size += 1
9         return True
10
11    # Case 2: locate predecessor at index - 1
12    pre = self.findAt(index - 1)
13    if pre is None:
14        print("Index out of range")
15        return False
16
17    # Splice new node between pre and pre.next
18    new_node.next = pre.next # MUST come first
19    pre.next = new_node
20    self.size += 1
21    return True
```

2.6.4 Deletion

Deleting a node at index i follows the mirror logic of insertion: locate the predecessor (index $i - 1$), then bypass the target node by setting `pre.next = pre.next.next`. Deleting the head requires updating `self.head` directly. Node deletion is left as a lab exercise.

2.6.5 Size

With a stored `size` counter, querying the length is $O(1)$:

Example 2.6.7 ($O(1)$ Size with Stored Counter).

```
1 def get_size(self):  
2     return self.size # incremented on insert, decremented on delete
```

Without a stored counter the only option is $O(n)$ traversal — always prefer maintaining the counter.

2.7 Comparison: Lists vs. Linked Lists

Property	Python list	Singly Linked List
Memory layout	Contiguous referential array	Non-contiguous nodes
Index access	$O(1)$ random access	$O(n)$ traversal required
Front insert/delete	$O(n)$ (must shift elements)	$O(1)$
Back insert	$O(1)$ amortised	$O(n)$ without tail pointer
Memory usage	May over-allocate (capacity buffer)	Exact; one node per element
Implementation complexity	Built-in; trivial	Requires manual pointer management

Key Result: When to Choose a Linked List

Prefer a linked list when insertions and deletions at arbitrary positions (especially the front) are frequent and random-access by index is rare. Use a Python `list` when random access and back-appending dominate.

Chapter 3

Linked List Variations

This chapter extends the singly linked list introduced in Chapter 2 by covering two important structural variations — the **doubly linked list** (DLL) and the **circular linked list** — and by revisiting several singly linked list operations in the context of a maintained **size** counter.

3.1 Singly Linked List: Refined Operations

The core singly linked list operations were introduced in Chapter 2. This section revisits them with the **size** counter pattern fully integrated, and introduces the general **insert** method that calls **findAt** internally.

3.1.1 Maintaining a size Counter

Rather than counting nodes by traversal every time the list length is needed, the **LinkedList** class stores an integer **size** that is incremented on every insertion and decremented on every deletion.

Example 3.1.1 (LinkedList with Size Counter).

```
1 class LinkedList:
2     def __init__(self):
3         self.head = None
4         self.size = 0 # incremented/decremented on insert/delete
```

With this counter, the **findAt** (or **findNode**) method can reject an invalid index in $O(1)$ without traversing the list:

Example 3.1.2 (findNode with Bounds Check).

```
1 def findNode(self, index):
2     # O(1) bounds check using stored size
3     if self.head is None or index < 0 or index >= self.size:
4         return None
5     cur = self.head
6     while index > 0:
7         cur = cur.next
8         index -= 1
9     return cur
```

3.1.2 General Insert Using findNode

The **insert** method below delegates the predecessor lookup to **findNode(index - 1)**, producing clean, reusable code.

Example 3.1.3 (Insert at Arbitrary Index).


```

1 def insert(self, data, index):
2     if index < 0 or index > self.size:
3         print("Index out of range")
4         return False
5
6     new_node = Node(data)
7
8     if index == 0:          # insert at head
9         new_node.next = self.head
10        self.head = new_node
11        self.size += 1
12        return True
13
14    # Locate predecessor (node at index - 1)
15    prev_node = self.findNode(index - 1)
16    if prev_node is not None:
17        new_node.next = prev_node.next
18        prev_node.next = new_node
19        self.size += 1
20        return True
21    return False

```

Key Result: Two-Step Splice Rule

When inserting `new_node` after `prev_node`:

1. `new_node.next = prev_node.next` (forward link first)
2. `prev_node.next = new_node` (predecessor update second)

Reversing the order loses the tail of the list.

3.2 Doubly Linked List

Definition 3.2.1 (Doubly Linked List). A *doubly linked list* (DLL) is a linked list in which every node carries *two* pointers: **next** pointing to its successor and **prev** (or **pre**) pointing to its predecessor. This enables bidirectional traversal.

3.2.1 Node Structure

The only difference from a singly linked list node is the additional **pre** pointer, initialised to `None`:

Example 3.2.2 (DLL Node Class).

```

1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None # successor link
5         self.pre = None  # predecessor link

```

The `LinkedList` class is unchanged — it still stores `head` and optionally `size`.

3.2.2 Display

Forward traversal of a DLL is identical to a singly linked list because we follow `next` in both cases. The `pre` pointer simply goes unused during a forward pass.

Example 3.2.3 (DLL Forward Print).

```
1 def print_list(self):
2     current = self.head
3     while current:
4         print(current.data, end=" -> ")
5         current = current.next
6     print("None")
```

3.2.3 Search

Searching by value also mirrors the singly linked list approach:

Example 3.2.4 (DLL Search by Value).

```
1 def search(self, data):
2     current = self.head
3     while current:
4         if current.data == data:
5             return True
6         current = current.next
7     return False
```

3.2.4 Size

Size computation via traversal is again the same as for a singly linked list. In practice the counter pattern (Section 3.1.1) is preferred.

Example 3.2.5 (DLL Size by Traversal).

```
1 def size(self):
2     count = 0
3     current = self.head
4     while current:
5         count += 1
6         current = current.next
7     return count
```

3.2.5 Insertion

Insertion into a DLL is more involved than in a singly linked list because *four* pointers must be updated to splice a new node between `current.pre` and `current`. Suppose we want to insert `new_node` immediately *before* the node pointed to by `current` (i.e. at `current`'s index):

1. `new_node.pre = current.pre` – new node looks back to predecessor
2. `new_node.next = current` – new node looks forward to current
3. `current.pre = new_node` – current looks back to new node
4. `new_node.pre.next = new_node` – predecessor looks forward to new node

Steps 3 and 4 can be swapped (an equivalent variant uses `current.pre.next = new_node` before `current.pre = new_node` — both orderings are correct because `new_node.pre` is set in Step 1 and does not change afterwards).

Example 3.2.6 (DLL Insert Before `current` (middle of list)).

```

1  # current = the node already at the target index
2  new_node      = Node(data)
3  current      = self.head
4  for i in range(index):
5      current = current.next
6
7  new_node.pre   = current.pre # step 1
8  new_node.next  = current    # step 2
9  current.pre    = new_node    # step 3
10 new_node.pre.next = new_node # step 4
11 self.size      += 1

```

Warning: Head Insertion is a Special Case

The four-step splice above assumes `current.pre` is not `None`. When inserting at index 0 (before the head), `current.pre` is `None`, so Step 4 would crash. Always handle head insertion separately.

The full production implementation handles all three cases — invalid index, head insertion, and middle/tail insertion:

Example 3.2.7 (Full DLL `insert_at`).

```

1  def insert_at(self, index, data):
2      if index < 0 or index > self.size:
3          raise ValueError("Invalid position")
4
5      new_node = Node(data)
6
7      # Case 1: insert at head
8      if index == 0:
9          new_node.next = self.head
10         if self.head:
11             self.head.prev = new_node
12         self.head = new_node
13     else:
14         # Traverse to predecessor (node at index - 1)
15         current = self.head
16         for i in range(index - 1):
17             current = current.next
18
19         # Splice new node after current
20         new_node.prev = current
21         new_node.next = current.next
22         if current.next:
23             current.next.prev = new_node
24         current.next = new_node
25
26     self.size += 1

```

Exam Tip: Insertion Order Variants

The slides show two equivalent orderings for the four pointer updates. In an exam, either is acceptable provided the final state is consistent: both `pre` and `next` of `new_node` are set before either neighbouring node's pointer is modified to point to `new_node`.

3.2.6 Deletion by Value

To delete the first node whose `data` matches the argument, traverse the list and, when found, bypass the node by updating the previous and next neighbours:

Example 3.2.8 (DLL Delete by Value).

```
1 def delete(self, data):
2     if self.head is None:
3         raise ValueError("List is empty")
4
5     current = self.head
6     while current:
7         if current.data == data:
8             # Patch predecessor's forward link
9             if current.prev:
10                current.prev.next = current.next
11            else:
12                # Deleting the head node
13                self.head = current.next
14
15            # Patch successor's backward link
16            if current.next:
17                current.next.prev = current.prev
18
19            self.size -= 1
20            return True      # node goes out of scope -> GC eligible
21
22        current = current.next
23    return False # data not found
```

Remark 3.2.9. When `delete` returns, the local variable `current` goes out of scope. If no other references to the node exist its reference count drops to zero, making it eligible for Python's garbage collector.

3.2.7 Deletion by Index

Deletion at a given index handles four cases: empty list, out-of-range index, head removal, and middle/tail removal.

Example 3.2.10 (DLL `remove_at`).

```
1 def remove_at(self, index):
2     if self.head is None:
3         print("List is empty")
4         return False
5     if index < 0 or index >= self.size:
6         print("Invalid index")
7         return False
8
9     # Case: remove head
10    if index == 0:
11        self.head = self.head.next
12        if self.head:
13            self.head.prev = None
14        self.size -= 1
15        return True
16
17    # Case: remove from middle or end
18    current = self.head
19    for i in range(index):
20        current = current.next
```

```

21
22     # Bypass current
23     current.prev.next = current.next
24     if current.next: # not the last node
25         current.next.prev = current.prev
26
27     self.size -= 1
28     return True

```

Key Result: DLL Deletion Pointer Updates

Removing a node `current` from a DLL requires at most two pointer updates:

1. `current.prev.next = current.next` (skip forward over `current`)
2. `current.next.prev = current.prev` (skip backward over `current`)

Each update is guarded by a `None` check for head and tail edge cases.

3.2.8 Advantages and Disadvantages

Aspect	Detail
<i>Advantages</i>	
Bidirectional traversal	Can scan forward or backward; essential for some algorithms
Easier deletion	Given a pointer to a node, deletion is $O(1)$ (no need to find predecessor separately)
Easier reverse insert	Inserting before a known node is $O(1)$
<i>Disadvantages</i>	
Extra memory	Each node stores an additional <code>pre</code> pointer
Slower manipulation	More links must be updated per operation
More error-prone	Extra pointer means more potential for bugs

3.2.9 Practical Applications

Doubly linked lists appear in contexts that require efficient bidirectional access, insertion, and deletion.

- **Browser history navigation:** Each visited page is a node; the back and forward buttons traverse `prev` and `next` pointers respectively.
- **Undo/Redo operations:** Text editors store each state change as a node, allowing efficient navigation through edit history in either direction.
- **LRU Cache (Least Recently Used):** Memory management systems use a DLL combined with a hash map to track cache entries; $O(1)$ insertion, deletion, and reordering are all achievable.

3.3 Circular Linked List

Definition 3.3.1 (Circular Linked List). A *circular linked list* is one in which the last node's `next` pointer points back to the first node rather than to `None`, forming a closed loop. There is no natural end to the sequence.

Two standard variants exist:

- **Circular singly linked list:** Only `next` pointers; the last node's `next` points to the head.
- **Circular doubly linked list:** Both `next` and `pre` pointers; the last node's `next` points to the head *and* the head's `pre` points to the last node.

3.3.1 Adapting Standard Operations

Most operations require only small adjustments compared to their non-circular counterparts:

- **Display, Search, Size:** The traversal stop condition changes from `current is None` to `current is self.head` (after the first step). Without this change, traversal loops forever.
- **Insert and Delete:** There is no longer a special “last position” case requiring a `None` link — every node always has both a predecessor and a successor. However, operations that affect the first node must still update `head` if the node being inserted or removed is the head.

Warning: Infinite Loop Risk

Because the list never reaches `None`, any traversal that checks only `while current:` will loop infinitely. Always use a sentinel or compare `current` against the starting node to detect when a full cycle is complete.

3.3.2 Advantages and Disadvantages

Aspect	Detail
<i>Advantages</i>	
Continuous cycling	Starting from any node, every other node is reachable — ideal for applications that cycle repeatedly through all elements
<i>Disadvantages</i>	
Infinite loop risk	Flawed stop conditions cause non-terminating traversals

3.3.3 Practical Applications

- **Round-robin scheduling:** Operating system schedulers cycle through a process queue repeatedly; a circular linked list maps directly onto this model, with no special-case “wrap-around” logic needed.
- **Circular buffers:** Used in streaming, data buffering, and telecommunications to manage a fixed-size window of incoming and outgoing data efficiently.

3.4 Comparison: SLL, DLL, and Circular

Property	Singly LL	Doubly LL	Circular LL
Traversal direction	Forward only	Forward and backward	Any direction (loops)
Memory per node	1 pointer	2 pointers	1 or 2 pointers
Head insertion	$O(1)$	$O(1)$	$O(1)$
Deletion given node	$O(n)$ (need predecessor)	$O(1)$	$O(1)$ if doubly circular
Tail $\rightarrow$ head access	$O(n)$	$O(n)$	$O(1)$ via next
Risk of infinite loop	None	None	High if stop condition wrong

Key Result: Choosing the Right Variant

- Use a **singly linked list** when memory is tight and only forward traversal and front insertion/deletion are needed.
- Use a **doubly linked list** when bidirectional traversal or $O(1)$ deletion given a node pointer is required (e.g., LRU cache, undo/redo).
- Use a **circular linked list** when the application naturally cycles repeatedly over all elements (e.g., round-robin scheduling, circular buffers).

Chapter 4

Stacks

4.1 What is a Stack?

A **stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle: the most recently inserted element is always the first one to be removed. Intuitively, think of a stack of plates in a cafeteria — you always take the topmost plate, and any new plate goes on top.

Definition 4.1.1 (Stack). A stack is a collection of elements with two principal operations:

- **Push** — insert an element onto the top of the stack.
- **Pop** — remove and return the element from the top of the stack.

All insertions and deletions occur at one end only, called the *top*. The ordering discipline is **LIFO**: the last element pushed is the first element popped.

A stack can be implemented using either a Python list or a linked list. The lecture covers both approaches, with the linked-list implementation explored in two versions: a self-contained **Stack** class (Working Example 01) and a version that delegates to a separate **LinkedList** class (Working Example 02).

4.2 Core Stack Operations

Five operations form the public interface of a stack:

Operation	Description
<code>push(data)</code>	Add <code>data</code> to the top of the stack.
<code>pop()</code>	Remove and return the top element; raise <code>IndexError</code> if empty.
<code>peek()</code>	Return the top element without removing it; raise <code>IndexError</code> if empty.
<code>is_empty()</code>	Return <code>True</code> if the stack contains no elements, <code>False</code> otherwise.
<code>get_size()</code>	Return the number of elements currently in the stack.

All five operations run in $O(1)$ time when the stack is backed by a linked list, because every action touches only the head (top) node.

4.3 Implementation via Linked List (Version 1)

The first implementation embeds the linked-list logic directly inside the **Stack** class, keeping everything in one self-contained unit.

4.3.1 Node and Stack Classes

Each element in the stack is wrapped in a `Node` object that stores a data value and a pointer to the next node. The `Stack` class maintains a `top` pointer and a `size` counter.

Example 4.3.1 (Stack and Node Skeleton).

```
1 class Node:
2     def __init__(self, data):
3         self.data = data # stores the value
4         self.next = None # points to the next node
5
6 class Stack:
7     def __init__(self):
8         self.top = None # points to the top node
9         self.size = 0   # tracks the number of nodes
```

4.3.2 peek()

`peek()` returns `self.top.data` without modifying the stack. It must first guard against an empty stack.

Example 4.3.2 (`peek()`).

```
1 def peek(self):
2     if self.is_empty():
3         raise IndexError("Peek from empty stack")
4     return self.top.data
```

4.3.3 push()

Inserting at the front of the linked list corresponds to pushing onto the top of the stack. The four-step procedure is:

1. Create a new `Node` containing the data.
2. Set `new_node.next` to the current `self.top` (link the new node to the existing chain).
3. Update `self.top` to point to the new node.
4. Increment `self.size` by 1.

Example 4.3.3 (`push()`).

```
1 def push(self, data):
2     new_node = Node(data)
3     new_node.next = self.top # link to existing top
4     self.top = new_node # new node becomes the top
5     self.size += 1
```

Key Result: `push()` is $O(1)$

Because `push()` only manipulates the `top` pointer and creates a single node, it executes in constant time regardless of stack size.

4.3.4 pop()

`pop()` removes the top node and returns its data. The five-step procedure is:

1. Check if the stack is empty; raise `IndexError` if so.
2. Save a reference to the current top node (`popped_node = self.top`).
3. Advance `self.top` to `self.top.next`.
4. Decrement `self.size` by 1.
5. Return `popped_node.data`.

Example 4.3.4 (`pop()`).

```
1 def pop(self):
2     if self.is_empty():
3         raise IndexError("Pop from empty stack")
4     popped_node = self.top      # save the top node
5     self.top     = self.top.next # advance the top pointer
6     self.size    -= 1
7     return popped_node.data
```

Warning: Popping from an Empty Stack

Calling `pop()` or `peek()` on an empty stack raises an `IndexError`. Always check `is_empty()` before popping in contexts where the stack could be exhausted.

4.3.5 is_empty() and get_size()

Example 4.3.5 (`is_empty()` and `get_size()`).

```
1 def is_empty(self):
2     return self.top is None # True when no nodes exist
3
4 def get_size(self):
5     return self.size        # O(1) because size is maintained
```

`is_empty()` simply checks whether `self.top` is `None`. `get_size()` returns the pre-maintained counter, so both are $O(1)$.

4.3.6 Working Example 01 — Full Stack via Linked List

The following listing shows the complete, self-contained implementation and a driver that exercises all five operations.

Example 4.3.6 (Working Example 01).

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class Stack:
7     def __init__(self):
8         self.top = None
9         self.size = 0
10
```

```

11 def peek(self):
12     if self.is_empty():
13         raise IndexError("Peek: Empty stack")
14     return self.top.data
15
16 def push(self, data):
17     new_node = Node(data)
18     new_node.next = self.top
19     self.top = new_node
20     self.size += 1
21
22 def pop(self):
23     if self.is_empty():
24         raise IndexError("Pop: Empty stack")
25     popped_node = self.top
26     self.top = self.top.next
27     self.size -= 1
28     return popped_node.data
29
30 def is_empty(self):
31     return self.top is None
32
33 def get_size(self):
34     return self.size
35
36 if __name__ == "__main__":
37     s = Stack()
38     s.push(10)
39     s.push(20)
40     s.push(30)
41     print("Size:", s.get_size())      # 3
42     print("Top element:", s.peak())   # 30
43     print("Popped:", s.pop())         # 30
44     print("Popped:", s.pop())         # 20
45     print("New size:", s.get_size())  # 1
46     print("New top:", s.peak())       # 10
47     print("Is empty?", s.is_empty())  # False
48     print("Popped:", s.pop())         # 10
49     print("Is empty now?", s.is_empty()) # True

```

Exam Tip: Method Call Translation

When Python evaluates `s.push(10)`, it internally translates this to `Stack.push(s, 10)`: the instance `s` becomes `self`, and `10` becomes `data`. All instance attributes (`self.top`, `self.size`) therefore operate on `s`'s own state, not on some global copy.

4.4 Implementation via Linked List (Version 2)

Version 2 achieves the same behaviour by composing an existing full-featured `LinkedList` class rather than hard-coding linked-list logic inside `Stack`. This separation of concerns makes each class independently testable and reusable.

4.4.1 Class Structure

The `Stack` stores a `LinkedList` instance as `self.ll`. All structural mutations delegate to `ll.insert_node(0, item)` and `ll.remove_node(0)`, which operate on the head — corresponding to the stack's top.

Example 4.4.1 (Version 2 — Stack delegating to `LinkedList`).

```

1 class Stack:
2     def __init__(self):
3         self.ll = LinkedList()
4
5     def push(self, item):
6         # Insert at index 0 (head = top of stack)
7         self.ll.insert_node(0, item)
8
9     def pop(self):
10        if self.is_empty():
11            raise IndexError("Pop from empty stack")
12        item = self.ll.head.item
13        self.ll.remove_node(0)
14        return item
15
16    def peek(self):
17        if self.is_empty():
18            raise IndexError("Peek from empty stack")
19        return self.ll.head.item
20
21    def is_empty(self):
22        return self.ll.size == 0
23
24    def get_size(self):
25        return self.ll.size

```

The key differences from Version 1 are summarised below.

Operation	Version 1	Version 2
push()	Manually links new node to <code>self.top</code>	<code>self.ll.insert_node(0, data)</code>
pop()	Manually advances <code>self.top</code>	<code>self.ll.remove_node(0)</code>
peek()	Returns <code>self.top.data</code>	Returns <code>self.ll.head.item</code>
is_empty()	<code>self.top</code> is <code>None</code>	<code>self.ll.size == 0</code>
get_size()	Returns <code>self.size</code>	Returns <code>self.ll.size</code>

Key Result: Two Valid Implementations

Both Version 1 and Version 2 expose the same five-operation interface and deliver identical $O(1)$ time complexity for every operation. The choice between them is an engineering decision — Version 1 is more self-contained; Version 2 reuses existing `LinkedList` infrastructure and keeps `Stack` thin.

4.5 Applications of Stacks

Stacks appear throughout computer science wherever LIFO access patterns arise naturally.

4.5.1 Operating Systems

- **Function call stack:** Every function call pushes a *stack frame* (containing local variables, parameters, and the return address) onto the call stack. When a function returns, its frame is popped and execution resumes at the saved return address.
- **Undo / Redo:** Text editors push each editing action onto an undo stack. `Ctrl-Z` pops the most recent action and reverses it.

- **Memory stack:** The runtime stack stores local variables and manages function execution contexts.

4.5.2 Programming and Algorithms

- **Nesting and bracket matching:** When parsing source code or markup, opening brackets/tags are pushed; each closing symbol is matched against the top of the stack. A mismatch or non-empty stack at end-of-input signals a syntax error.
- **Expression evaluation:** Stacks underpin postfix (Reverse Polish Notation) evaluation and infix-to-postfix conversion, correctly handling operator precedence without parentheses.
- **Backtracking algorithms:** Depth-first search (DFS) in graphs and trees, maze solving, and Sudoku solving all use a stack (explicit or via the call stack through recursion) to track the frontier of unexplored choices and to revert to the last decision point when a dead end is reached.
- **Browser history:** Visited URLs are pushed as the user navigates forward; the Back button pops them.

Exam Tip: Recognising Stack Problems

A problem is a strong candidate for a stack solution when it involves: (1) matching or validating nested structures (brackets, tags, calls), (2) processing elements in the reverse of their arrival order, or (3) needing to “undo” or “backtrack” to a recent state.

Chapter 5

Queues

5.1 The Queue Abstract Data Type

A **queue** is a linear data structure that enforces a *First In, First Out* (FIFO) discipline: elements are inserted at one end (the **rear**) and removed from the opposite end (the **front**). This mirrors everyday queuing behaviour — the first person to join the line is the first to be served.

Definition 5.1.1 (Queue). A **queue** is a collection of elements supporting insertion only at the rear and removal only at the front, such that items are retrieved in the same order in which they were inserted (FIFO).

A queue can be implemented using either a Python list or a linked list. The linked-list implementation is preferred when efficiency matters, because a **rear pointer** avoids the $O(n)$ traversal that would otherwise be required to locate the tail on every insertion.

Key Result: FIFO Discipline

In a queue, insertion happens at the **rear** and deletion happens at the **front**. The element that has been waiting the longest is always the next to leave.

5.2 Core Queue Operations

The standard interface for a queue exposes five methods.

Method	Description
<code>getFront()</code>	Inspect the element at the front without removing it.
<code>enqueue(data)</code>	Add a new element at the rear of the queue.
<code>dequeue()</code>	Remove and return the element at the front of the queue.
<code>isEmpty()</code>	Return <code>True</code> if the queue contains no elements.
<code>getSize()</code>	Return the current number of elements in the queue.

5.3 Implementation via Linked List (Version 1)

The first implementation embeds the linked-list logic directly inside the `Queue` class, using explicit `front`, `rear`, and `size` attributes.

5.3.1 Node and Queue Classes

Example 5.3.1 (Queue Skeleton — Version 1). The `Node` class stores a data value and a `next` pointer. The `Queue` class maintains `front` and `rear` pointers plus a running `size` count.

```

1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class Queue:
7     def __init__(self):
8         self.front = None # points to the first element
9         self.rear = None  # points to the last element
10        self.size = 0

```

Remark 5.3.2. Maintaining a dedicated **rear** pointer is essential for $O(1)$ enqueue. Without it, every insertion would require traversing the entire list to find the tail — degrading the operation to $O(n)$.

5.3.2 getFront()

Example 5.3.3 (`getFront()` — inspect without removing). Returns the data stored in the front node. Raises `IndexError` if the queue is empty.

```

1 def getFront(self):
2     if self.isEmpty():
3         raise IndexError("Peek from empty queue")
4     return self.front.data

```

5.3.3 enqueue()

Inserting at the rear proceeds in five steps:

1. Create a new `Node` holding `data`.
2. If the queue is empty, set `front` to the new node.
3. Otherwise, link the current rear node's `next` to the new node.
4. Advance `rear` to the new node.
5. Increment `size`.

Example 5.3.4 (`enqueue()` — insert at rear).

```

1 def enqueue(self, data):
2     new_node = Node(data)
3     if self.isEmpty():
4         self.front = new_node # Step 2: first element
5     else:
6         self.rear.next = new_node # Step 3: link to tail
7     self.rear = new_node # Step 4: advance rear
8     self.size += 1 # Step 5: update count

```

Remark 5.3.5. `new_node` is a local variable; once `enqueue` returns, the name goes out of scope. The node itself persists in memory because `self.front` and/or `self.rear` still hold a reference to it.

Exam Tip: Empty-Queue Edge Case in enqueue

When enqueueing into an empty queue, *both* `front` and `rear` must point to the single new node. A common mistake is only setting one of them, leaving the other as `None`.

5.3.4 dequeue()

Removing from the front proceeds in six steps:

1. If the queue is empty, raise `IndexError`.
2. Store the current `front` node in a temporary variable `dequeued_node`.
3. Advance `front` to `front.next`.
4. If `front` is now `None` (queue became empty), set `rear` to `None` as well.
5. Decrement `size`.
6. Return `dequeued_node.data`.

Example 5.3.6 (`dequeue()` — remove from front).

```
1 def dequeue(self):
2     if self.isEmpty():
3         raise IndexError("Dequeue from empty queue")
4     dequeued_node = self.front      # Step 2
5     self.front    = self.front.next # Step 3
6     if self.front is None:         # Step 4
7         self.rear = None
8     self.size -= 1                 # Step 5
9     return dequeued_node.data      # Step 6
```

Warning: Forgetting to Reset rear on Last Dequeue

When the last element is dequeued, `self.front` becomes `None`, but `self.rear` still points to the old node. Subsequent `enqueue` calls would then corrupt the list. Always reset `self.rear = None` whenever `self.front` becomes `None` after a dequeue.

Remark 5.3.7. After `dequeue` returns, `dequeued_node` goes out of scope. If neither `front` nor `rear` points to that node any longer, Python's garbage collector reclaims its memory automatically. Even though the removed node's `next` field still points to the new front, that does not prevent collection — no live root references remain to the node itself.

5.3.5 isEmpty() and getSize()

Example 5.3.8 (Utility methods).

```
1 def isEmpty(self):
2     return self.front is None
3
4 def getSize(self):
5     return self.size
```

5.3.6 Working Example 01 — Full Queue via Direct Linked List

Example 5.3.9 (Queue working example 01). The following driver code enqueues three integers, inspects the state, dequeues one element, then inspects again.

```
1 if __name__ == "__main__":
2     queue = Queue()
3     queue.enqueue(10)
4     queue.enqueue(20)
```



```

5 queue.enqueue(30)
6 # State: front -> 10 -> 20 -> 30 <- rear
7 print("Front before dequeue:", queue.getFront()) # 10
8 print("Size before dequeue:", queue.getSize()) # 3
9 print("Dequeued element :", queue.dequeue()) # 10
10 # State: front -> 20 -> 30 <- rear
11 print("Front after dequeue:", queue.getFront()) # 20
12 print("Size after dequeue:", queue.getSize()) # 2

```

5.4 Implementation via Linked List (Version 2)

Version 2 delegates all node management to a reusable `LinkedList` class, keeping the `Queue` class thin. The `LinkedList` exposes `insert_node(index, value)` and `remove_node(index)`, which the queue operations call at the appropriate positions.

5.4.1 Class Structure

Example 5.4.1 (Queue Version 2 — delegating to `LinkedList`). `LinkedList` tracks `head`, `tail`, and `size`; `Queue` holds a single `LinkedList` instance.

```

1 class ListNode:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class LinkedList:
7     def __init__(self):
8         self.size = 0
9         self.head = None
10        self.tail = None
11
12 class Queue:
13     def __init__(self):
14         self.ll = LinkedList()

```

5.4.2 Adapted Core Operations

The queue methods change only in how they interact with the underlying list. The table below compares Version 1 and Version 2 side-by-side.

Method	Version 1	Version 2
<code>getFront</code>	<code>return self.front.data</code>	<code>return self.ll.head.data</code>
<code>enqueue</code>	manual node wiring at tail	<code>self.ll.insert_node(self.ll.size, data)</code>
<code>dequeue</code>	manual front removal	<code>self.ll.remove_node(0)</code>
<code>isEmpty</code>	<code>self.front is None</code>	<code>self.ll.size == 0</code>
<code>getSize</code>	<code>return self.size</code>	<code>return self.ll.size</code>

Example 5.4.2 (Queue Version 2 — full class).

```

1 class Queue:
2     def __init__(self):
3         self.ll = LinkedList()
4
5     def enqueue(self, data):
6         # append to tail: O(1) with tail pointer

```

```

7         self.ll.insert_node(self.ll.size, data)
8
9     def dequeue(self):
10         if self.isEmpty():
11             raise IndexError("Dequeue from empty queue")
12         data = self.ll.head.data
13         self.ll.remove_node(0)
14         return data
15
16     def getFront(self):
17         if self.isEmpty():
18             raise IndexError("Peek from empty queue")
19         return self.ll.head.data
20
21     def getSize(self):
22         return self.ll.size
23
24     def isEmpty(self):
25         return self.ll.size == 0

```

5.5 Combined Stack–Queue Example: Reversing a Queue

A classic interview problem illustrates how stacks and queues complement each other: given a queue, reverse its elements *in place* using only standard stack and queue operations.

Algorithm.

1. Dequeue every element from the queue and push it onto a stack. After this loop the stack top holds the element that was at the *rear* of the original queue.
2. Pop every element from the stack and enqueue it back into the queue. Because a stack reverses insertion order, the queue now holds the elements in reversed order.

Example 5.5.1 (`reverse_queue()` using a Stack).

```

1  def reverse_queue(queue):
2      stack = Stack()
3
4      # Phase 1: drain queue into stack (reverses order)
5      while not queue.isEmpty():
6          stack.push(queue.dequeue())
7
8      # Phase 2: drain stack back into queue
9      while not stack.is_empty():
10         queue.enqueue(stack.pop())
11
12     return queue
13
14 # Driver
15 if __name__ == "__main__":
16     q = Queue()
17     q.enqueue(10); q.enqueue(20); q.enqueue(30)
18     q.printQueue()      # 10 20 30
19     q = reverse_queue(q)
20     q.printQueue()      # 30 20 10

```

Key Result: Stack as a Reversal Tool

Transferring elements from a queue to a stack and back reverses their order in $O(n)$ time and $O(n)$ extra space. This exploits the LIFO property of the stack to undo the FIFO ordering of the queue.

5.6 Applications of Queues

5.6.1 Operating Systems

Queues are pervasive in system software:

- **CPU scheduling:** Ready processes are held in a run queue; the scheduler dispatches them in arrival order (FIFO) or according to a priority variant.
- **Print spooling:** Print jobs are queued so that multiple users can submit work without blocking the sender.
- **Network packet buffering:** Incoming packets are held in a queue at each router interface; the link layer drains the queue at the line rate.

5.6.2 Graph Algorithms

Breadth-First Search (BFS) uses a queue to explore a graph level by level. Starting from a source vertex, all immediate neighbours are enqueued before any of their successors; this guarantees that vertices are visited in non-decreasing order of their distance from the source.

5.6.3 Programming and Simulation

- **Event-driven simulation:** GUI toolkits maintain an event queue; mouse clicks and key presses are processed in the order they arrive.
- **Producer–consumer pipelines:** A shared queue decouples a fast producer from a slower consumer, providing a natural buffer.
- **Streaming data buffering:** Audio and video decoders buffer compressed frames in a queue, smoothing out network jitter before playback.

Exam Tip: Recognising Queue Problems

A problem calls for a queue when: (1) elements must be processed in the *same* order they arrive (FIFO), (2) you need level-by-level (BFS-style) exploration of a structure, or (3) a buffer is required between a producer and a consumer operating at different speeds. Contrast with stacks, which are appropriate when the most recently added item must be processed first (LIFO).

Chapter 6

Priority Queues and Stack/Queue Applications

6.1 Priority Queue

Definition 6.1.1 (Priority Queue). A **priority queue** is an abstract data type similar to a regular queue or stack, in which each element additionally has a *priority* associated with it. An element with higher priority is served before an element with lower priority. Formally, a priority queue ADT supports the operations **Insert**, **Delete-Min** (or **Delete-Max**), and **Peek** (find-min or find-max).

Unlike a standard queue where the first-in-first-out (FIFO) rule governs removal, a priority queue removes elements *on the basis of priority*: the element with the highest priority is dequeued first, regardless of insertion order.

6.1.1 Types of Priority Queue

There are two orderings in which priorities can be assigned:

- **Ascending-order priority queue** — gives the highest priority to the *smallest* value. For example, given the values {20, 40, 15, 50, 8, 7}, the queue stores them as 7, 8, 15, 20, 40, 50; element 7 has highest priority and is dequeued first, while 50 has lowest priority.
- **Descending-order priority queue** — gives the highest priority to the *largest* value. The same input is stored as 50, 40, 20, 15, 8, 7; element 50 has highest priority and 7 has lowest priority.

6.1.2 Characteristics of a Priority Queue

A queue qualifies as a priority queue if it satisfies all three of the following:

1. Each item has a priority value associated with it.
2. The item with the highest priority is moved to the front and processed (deleted) first.
3. If two elements share the same priority value, the priority queue falls back to the FIFO principle for the dequeue operation.

6.1.3 Implementation Strategies

A priority queue can be implemented using several underlying data structures:

- **Arrays** — either unordered (fast insert, slow delete) or ordered (slow insert, fast delete).
- **Linked list** — the approach covered in this chapter; items are inserted in sorted order so that the front always holds the highest-priority element.

- **Binary heap** — min-heap or max-heap; $O(\log n)$ insert and delete.
- **Binary search tree** — supports all operations in $O(\log n)$ on average.

6.2 Implementing a Priority Queue via Linked List

In the linked-list implementation the list is kept in *sorted order* at all times, so the head node always holds the element with the highest priority. Lower priority numbers are treated as higher priority (ascending-order queue).

6.2.1 Node and Queue Classes

Example 6.2.1 (PriorityNode and PriorityQueue skeleton). Each node stores both the data value and a numeric priority. The queue tracks the head of the sorted list and the current size.

```

1 class PriorityNode:
2     def __init__(self, data, priority):
3         self.data = data
4         self.priority = priority
5         self.next = None
6
7 class PriorityQueue:
8     def __init__(self):
9         self.head = None
10        self.size = 0

```

6.2.2 Core Operations

The five core operations mirror those of a standard queue, extended with priority awareness.

Operation	Description
peek()	Inspect the highest-priority item without removing it.
enqueue()	Insert an item at the correct position based on its priority.
dequeue()	Remove and return the item with the highest priority (head).
isEmpty()	Return <code>True</code> if the queue contains no items.
getSize()	Return the current number of items in the queue.

6.2.2.1 peek()

Example 6.2.2 (peek() — inspect the front without removal). Because the list is sorted, the highest-priority element always resides at the head.

```

1 def peek(self):
2     if self.isEmpty():
3         raise IndexError("Queue is empty")
4     return self.head.data

```

6.2.2.2 enqueue()

Insertion maintains the sorted invariant by traversing the list to find the correct position:

- If the queue is empty, or the new node's priority is *strictly lower* than the head's priority, prepend the new node (it becomes the new head).

- Otherwise, walk forward while `current.next.priority ≤ new priority`, then splice the new node between `current` and `current.next`.

The condition `<=` (not `<`) in the while loop ensures that among nodes of equal priority, the newly inserted node is placed *after* existing nodes of the same priority, preserving FIFO within a priority level.

Example 6.2.3 (`enqueue()` — sorted insertion).

```

1 def enqueue(self, data, priority):
2     new_node = PriorityNode(data, priority)
3     # Case 1: empty queue or new node has highest priority
4     if self.isEmpty() or priority < self.head.priority:
5         new_node.next = self.head
6         self.head = new_node
7     else:
8         # Case 2: traverse to find insertion point
9         current = self.head
10        while current.next and current.next.priority <= priority:
11            current = current.next
12        # Splice new_node between current and current.next
13        new_node.next = current.next
14        current.next = new_node
15    self.size += 1

```

Example 6.2.4 (Enqueue trace — inserting priority-4 node into existing list). Existing list: P=2:40 → P=4:50 → P=5:60. Insert data=30, priority=4.

1. `new_node = PriorityNode(30, 4)`.
2. `priority (4) < head.priority (2)` is False $\Rightarrow$ go to Case 2.
3. `current = head` (node P=2:40).
4. While loop: `current.next.priority (4) <= 4` is True $\Rightarrow$ `current = current.next` (node P=4:50).
5. While loop: `current.next.priority (5) <= 4` is False $\Rightarrow$ exit.
6. Splice: `new_node.next = P=5:60`; `current.next = new_node`.
7. Result: P=2:40 → P=4:50 → P=4:30 → P=5:60.

Both P=4 nodes are retained in FIFO order (50 before 30, since 50 was inserted first).

Warning: `<=` vs. `<` in the while condition

Using `<` instead of `<=` in the while loop would insert the new node *before* existing nodes of the same priority, violating the FIFO requirement for equal-priority elements. Always use `<=` to maintain stability.

6.2.2.3 `dequeue()`

Example 6.2.5 (`dequeue()` — remove highest-priority element). The head is always the highest-priority node, so removal is $O(1)$.

```

1 def dequeue(self):
2     if self.isEmpty():
3         raise IndexError("Queue is empty")
4     removed_data = self.head.data
5     self.head = self.head.next
6     self.size -= 1
7     return removed_data

```

6.2.2.4 isEmpty() and getSize()

Example 6.2.6 (Utility methods).

```

1 def isEmpty(self):
2     return self.head is None
3
4 def getSize(self):
5     return self.size

```

Key Result: Priority Queue via Sorted Linked List

Maintaining the list in sorted order gives $O(1)$ dequeue and peek, but $O(n)$ enqueue (worst case, traversal to tail). For applications requiring fast insertion, a binary heap is preferred ($O(\log n)$ both ways).

6.3 Applications of Priority Queues

Priority queues appear throughout computer science:

- **Graph algorithms** — Dijkstra's shortest-path algorithm and Prim's minimum spanning tree algorithm both use a priority queue to select the next vertex with the minimum tentative distance or edge weight.
- **Heuristic search** — the A* search algorithm uses a priority queue ordered by $f = g + h$ (cost so far plus heuristic estimate).
- **Heap sort** — building a max-heap and repeatedly extracting the maximum produces a sorted sequence in $O(n \log n)$.
- **Operating systems** — load balancing and interrupt handling assign priorities to tasks; the scheduler always runs the highest-priority ready task.
- **Data compression** — Huffman coding builds an optimal prefix tree by repeatedly merging the two lowest-frequency symbols, tracked via a min-heap.
- **Traffic management** — traffic light controllers grant priority to directions with heavier vehicle flow.

6.4 Stack Applications

6.4.1 Balanced Parentheses Checking

A fundamental stack application is verifying that brackets in an expression are properly nested and matched. Simple counting of opening and closing brackets is insufficient because it cannot detect mismatches such as ([)].

Algorithm. Scan the expression left to right:

1. If the current character is an opening bracket ((, [, or {), push it onto the stack.
2. If the current character is a closing bracket (),], or }), pop the stack and verify that the popped bracket is the matching opener. If the stack is empty or the brackets do not match, the expression is unbalanced.
3. After processing all characters, the expression is balanced if and only if the stack is empty.

Example 6.4.1 (Balanced parentheses trace). Expression: (([])).

Step through left to right: push (, push (, push [, pop [(matches)], pop ((matches)), pop ((matches)). Stack is empty at end $\Rightarrow$ **Balanced**.

Counter-example ([] ({ () } [())]) is unbalanced because when) is encountered the top of the stack is [, which does not match.

Exam Tip: Stack Invariant for Bracket Checking

At any point during the scan, the stack holds exactly the opening brackets that have been opened but not yet closed, in the order they were encountered. When the stack is empty and a closing bracket arrives, the expression is immediately unbalanced (nothing to match).

6.5 Algebraic Expression Notation

Arithmetic expressions can be written in three equivalent notations that differ in where the operator is placed relative to its operands.

6.5.1 Infix Notation

Definition 6.5.1 (Infix Notation). In **infix** notation the operator is placed *between* its two operands, as in $A + B$ or $A * (B + C) / D$. Infix is the standard human-readable form, but requires knowledge of operator precedence, associativity, and parentheses to parse unambiguously.

The operator precedence table used in this course (from highest to lowest) is:

Operators	Precedence
*, /, %	Highest
+, -	
<<, >>	
&&	
=	Lowest

Although infix is natural for humans, computers must do extra work to resolve precedence and parentheses before evaluating; prefix and postfix notations eliminate this ambiguity entirely.

6.5.2 Postfix (Reverse Polish) Notation

Definition 6.5.2 (Postfix Notation). In **postfix** (Reverse Polish Notation, RPN) the operator is placed *after* its operands. The expression $A + B$ in infix becomes $AB+$ in postfix. Postfix does not require precedence rules or parentheses; operators are applied in left-to-right order as they are encountered.

For example, the infix expression $AB + C *$ (postfix) evaluates addition *before* multiplication because $+$ appears first.

6.5.3 Prefix (Polish) Notation

Definition 6.5.3 (Prefix Notation). In **prefix** (Polish) notation the operator is placed *before* its operands. $A + B$ becomes $+AB$; $(A + B) * C$ becomes $*+ABC$. Like postfix, prefix eliminates the need for parentheses and precedence rules.

Key Result: Notation Comparison

Property	Infix	Prefix	Postfix
Operator position	between	before	after
Needs precedence rules	Yes	No	No
Needs parentheses	Yes	No	No
Evaluation direction	varies	right-to-left	left-to-right
Easy for computers	No	Yes	Yes

6.6 Converting Between Notations Using a Stack

6.6.1 Infix to Postfix Conversion

The standard shunting-yard algorithm converts infix to postfix in a single left-to-right pass using an operator stack.

Algorithm (Infix $\rightarrow$ Postfix). For each token in the infix expression:

1. **Operand** — write it directly to the output.
2. **(** — push it onto the stack.
3. **)** — pop and output operators until **(** is popped; discard both parentheses (do not add them to output).
4. **Operator X** — while the stack is non-empty and the top operator has *equal or higher* precedence than X , pop and output the top operator. Then push X .
5. **End of input** — pop and output all remaining operators from the stack.

Example 6.6.1 (Infix to Postfix: $1 + 2 * 3 + 4$).

Token	Action	Output so far	Stack
1	Operand — output	1	(empty)
+	Push	1	+
2	Operand — output	1 2	+
*	Push (* ; +)	1 2	+ *
3	Operand — output	1 2 3	+ *
+	Pop *, pop +, push +	1 2 3 * +	+
4	Operand — output	1 2 3 * + 4	+
END	Pop +	1 2 3 * + 4 +	(empty)

Result: postfix $1\ 2\ 3\ *\ +\ 4\ +$.

Example 6.6.2 (Infix to Postfix: $(7 - ((3 * 4) + 8)/5)$).

No.	Token	Stack	Postfix output
1	(	(	
2	7	(	7
3	-	(-	7
4	(	(-(	7
5	(	(-((	7
6	3	(-((	7 3
7	*	(-((*	7 3
8	4	(-((*	7 3 4
9	)	(-(	7 3 4 *
10	+	(-(+	7 3 4 *
11	8	(-(+	7 3 4 * 8
12	)	(-	7 3 4 * 8 +
13	/	(-/	7 3 4 * 8 +
14	5	(-/	7 3 4 * 8 + 5
15	)	(empty)	7 3 4 * 8 + 5 / -

Result: postfix 7 3 4 * 8 + 5 / -.

6.6.2 Evaluating a Postfix Expression

Once an expression is in postfix form, evaluation uses a single operand stack in a single left-to-right pass.

Algorithm (Evaluate Postfix).

1. Create an empty stack S .
2. For each token c from left to right:
 - If c is an **operand**: push $\text{int}(c)$ onto S .
 - If c is an **operator**: pop **operand1** (top), then pop **operand2** (new top). Compute **operand2 op operand1** and push the result. (Note the order: second-popped is the *left* operand.)
3. Pop the single remaining value from S — this is the result.

Warning: Operand Ordering When Evaluating

When an operator is encountered, the *first* value popped is **operand1** (right operand) and the *second* is **operand2** (left operand). The expression is therefore evaluated as **operand2** **<op>** **operand1**. Swapping this order gives wrong results for non-commutative operators such as $-$ and $/$.

Example 6.6.3 (Evaluate postfix 1 2 3 * + 4 +).

Token	Stack after action
1	[1]
2	[1, 2]
3	[1, 2, 3]
*	[1, 6] $(2 * 3 = 6)$
+	[7] $(1 + 6 = 7)$
4	[7, 4]
+	[11] $(7 + 4 = 11)$
Result	11

Matches the expected value of $1 + 2 \times 3 + 4 = 11$.

Example 6.6.4 (Evaluate postfix $7\ 3\ 4\ *\ 8\ +\ 5\ /\ -$).

Token	Stack after action
7	[7]
3	[7, 3]
4	[7, 3, 4]
*	[7, 12] ($3 * 4 = 12$)
8	[7, 12, 8]
+	[7, 20] ($12 + 8 = 20$)
5	[7, 20, 5]
/	[7, 4] ($20/5 = 4$)
-	[3] ($7 - 4 = 3$)
Result	3

Matches $7 - ((3 \times 4) + 8)/5 = 7 - 20/5 = 7 - 4 = 3$.

6.6.3 Prefix to Infix Conversion

Converting a prefix expression to infix uses a stack, but the scan proceeds *right to left*.

Algorithm (Prefix $\rightarrow$ Infix).

1. Read the prefix expression from **right to left**.
2. If the current token is an **operand**, push it onto the stack.
3. If the current token is an **operator**, pop two operands: let **op1** = first pop and **op2** = second pop. Concatenate as (**op1 operator op2**) and push the resulting string back.
4. The final string remaining in the stack is the infix expression.

Example 6.6.5 (Prefix to Infix: $=\ y\ \&\&\ \<\<\ a\ b\ \>\>\ c\ +\ d\ e$). Scanning right to left: **e**, **d**, **+**, **c**, **>>**, **b**, **a**, **<<**, **&&**, **y**, **=**.

1. Push **e**.
2. Push **d**.
3. **+**: pop **d**, pop **e** $\Rightarrow$ push **d + e**.
4. Push **c**.
5. **>>**: pop **c**, pop **d+e** $\Rightarrow$ push **c >> d + e**.
6. Push **b**.
7. Push **a**.
8. **<<**: pop **a**, pop **b** $\Rightarrow$ push **a << b**.
9. **&&**: pop **a<<b**, pop **c>>d+e** $\Rightarrow$ push **a << b && c >> d + e**.
10. Push **y**.
11. **=**: pop **y**, pop the **&&**-expression $\Rightarrow$ push **y = a << b && c >> d + e**.

Result: **y = a << b && c >> d + e**.

6.7 Queue Applications: Palindromes and Scheduling

6.7.1 Palindrome Detection

A **palindrome** is a string that reads the same forwards and backwards (e.g. `racecar`). A classic approach exploits both a stack and a queue simultaneously:

1. Enqueue each character of the string into a queue and push it onto a stack.
2. Dequeue from the queue (gives the original order) and pop from the stack (gives the reversed order) simultaneously.
3. If every pair of characters matches, the string is a palindrome.

This works because the queue preserves insertion order while the stack reverses it.

6.7.2 Scheduling Applications

Queues are the foundation of scheduling in many system contexts:

- **Multitasking** — the OS maintains a run queue of ready processes; round-robin scheduling dequeues the front process, runs it for a time slice, then re-enqueues it.
- **Network packet scheduling** — routers buffer packets in FIFO queues at each interface; priority queues implement differentiated services (QoS) by assigning urgency levels to traffic classes.
- **Job scheduling** — batch systems (e.g. HPC clusters) queue submitted jobs and dispatch them when resources become available.
- **Mailbox / message queues** — inter-process communication systems use queues so that the sender and receiver do not need to be active simultaneously.

Exam Tip: Choosing Between Stack, Queue, and Priority Queue

- Use a **stack** (LIFO) when the most recently added item must be processed first: balanced brackets, undo/redo, DFS, recursive-to-iterative conversion.
- Use a **queue** (FIFO) when elements must be processed in arrival order: BFS, print spooling, producer–consumer buffers.
- Use a **priority queue** when elements have varying urgency and the most important element must always be served next: Dijkstra, A*, OS interrupt handling, Huffman coding.

Chapter 7

Binary Trees

7.1 From Linear to Non-Linear Data Structures

The data structures examined in previous chapters — linked lists, stacks, queues, and priority queues — are all *linear*: every element has at most one predecessor and one successor, so the collection can be laid out in a straight line. A **binary tree** is one of the simplest *non-linear* data structures. Rather than a chain, it imposes a *hierarchical* parent-child relationship among its nodes.

7.2 Tree Terminology

Definition 7.2.1 (Tree). A **tree** T is a set of **nodes** storing elements such that the nodes have a *parent-child* relationship satisfying:

- If T is non-empty, it has exactly one distinguished node called the **root**, which has no parent.
- Every node $v \neq \text{root}$ has a unique **parent** node w ; every node whose parent is w is a **child** of w .

The following terms describe relationships and parts of a tree.

- **Root** — the unique top-level node; the only node with no parent.
- **Edge** — a directed link from a parent node to a child node.
- **Leaf (external node)** — a node with no children.
- **Internal node (non-leaf)** — any node that has at least one child.
- **Siblings** — nodes that share the same parent.
- **Ancestor** — any node on the path from a given node up to (and including) the root; i.e. the parent, grandparent, etc.
- **Descendant** — any node reachable by following child links downward from a given node; i.e. children, grandchildren, etc.
- **Subtree** — a node together with all of its descendants.
- **Height of a tree** — the number of edges on the longest path from the root to any leaf.
- **Level / Depth** — the root is at level 0 (or level 1 in some conventions); each child is one level below its parent.

Example 7.2.2 (Tree Vocabulary in Action). Consider the tree with root A , whose children are B, C, D, E . Node B has children F, G, H ; node H has children L, M ; node D has child I ; node E has children J, K ; node J has child N .

- A is the **root**; it is an **ancestor** of every other node.
- B, C, D, E are **siblings** and **children** of A .
- F, G, H, L, M are **descendants** of B .
- The **subtree** rooted at H contains $\{H, L, M\}$.
- L, M, F, G, C, I, K, N are **leaves** (no children).

7.3 Binary Trees

Definition 7.3.1 (Binary Tree). A **binary tree** is a tree in which every node has *at most two children*, conventionally called the **left child** and the **right child**.

Each node in a binary tree stores three pieces of information: its data value, a reference to its left subtree (or `None` if absent), and a reference to its right subtree (or `None` if absent).

Example 7.3.2 (Binary Tree Node Class in Python). The standard Python implementation mirrors the linked-list node pattern from Chapter 2:

```

1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.left = None # left child (or None)
5         self.right = None # right child (or None)

```

7.3.1 Operations on a Binary Tree

The principal operations on a binary tree are:

- **Insert** a node.
- **Delete** a node.
- **Search** for a node.
- **Traverse** — visit every node exactly once in a defined order.

Traversal is covered in detail in Section 7.4.

7.4 Binary Tree Traversal

A **traversal** visits every node in the tree exactly once. Traversals differ in the *order* in which a node's data is processed relative to its subtrees. The two major families are:

- **Depth-First Traversal (DFS)** — explore as deep as possible along each branch before backtracking. Three sub-orders exist: pre-order, in-order, and post-order.
- **Breadth-First Traversal (BFS)** — visit all nodes level by level, from top to bottom. Also called *level-order traversal*.

7.4.1 Pre-order Traversal

Definition 7.4.1 (Pre-order). 1. **Visit** (process) the current node.

2. Recursively traverse the **left** subtree.
3. Recursively traverse the **right** subtree.

Example 7.4.2 (Pre-order Code and Output). For the binary tree with root *E*, left subtree rooted at *B* (children *A* and *C*; *C* has child *D*), and right subtree rooted at *G* (children *F* and *I*; *I* has child *H*):

```
1 def pre_order_traversal(self, node):
2     if node:
3         print(node.data, end=" ")
4         self.pre_order_traversal(node.left)
5         self.pre_order_traversal(node.right)
```

Output: E B A C D G F I H

Key Result: Pre-order Mnemonic

Pre-order: **Root** → Left → Right (“*process before going down*”). The root is always the *first* element printed.

7.4.2 In-order Traversal

Definition 7.4.3 (In-order). 1. Recursively traverse the **left** subtree.

2. **Visit** (process) the current node.
3. Recursively traverse the **right** subtree.

Example 7.4.4 (In-order Code and Output). Using the same tree as Example 7.4.2:

```
1 def in_order_traversal(self, node):
2     if node:
3         self.in_order_traversal(node.left)
4         print(node.data, end=" ")
5         self.in_order_traversal(node.right)
```

Output: A B C D E F G H I

Exam Tip: In-order on a BST

When applied to a *Binary Search Tree* (BST), in-order traversal produces the node values in **sorted (ascending) order**. This is one of the most commonly tested properties.

7.4.3 Post-order Traversal

Definition 7.4.5 (Post-order). 1. Recursively traverse the **left** subtree.

2. Recursively traverse the **right** subtree.
3. **Visit** (process) the current node.

Example 7.4.6 (Post-order Code and Output). Using the same tree as Example 7.4.2:

```

1 def post_order_traversal(self, node):
2     if node:
3         self.post_order_traversal(node.left)
4         self.post_order_traversal(node.right)
5         print(node.data, end=" ")

```

Output: A D C B F H I G E

Key Result: Post-order Mnemonic

Post-order: Left $\rightarrow$ Right $\rightarrow$ **Root** (“process after both children return”). The root is always the *last* element printed. Post-order is natural for deletion and for evaluating expression trees.

7.4.4 Summary: Depth-First Traversal Orders

Traversal	Order	Root position	Output (example tree)
Pre-order	Root $\rightarrow$ L $\rightarrow$ R	First	E B A C D G F I H
In-order	L $\rightarrow$ Root $\rightarrow$ R	Middle	A B C D E F G H I
Post-order	L $\rightarrow$ R $\rightarrow$ Root	Last	A D C B F H I G E

7.4.5 Breadth-First (Level-Order) Traversal

Depth-first traversal uses recursion (implicitly a call-stack). Level-order traversal instead uses an explicit **queue** (recall Chapter 5) to process all nodes at each depth before descending.

Definition 7.4.7 (Level-Order / BFS Algorithm). 1. Enqueue the root node.

2. While the queue is non-empty:

- Dequeue the front node; visit it.
- Enqueue its left child (if it exists).
- Enqueue its right child (if it exists).

Example 7.4.8 (Level-Order Code). The implementation reuses the **Queue** class from Chapter 5:

```

1 def level_order_traversal(self):
2     if not self.root:
3         return
4     queue = Queue()
5     queue.enqueue(self.root)
6     while not queue.is_empty():
7         current = queue.dequeue()
8         print(current.data, end=" ")
9         if current.left:
10            queue.enqueue(current.left)
11        if current.right:
12            queue.enqueue(current.right)

```

For the tree in Example 7.4.2, the output is: E B G A C F I D H (level by level, left to right).

7.5 Applications of Tree Traversal

7.5.1 Counting Nodes (Size)

The size of a tree satisfies a simple recursive relation:

$$\text{size}(T) = \begin{cases} 0 & \text{if } T \text{ is empty (None)} \\ 1 + \text{size}(\text{left subtree}) + \text{size}(\text{right subtree}) & \text{otherwise} \end{cases}$$

Example 7.5.1 (Count Nodes).

```
1 def count_nodes(self, node):
2     if not node:
3         return 0
4     return 1 + self.count_nodes(node.left) \
5         + self.count_nodes(node.right)
```

This is a post-order computation: each node returns the count of its subtree to its parent.

7.5.2 Finding k -th Level Descendants

Given a node X and an integer $k \geq 0$, return all nodes exactly k levels below X . The key insight is to *decrement* k at each recursive call and collect nodes when $k = 0$.

Example 7.5.2 (k -th Level Descendants).

```
1 def kth_level_descendants(self, node, k):
2     if node is None:
3         return []
4     if k == 0:
5         # This node is exactly k levels below the start
6         return [node.data]
7     left = self.kth_level_descendants(node.left, k - 1)
8     right = self.kth_level_descendants(node.right, k - 1)
9     return left + right
```

Usage: `bt.kth_level_descendants(bt.root, 2)` returns all grandchildren of the root.

7.5.3 Calculating the Height of a Node

Definition 7.5.3 (Height of a Node). The **height** of a node is the number of edges on the longest downward path from that node to a leaf. By convention:

- A **leaf** node has height 0.
- An **empty** (None) node returns -1 as a sentinel value.

Therefore: $\text{height}(v) = 1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$.

Example 7.5.4 (Calculate Height).

```
1 def calculate_height(self, node):
2     if node is None:
3         return -1 # empty sentinel
4     left_h = self.calculate_height(node.left)
5     right_h = self.calculate_height(node.right)
6     return 1 + max(left_h, right_h)
```

Key Result: Height Recurrence

$$\begin{array}{lclclclclcl} \text{height}(\text{None}) & = & -1, & \text{height}(\text{leaf}) & = & 0, & \text{height}(v) & = & 1 + \\ & & & \text{max}(\text{height}(v.\text{left}), \text{height}(v.\text{right})). \end{array}$$

7.6 Pre-order Traversal with an Explicit Stack

Recursive DFS implicitly uses the call stack. It can be converted to an iterative form using an explicit stack (recall Chapter 4), which can be important for very deep trees where recursion depth limits apply.

Definition 7.6.1 (Iterative Pre-order Algorithm). 1. Push the root onto the stack.

2. While the stack is non-empty:

- (a) Pop the top node; **visit** it.
- (b) Push its *right* child (if it exists).
- (c) Push its *left* child (if it exists).

Note: the right child is pushed *before* the left so that the left child is processed first (LIFO order).

Example 7.6.2 (Iterative Pre-order on a Concrete Tree). Consider the tree: root 14; left child 84 (children 13, 16); right child 43 (children 33, 97); 13 has child 53; 16 has children 99, 72; 33 has child 64.

The iterative algorithm produces:

$$14 \rightarrow 84 \rightarrow 13 \rightarrow 53 \rightarrow 16 \rightarrow 99 \rightarrow 72 \rightarrow 43 \rightarrow 33 \rightarrow 64 \rightarrow 97$$

which is identical to recursive pre-order on the same tree.

Warning: Push Order Matters

When using an explicit stack for pre-order, always push the **right** child *before* the left child. Because a stack is LIFO, this ensures the left child is popped and visited first — matching the recursive left-before-right semantics. Reversing the push order produces a right-before-left pre-order traversal.

Exam Tip: Choosing a Traversal

- Use **pre-order** to copy or serialize a tree (root arrives before children).
- Use **in-order** to retrieve BST keys in sorted order.
- Use **post-order** to safely delete a tree (children freed before parent) or evaluate expression trees.
- Use **level-order** to find the shortest path in an unweighted tree or to print the tree level by level.

Chapter 8

Binary Search Trees

8.1 The BST Property

A **Binary Search Tree (BST)** is a binary tree in which every node satisfies a global ordering constraint, not merely a local one. Recall from Chapter 7 that a binary tree places no restriction on the relative values of nodes; a BST imposes a strict rule at every node simultaneously.

Definition 8.1.1 (Binary Search Tree). A binary tree T is a **Binary Search Tree** if and only if, for every node v :

1. The value of v is **greater than** every value in v 's left subtree.
2. The value of v is **less than** every value in v 's right subtree.
3. Both the left and right subtrees of v are themselves BSTs.

Duplicate values are not permitted.

The rule can be remembered as the $L < C < R$ invariant: at any node C , everything to its left (L) is smaller and everything to its right (R) is larger.

Warning: Local vs. Global Ordering

It is *not* sufficient to check only the immediate children of a node. For example, a node with value 31 in the left subtree of 70 must be less than 70, but its own right subtree must also be less than 70 even though 70 is not 31's direct parent. A tree that satisfies the parent-child comparison locally but violates the global ordering is **not** a BST.

8.2 BST Insertion

When inserting into an ordinary binary tree there is no prescribed position for new nodes. In a BST, however, the BST property uniquely determines where each new value must go.

The public method handles the empty-tree edge case; the recursive helper `_insert` descends to the correct leaf position.

Definition 8.2.1 (BST Insertion Algorithm). To insert a value `data` rooted at `current_node`:

1. If `data < current_node.data`: go left.
 - If the left child is `None`, attach a new node there.
 - Otherwise, recurse on the left child.
2. If `data > current_node.data`: go right (symmetric).
3. If `data = current_node.data`: ignore (no duplicates).

After insertion the tree remains a valid BST.

Example 8.2.2 (BST Insertion). The public and private methods together implement the algorithm above:

```
1 def insert(self, data):
2     if self.root is None:
3         self.root = Node(data)
4     else:
5         self._insert(data, self.root)
6
7 def _insert(self, data, current_node):
8     if data < current_node.data:
9         if current_node.left is None:
10            current_node.left = Node(data)
11        else:
12            self._insert(data, current_node.left)
13    else:
14        if current_node.right is None:
15            current_node.right = Node(data)
16        else:
17            self._insert(data, current_node.right)
```

Trace — inserting 'D' into the tree rooted at 'H':

1. 'D' < 'H' $\Rightarrow$ go left to 'E'.
2. 'D' < 'E' $\Rightarrow$ go left to 'B'.
3. 'D' > 'B' $\Rightarrow$ go right to 'C'.
4. 'D' > 'C', right child of 'C' is None $\Rightarrow$ attach 'D' as the right child of 'C'.

Exam Tip: Insertion Order Determines Tree Shape

Inserting the *same* set of values in a different order generally produces a *different* tree shape (and therefore different height and search cost). A sorted sequence inserted one by one produces a maximally skewed (linear) tree.

8.3 BST Searching

Searching a BST exploits the BST property to eliminate half the remaining candidates at each step — a **decrease-and-conquer** strategy. At each node, the search value is compared and the algorithm descends *only* the relevant subtree, never both.

Definition 8.3.1 (BST Search Algorithm). To search for `data` starting at `current_node`:

1. If `current_node` is None: return False (not found).
2. If `data == current_node.data`: return True.
3. If `data < current_node.data`: recurse on the left child.
4. Otherwise: recurse on the right child.

Example 8.3.2 (BST Search).

```
1 def _search(self, data, current_node):
2     if current_node is None:
3         return False
4     elif data == current_node.data:
5         return True
```

```

6     elif data < current_node.data:
7         return self._search(data, current_node.left)
8     else:
9         return self._search(data, current_node.right)

```

Searching for 31 in a tree rooted at 70: $70 \xrightarrow{31 < 70} 31 \Rightarrow$ found.

Searching for 100: $70 \xrightarrow{100 > 70} 93 \xrightarrow{100 > 93} 94 \xrightarrow{100 > 94} 99 \xrightarrow{100 > 99} \text{None} \Rightarrow$ not found.

Key Result: BST Search Complexity

In a *balanced* BST of n nodes, each comparison eliminates roughly half the remaining nodes, giving $O(\log n)$ worst-case search time. In a *skewed* BST (degenerate to a linked list), the worst case degrades to $O(n)$.

8.4 BST In-Order Traversal and Sorted Output

A particularly important property of BSTs connects them directly to sorting.

Theorem 8.4.1 (In-Order Traversal of a BST Produces a Sorted Sequence). *The in-order traversal (left $\rightarrow$ node $\rightarrow$ right) of any BST visits the nodes in non-decreasing order of their keys.*

Intuition. The $L < C < R$ invariant guarantees that, at every node, all left descendants have been visited (and are smaller) before the node itself is processed, and all right descendants (which are larger) are visited afterwards. This is precisely sorted order.

As a corollary, the minimum value of a BST is the **left-most** node, and the maximum value is the **right-most** node — both reachable in $O(h)$ time where h is the tree height.

8.5 BST Deletion

Deletion is the most complex of the four basic BST operations because removing a node may break the structural integrity of the tree, and the BST property must be restored. After deleting any node X , the result must still be a valid BST.

The procedure branches on the number of children X has.

8.5.1 Case 0: Leaf Node (No Children)

Definition 8.5.1 (Deletion — Case 0). If the node to delete X is a leaf, simply set the relevant child pointer of X 's parent to **None**. No rebalancing or successor logic is needed.

Example 8.5.2 (Deleting a Leaf). In the tree with root 27 and children 16 (with right child 17) and 33, deleting 17: the right-child pointer of 16 is set to **None**. All remaining nodes are still in BST order.

8.5.2 Case 1: One Child

Definition 8.5.3 (Deletion — Case 1). If the node to delete X has exactly one child Y , redirect X 's parent's child pointer directly to Y . This “splices out” X exactly as removing a node from a linked list. All keys remain in BST order because Y 's entire subtree was already in the correct position relative to X 's ancestors.

Example 8.5.4 (Deleting a Node with One Child). Deleting node 16 (which has one child, a subtree rooted at 20 with children 18 and 25) from a tree rooted at 27: the left-child pointer of 27 is updated to point to 20. The subtrees rooted at 18 and 25 move up one level but remain in correct BST order.

8.5.3 Case 2: Two Children

Deleting a node with two children is the most involved case. The key idea is to replace the deleted node's value with that of its **in-order successor** and then delete the successor, which has at most one child.

Definition 8.5.5 (In-Order Successor). The **in-order successor** of a node X in a BST is the node with the smallest key greater than X . It is found by moving to X 's right child and then following left-child pointers as far as possible:

1. Set pointer $S \leftarrow X.\text{right}$.
2. While $S.\text{left} \neq \text{None}$: set $S \leftarrow S.\text{left}$.
3. S is the successor.

The successor can have a *right* child but **cannot** have a left child (if it did, that left child would be an even smaller successor, contradicting the definition). Therefore the successor is always removable using Case 0 or Case 1.

Definition 8.5.6 (Deletion — Case 2 Algorithm). 1. Locate the in-order successor S of X using the algorithm above.

2. Copy S 's key into X 's node (overwriting X 's original key).
3. Recursively delete S from the right subtree (this reduces to Case 0 or 1).

The BST property is preserved because S is the smallest key larger than everything in X 's left subtree, so placing S at X 's position maintains $L < C < R$ at every ancestor.

Example 8.5.7 (Deleting a Two-Child Node). In the tree with root 44; left child 27 (children 16, 33); right child 65: delete node 27.

1. Find the in-order successor of 27: move to 33 (right child of 27), then follow left pointers to 28 — the successor is 28.
2. Copy 28 into the node currently holding 27.
3. Delete the original node 28 from the right subtree of (now) 28. Node 28 was a leaf (or had only a right child), so Case 0 or 1 applies.

Result: the tree rooted at 44; left child 28 (children 16, 33); right child 65 — a valid BST.

Key Result: Full Deletion Algorithm

1. Locate the node to delete and its parent.
2. **Case 0** (leaf): set parent's child pointer to **None**.
3. **Case 1** (one child): set parent's child pointer to the node's child.
4. **Case 2** (two children): copy the in-order successor's key in; delete the successor (reduces to Case 0 or 1).

The BST property is maintained after every case.

Exam Tip: Identifying the Successor Quickly

In a BST deletion question, the in-order successor of node X is always the *leftmost* node in X 's *right subtree*. If X has no right subtree, the alternative is to use the in-order *predecessor* — the rightmost node in the left subtree — which is equally valid.

Chapter 9

AVL Trees

9.1 Tree Terminology Review

Before studying AVL trees, it is useful to consolidate the terminology relating to tree structure and balance.

Definition 9.1.1 (Height of a Tree). The **height** of a tree is the number of *edges* on the longest path from the root to a leaf. A leaf node has height 0; an empty (**None**) node is treated as having height -1 .

Definition 9.1.2 (Depth / Level of a Node). The **depth** (or **level**) of a node is the number of edges from that node to the root. The root itself has depth 0.

Note the complementary relationship: height is measured *downwards* from a node, whereas depth is measured *upwards* to the root.

Definition 9.1.3 (Empty Binary Tree). An **empty binary tree** is a tree containing no nodes. It is still considered a valid (trivial) tree.

Definition 9.1.4 (Perfect Binary Tree). A **perfect binary tree** of height H has no missing nodes: all leaves are at depth H and every internal node has exactly two children. A perfect tree of height H contains exactly $2^{H+1} - 1$ nodes.

Definition 9.1.5 (Complete Binary Tree). A **complete binary tree** of height H is full at all levels 0 through $H - 1$, with level H filled from *left to right*. The number of nodes n satisfies

$$2^H \leq n \leq 2^{H+1} - 1, \quad \text{equivalently,} \quad H = \lfloor \log_2 n \rfloor.$$

This gives the **minimal height** of a complete tree with n nodes.

Key Result: Height Bound for Complete Trees

For a complete binary tree with n nodes and height H :

$$2^H \leq n < 2^{H+1} \implies H \leq \log_2 n < H + 1 \implies H = \lfloor \log_2 n \rfloor.$$

This is the *minimum possible height* for any binary tree with n nodes and is why balanced trees give $O(\log n)$ operations.

Definition 9.1.6 (Balanced Binary Tree). A **balanced binary tree** is one in which the heights of the left and right subtrees of *every* node differ by at most 1.

9.2 Tree Balancing and Motivation

In Chapter 8 we saw that BST search, insertion, and deletion all run in $O(h)$ time, where h is the height of the tree. In the best case — a balanced tree — $h = O(\log n)$. However, inserting

keys in sorted order produces a maximally *skewed* tree with $h = n - 1$, degrading all operations to $O(n)$.

A **most balanced BST** is one where every internal node has two children except possibly at the bottom two levels. Although optimal, such a tree is expensive to rebuild from scratch on every insertion. Two practical approaches exist.

Approach 1 — Sort and Reconstruct. Perform an in-order traversal to extract all keys in sorted order, store them in a list, then recursively build a balanced BST by always choosing the median element as the root.

Example 9.2.1 (Sort and Reconstruct). In-order traversal of the skewed tree with keys $\{7, 14, 23, 31, 70, 71, 73, 93, 94, 99\}$ yields the sorted list. Then:

```

1 def tree_balance(data, first, last):
2     if last >= first:
3         # middle index is rounded down
4         middle = (first + last) // 2
5         root = Node(data[middle])
6         root.left = tree_balance(data, first, middle - 1)
7         root.right = tree_balance(data, middle + 1, last)
8         return root
9     return None

```

The median element (index $(\text{first} + \text{last}) // 2$) becomes the root; the left half builds the left subtree and the right half builds the right subtree, recursively. The resulting tree has height $\lfloor \log_2 n \rfloor$.

The drawback of Approach 1 is that it requires $O(n)$ extra storage for the sorted list and rebuilds the *entire* tree after every structural change.

Approach 2 — AVL Tree (local rebalancing). The AVL tree, invented by Adelson-Velskii and Landis in 1962, maintains balance *locally*: after each insertion or deletion, only the nodes along the affected path are inspected, and at most a constant number of rotations restore the balance invariant.

9.3 AVL Trees and the Balance Factor

Definition 9.3.1 (AVL Tree). An **AVL tree** is a BST in which the heights of the left and right subtrees of every node differ by at most 1. This invariant is maintained after every insertion and deletion using local rotations.

Each AVL node stores an explicit **height** attribute, initialised to 0 at creation:

Example 9.3.2 (AVL Node Definition).

```

1 class AVLNode:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6         self.height = 0 # leaf height = 0; None height = -1

```

Definition 9.3.3 (Balance Factor). The **balance factor** of a node is:

$$\text{BF}(\text{node}) = \text{height}(\text{node.left}) - \text{height}(\text{node.right}),$$

where an empty (**None**) subtree has height -1 and a leaf has height 0. In a valid AVL tree, $\text{BF} \in \{-1, 0, 1\}$ for every node.

Example 9.3.4 (Computing Height and Balance Factor).

```
1 def _get_height(self, node):
2     # None node returns -1; leaf returns 0
3     if not node:
4         return -1
5     return node.height
6
7 def _get_balance(self, node):
8     if not node:
9         return 0
10    return self._get_height(node.left) - self._get_height(node.right)
```

The stored `node.height` equals $1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$ and must be updated whenever the subtree structure changes.

Warning: BF Outside $\{-1, 0, 1\}$ Signals an Imbalance

After any insertion, immediately recompute heights and balance factors along the insertion path back to the root. A balance factor of ± 2 at any ancestor signals that a rotation is required at that node before continuing upward.

9.4 AVL Insertion: Cases

Inserting into an AVL tree begins exactly like BST insertion. Afterwards, the algorithm walks back up the insertion path updating heights and balance factors. Three cases arise.

9.4.1 Case 1 — All Ancestors Have Balance Factor Zero

If every node along the path from the root to the newly inserted node had $\text{BF} = 0$ before the insertion, then after inserting the new node, all balance factors remain in $\{-1, 0, 1\}$. **No rotation is needed.**

Example 9.4.1 (Insertion Case 1). Inserting node 25 into the tree rooted at 70, where 25 becomes the right child of leaf 23. Node 23 had $\text{BF} = 0$; after insertion its BF becomes -1 . All ancestor balance factors shift by at most ± 1 and none exceed ± 1 . The tree remains a valid AVL tree with no rotations required.

9.4.2 Case 2 — Insertion into the Shorter Subtree of a Non-Zero BF Ancestor

If a non-zero BF ancestor exists but the new node is placed in that ancestor's *shorter* subtree, the ancestor's balance factor moves toward 0. **No rotation is needed.**

Example 9.4.2 (Insertion Case 2). Inserting node 80 into the tree where node 93 has $\text{BF} = -1$ (its right subtree, containing 99 and 97, is taller). Node 73 is the parent for 80 and belongs to the *shorter* left subtree of 93. After insertion, 93's BF becomes 0 — more balanced, not less — so no rotation is required.

9.4.3 Case 3 — Insertion into the Taller Subtree (Rotation Required)

If the new node is placed in the *taller* subtree of a non-zero BF ancestor, that ancestor's balance factor becomes ± 2 , violating the AVL invariant. A rotation (or double rotation) at the violated ancestor restores balance. Four sub-cases arise depending on the *direction* of the imbalance.

Key Result: Four AVL Rotation Cases

Let A be the deepest ancestor with $|\text{BF}| = 2$ after an insertion.

- **LL** ($\text{BF}(A) = +2$, $\text{BF}(A.\text{left}) \geq 0$): single *right rotation* at A .
- **RR** ($\text{BF}(A) = -2$, $\text{BF}(A.\text{right}) \leq 0$): single *left rotation* at A .
- **LR** ($\text{BF}(A) = +2$, $\text{BF}(A.\text{left}) < 0$): left rotation at $A.\text{left}$, then right rotation at A .
- **RL** ($\text{BF}(A) = -2$, $\text{BF}(A.\text{right}) > 0$): right rotation at $A.\text{right}$, then left rotation at A .

9.5 Rotation Operations

All four cases reduce to at most two applications of the two primitive operations: **right rotation** and **left rotation**. Both are $O(1)$ — they only reassign three pointers and update two heights.

9.5.1 Right Rotation (LL Case)

A right rotation at node `cur` promotes `cur`'s left child to take `cur`'s position, as illustrated below for the LL case (inserting 95 under 97, causing node 99 to have $\text{BF} = +2$):

Example 9.5.1 (Right Rotation — LL Case). Inserting 95 makes node 99 unbalanced ($\text{BF} = +2$, left-heavy). Right rotation about node 99:

```
1 def right_rotate(self, cur):
2     # cur = 99 (unbalanced node)
3     x = cur.left    # x = 97 (left child, becomes new root)
4     xRChild = x.right # xRChild = None (97 has no right child)
5
6     # Reassign pointers
7     cur.left = xRChild # 99.left = None
8     x.right = cur      # 97.right = 99
9
10    # Update heights bottom-up
11    cur.height = 1 + max(self._get_height(cur.left),
12                        self._get_height(cur.right))
13    x.height = 1 + max(self._get_height(x.left),
14                      self._get_height(x.right))
15    return x          # new subtree root is 97
```

After the rotation: node 97 becomes the right child of 93; node 99 becomes 97's right child; node 95 remains 97's left child. All balance factors in $\{-1, 0, 1\}$.

Exam Tip: Right Rotation Mnemonic

Think of a right rotation as “pulling the left child up”. The key pointer reassignments are: (1) give the left child's right subtree to `cur`'s left, (2) make `cur` the left child's right child, (3) update `cur`'s height before the new root's height. Always update **the lower node first** (`cur` before `x`).

9.5.2 Left Rotation (RR Case)

A left rotation at node `cur` is the mirror image: `cur`'s right child is promoted. For the RR case (inserting 12 makes node 7 have $\text{BF} = -2$, right-heavy):

Example 9.5.2 (Left Rotation — RR Case). Inserting 12 under node 10 makes node 7 unbalanced ($BF = -2$, right-heavy). Left rotation about node 7:

```

1 def left_rotate(self, cur):
2     # cur = 7 (unbalanced node)
3     x      = cur.right # x = 10 (right child, becomes new root)
4     xLChild = x.left   # xLChild = None (10 has no left child)
5
6     # Reassign pointers
7     cur.right = xLChild # 7.right = None
8     x.left   = cur     # 10.left = 7
9
10    # Update heights bottom-up
11    cur.height = 1 + max(self._get_height(cur.left),
12                        self._get_height(cur.right))
13    x.height   = 1 + max(self._get_height(x.left),
14                        self._get_height(x.right))
15    return x        # new subtree root is 10

```

After rotation: 10 becomes the left child of 14; 7 becomes 10's left child; 12 remains 10's right child.

9.5.3 Left-Right Double Rotation (LR Case)

When $BF(A) = +2$ but the imbalance is caused by the *right* subtree of A 's left child, a single right rotation does not suffice. A left rotation at A 's left child first converts it to the LL case, and then a right rotation at A completes the fix.

Example 9.5.3 (LR Double Rotation). Inserting 98 into the tree where 99 has $BF = +2$ and its left child 97 has $BF = -1$ (LR case):

1. **Left rotation about node 97:** node 98 rises to the position held by 97; 97 becomes 98's left child. Node 99 now has left child 98 with $BF = 0$ at 98, and $BF(99) = +2$ remains — but the sub-case is now LL.
2. **Right rotation about node 99:** node 98 rises further, taking 99's position; 99 becomes 98's right child; 97 is 98's left child. All balance factors return to $\{-1, 0, 1\}$.

9.5.4 Right-Left Double Rotation (RL Case)

When $BF(A) = -2$ but the imbalance is caused by the *left* subtree of A 's right child, a right rotation at A 's right child first converts it to the RR case, then a left rotation at A completes the fix.

Example 9.5.4 (RL Double Rotation). Inserting 8 into the tree where node 7 has $BF = -2$ and its right child 10 has $BF = +1$ (RL case):

1. **Right rotation about node 10:** node 8 rises, taking 10's place; 10 becomes 8's right child. Node 7 now has right child 8 — sub-case is now RR.
2. **Left rotation about node 7:** node 8 rises to 7's position; 7 becomes 8's left child; 10 is 8's right child. All balance factors are restored.

Exam Tip: Identifying the Rotation Case Quickly

Given an unbalanced node A ($|BF| = 2$):

1. $BF(A) > 0$: imbalance is *left-heavy*; look at A 's left child L .
 - $BF(L) \geq 0$ (LL): single right rotation at A .
 - $BF(L) < 0$ (LR): left rotate at L , then right rotate at A .
2. $BF(A) < 0$: imbalance is *right-heavy*; look at A 's right child R .
 - $BF(R) \leq 0$ (RR): single left rotation at A .
 - $BF(R) > 0$ (RL): right rotate at R , then left rotate at A .

9.6 AVL Insertion: Symmetric Structure of Rotations

An important conceptual observation is that the four cases are *symmetric*: LL and RR are mirrors of each other (single rotation each), and LR and RL are mirrors of each other (double rotation each). The generic structure is:

Imbalance	Rotation(s)	Complexity
LL	Right rotate at A	$O(1)$
RR	Left rotate at A	$O(1)$
LR	Left at A .left, then Right at A	$O(1)$
RL	Right at A .right, then Left at A	$O(1)$

In all four cases, after the rotation the subtree rooted at the former A has the *same height* as it did before the insertion. Therefore, only *one* round of rotations (at the deepest violated ancestor) is ever needed to restore the AVL property for the entire tree — balance propagates upward automatically.

Key Result: AVL Insertion Complexity

Each insertion into an AVL tree of n nodes requires:

- $O(\log n)$ time to locate the insertion position (tree height is $O(\log n)$).
- $O(\log n)$ time to walk back up and update heights.
- $O(1)$ rotations (at most two) to restore balance.

Total: $O(\log n)$ **per insertion.** This is optimal for comparison-based insertion into a sorted structure.

9.7 Other Self-Balancing BSTs

The AVL tree is the oldest self-balancing BST, but several alternatives have been developed, each trading different constants in time and space:

- **Red-Black Trees** — Used in the C++ STL and Java `TreeMap`. Guarantee $O(\log n)$ operations with slightly looser balance (height $\leq 2 \log n$) but fewer rotations on average than AVL.

- **Splay Trees** — Self-adjusting; recently accessed nodes bubble to the root. No stored height, amortised $O(\log n)$ per operation.
- **Scapegoat Trees** — Rebuild subtrees lazily when a “scapegoat” node is found to be too heavy; $O(\log n)$ amortised with no rotation bookkeeping.
- **B-Trees** — Generalise binary trees to k -way branching; used in databases and file systems where disk-block reads dominate.
- **Treaps** — Combine BST key ordering with heap priority (assigned randomly), achieving $O(\log n)$ expected operations without explicit rebalancing logic.

Remark 9.7.1. The key insight shared by all these structures is that *local restructuring* (rotation or split/merge) can restore a global property (balanced height) in $O(1)$ extra work per operation. The challenge in algorithm design is identifying which local invariant is cheapest to maintain while still guaranteeing $O(\log n)$ global height.

Chapter 10

Introduction to Algorithms

10.1 What Is an Algorithm?

The word *algorithm* first appeared in Webster’s New World Dictionary after 1957, though its roots are far older. It is derived from the name of the 9th-century Persian mathematician Abū Ja’far Muḥammad ibn Mūsā al-Khwārizmī, whose works on algebra and arithmetic were widely translated into Latin as *algorismus*. The Euclidean algorithm for finding the greatest common divisor of two integers dates back to Euclid’s *Elements* (ca. 300 B.C.) and remains one of the oldest documented algorithms still in use today.

Definition 10.1.1 (Algorithm (Levitin)). An algorithm is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.

Definition 10.1.2 (Algorithm (Cormen et al.)). An algorithm is any well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**.

10.1.1 Properties of a Valid Algorithm

Three essential properties distinguish a proper algorithm from an informal description:

- **Correctness:** The output must be correct and consistent for every legitimate input instance. An algorithm that only works on some inputs is not a correct algorithm.
- **Precision:** The algorithm must consist of a series of well-defined, systematic steps. Each step must be unambiguous — words such as “maybe”, “roughly”, or “about” are not acceptable in a precise specification.
- **Finiteness:** The algorithm must terminate after a finite number of steps for every valid input. An infinite loop does not constitute an algorithm.

Core Properties of an Algorithm

Every valid algorithm must satisfy:

1. **Correctness** — right answer for all valid inputs.
2. **Precision** — each step is unambiguous.
3. **Finiteness** — terminates in a finite number of steps.

10.2 Algorithm vs. Program

An algorithm and a computer program are related but distinct concepts.

Definition 10.2.1 (Computer Program). A computer program is an instance, or concrete representation, of an algorithm expressed in a specific programming language. **Implementation** is the task of translating an algorithm into a computer program.

The key relationship is:

$$\text{Problem} \longrightarrow \text{Algorithm} \longrightarrow \text{Program}$$

The same algorithm can therefore be implemented in many different languages without changing its essential logic. As an illustration, the iterative summation algorithm (Method One from Section 10.3) can be written identically in Java, C, or Python — the algorithm is language-independent; only the syntax of its implementation changes.

Exam Tip: Algorithm $\neq$ Program

Do not conflate the two. An algorithm is a mathematical specification of a computation; a program is a language-specific realisation. A single algorithm can produce many different but equivalent programs.

10.3 Example 1: Arithmetic Series

Computing $\sum_{i=1}^n i$ is a classic problem illustrating that multiple correct algorithms can exist for the same problem. Three distinct approaches are given below.

Method One — Iterative Accumulation. Initialise a running total to zero, then add each integer from 1 to n in a loop.

Algorithm 1 Summing Arithmetic Sequence (Iterative)

```

1: function METHODONE( $n$ )
2:    $sum \leftarrow 0$ 
3:   for  $i \leftarrow 1$  to  $n$  do
4:      $sum \leftarrow sum + i$ 
5:   end for
6:   return  $sum$ 
7: end function

```

Method Two — Closed-Form Formula. Use the well-known arithmetic series identity $\sum_{i=1}^n i = \frac{n(n+1)}{2}$, requiring only a single arithmetic operation regardless of n .

Algorithm 2 Summing Arithmetic Sequence (Formula)

```

1: function METHODTWO( $n$ )
2:    $sum \leftarrow n \times (1 + n) / 2$ 
3:   return  $sum$ 
4: end function

```

Method Three — Recursion. Express the sum as $n + \text{sum}(1, \dots, n - 1)$ with base case $n = 1$.

Algorithm 3 Summing Arithmetic Sequence (Recursive)

```
1: function METHODTHREE( $n$ )
2:   if  $n = 1$  then
3:     return 1
4:   else
5:     return  $n + \text{METHODTHREE}(n - 1)$ 
6:   end if
7: end function
```

Comparing the Three Methods

Method	Strategy	Time Complexity	Extra Space
Method One	Iterative loop	$O(n)$	$O(1)$
Method Two	Closed-form formula	$O(1)$	$O(1)$
Method Three	Recursion	$O(n)$	$O(n)$ (call stack)

Method Two is strictly superior for this problem. However, not every problem admits a closed-form solution, making the iterative and recursive approaches broadly important.

10.4 Example 2: Fibonacci Sequence

The Fibonacci sequence is $1, 1, 2, 3, 5, 8, 13, \dots$, defined by the recurrence

$$F(n) = F(n - 1) + F(n - 2), \quad F(1) = F(2) = 1.$$

Two natural implementations arise directly from this definition.

Algorithm 4 Fibonacci: Recursive

```
1: function FIBONACCIRECURSIVE( $n$ )
2:   if  $n < 1$  then
3:     return 0
4:   end if
5:   if  $n = 1$  or  $n = 2$  then
6:     return 1
7:   end if
8:   return FIBONACCIRECURSIVE( $n - 1$ ) + FIBONACCIRECURSIVE( $n - 2$ )
9: end function
```

Algorithm 5 Fibonacci: Iterative

```
1: function FIBONACCIITERATIVE( $n$ )
2:   if  $n < 1$  then
3:     return 0
4:   end if
5:   if  $n = 1$  or  $n = 2$  then
6:     return 1
7:   end if
8:    $F_1 \leftarrow 1$ ;  $F_2 \leftarrow 1$ 
9:   for  $i \leftarrow 3$  to  $n$  do
10:     $F_i \leftarrow F_{i-2} + F_{i-1}$ 
11:     $F_{i-2} \leftarrow F_{i-1}$ 
12:     $F_{i-1} \leftarrow F_i$ 
13:   end for
14:   return  $F_n$ 
15: end function
```

Fibonacci: Recursive vs. Iterative

Algorithm	Time Complexity	Space Complexity
Recursive	$O(2^n)$ — exponential due to overlapping sub-problems	$O(n)$ call stack
Iterative	$O(n)$	$O(1)$

The iterative algorithm is dramatically more efficient. A further improvement (beyond the scope of this module) uses matrix exponentiation to achieve $O(\log n)$.

Warning: Naive Recursion for Fibonacci

The recursive Fibonacci computes $F(n-1)$ and $F(n-2)$ as completely independent calls, so sub-problems such as $F(n-3)$, $F(n-4)$, $\dots$ are recomputed exponentially many times. For $n = 50$, the recursive version makes over 10^{10} calls. Always prefer the iterative version (or memoisation / dynamic programming) in practice.

10.5 Computing Problem Types

Algorithms are studied in the context of well-recognised categories of computational problems. SC1007 and its follow-on course SC2001 address most of these categories.

10.5.1 Searching

The goal is to find a specific *search key* in a given data set.

Definition 10.5.1 (Linear / Sequential Search). Scan every element of the array from left to right until the target is found or the array is exhausted. Time complexity: $O(n)$ worst case.

Other searching strategies include binary search ($O(\log n)$ on sorted arrays), hash-table lookup ($O(1)$ expected), and constraint-propagation search for puzzles such as Sudoku.

10.5.2 Sorting

Rearrange the elements of a given list into a specified order (numerical, lexicographic, etc.). Sorting is fundamental: many other algorithms (searching, median-finding, deduplication) become efficient once the input is sorted. Covered in detail in SC2001.

Example 10.5.2 (Sorting Motivation). Finding the top 5% of students in a class, or computing the median grade, can be done in $O(n)$ after an $O(n \log n)$ sort — versus $O(n^2)$ with a naïve repeated scan.

10.5.3 Graph Problems

Definition 10.5.3 (Graph). A graph $G = (V, E)$ is a mathematical structure consisting of a set of **vertices** V and a set of **edges** E , where each edge connects one or two vertices.

Important graph problems include:

- **Path Finding** — find a path between two vertices (e.g., maze solving, GPS routing).
- **Travelling Salesman Problem (TSP)** — find the shortest tour that visits every vertex exactly once and returns to the start. NP-hard in general.
- **Shortest Path** — Dijkstra’s or Bellman-Ford algorithms.
- **Minimum Spanning Tree** — Prim’s or Kruskal’s algorithms.

10.5.4 String Processing

String algorithms operate on sequences of characters. Key examples:

- **String Matching** — locate a pattern (e.g., MICRO) within a longer text. Naïve $O(mn)$; Knuth-Morris-Pratt achieves $O(m + n)$.
- **Text Matching in Bioinformatics** — compare genomic sequences (e.g., SARS-CoV-2 isolates) to reference genomes.

10.5.5 Computational Geometric Problems

Geometric algorithms deal with points, lines, polygons, and polyhedra in 2D or 3D space. Examples include:

- Finding the **convex hull** of a set of points.
- Finding the **closest pair** of points.
- Testing whether a point lies inside or outside a polygon.
- Constructing a **Delaunay triangulation** or **Voronoi diagram**.
- Finding the shortest obstacle-avoiding path in a planar graph.
- Detecting and resolving **collision** between objects in 2D/3D space.

10.5.6 Numerical Problems and Optimisation

Numerical algorithms approximate continuous mathematical quantities using finite arithmetic. They are widely used in engineering and mathematical modelling.

Example 10.5.4 (Linear Programming). Minimise $3x + 5y$ subject to:

$$3x + 5y \geq 27, \quad -2x + y \leq 6, \quad 2x + 5y \leq 66, \quad 7x - 4y \leq 16.$$

The feasible region is a convex polygon; the optimum lies at a vertex, which can be found via the simplex algorithm or interior-point methods.

Classical numerical methods include Newton's method (root-finding), Gaussian elimination (linear systems), and gradient descent (optimisation).

10.5.7 Combinatorial Problems

Combinatorics concerns arrangements, patterns, assignments, schedules, and configurations. Applications include:

- **Cryptography** — key generation, permutation ciphers.
- **Information Transmission** — error-correcting codes.
- **Matching and Covering** — e.g., the minimum vertex cover problem: find the smallest set of vertices in a graph such that every edge is incident to at least one chosen vertex.

10.6 Selecting Data Structures

Choosing an appropriate data structure is a prerequisite for designing an efficient algorithm. The principal structures covered in SC1007 are:

Structure	Typical Use Cases
Array	Random access by index; fixed-size collections.
Linked List	Efficient insertion/deletion; dynamic size. Variants: singly, doubly, circular.
Stack & Queue	LIFO/FIFO access patterns; expression evaluation; BFS/DFS.
Tree (BST, AVL)	Ordered dynamic sets; $O(\log n)$ search, insert, delete.
Hash Table	$O(1)$ average-case lookup; dictionary/set implementations.
Graph	Relationship modelling; path-finding; network flows.

Remark 10.6.1. The choice of data structure directly determines the achievable time complexity of an algorithm. For example, storing n sorted integers in an array enables $O(\log n)$ binary search; storing them in an unsorted linked list forces $O(n)$ search.

10.7 Algorithm Design Strategies

A *design strategy* (or *algorithm paradigm*) is a general approach to solving problems that is applicable across many different domains. The key strategies covered in SC1007 and SC2001 are summarised below.

Definition 10.7.1 (Brute Force / Exhaustive Search). Try all possible candidate solutions and check each one. Correct by construction, but often impractical for large inputs due to exponential time complexity.

Definition 10.7.2 (Divide-and-Conquer). Recursively split the problem into independent sub-problems of the same type, solve each sub-problem, and combine the results. Examples: merge sort, quicksort, binary search.

Definition 10.7.3 (Decrease-and-Conquer). Reduce the problem to a single smaller instance of itself. Examples: insertion sort, computing $n!$ recursively, DFS/BFS on graphs.

Definition 10.7.4 (Greedy Strategy). At each step, make the locally optimal choice, hoping it leads to a globally optimal solution. Examples: Dijkstra's shortest path, Prim's MST, Huffman coding.

Definition 10.7.5 (Transform-and-Conquer). Transform the problem into a different, more tractable form before solving it. Example: converting an infix arithmetic expression to postfix (Reverse Polish Notation) using a stack, then evaluating the postfix form.

Definition 10.7.6 (Iterative Improvement). Start from a feasible solution and repeatedly apply local improvements until no further improvement is possible. Examples: simplex method, network flow algorithms.

Strategy	Key Idea
Brute Force	Enumerate all candidates
Divide-and-Conquer	Split $\rightarrow$ solve $\rightarrow$ combine
Decrease-and-Conquer	Reduce to one smaller instance
Greedy	Local optimum at each step
Transform-and-Conquer	Reformulate, then solve
Iterative Improvement	Local search from feasible solution

Exam Tip: Recognising the Right Paradigm

A reliable classification heuristic:

- If the problem has *independent* sub-problems $\rightarrow$ Divide-and-Conquer.
- If each step reduces the problem size by a constant $\rightarrow$ Decrease-and-Conquer.
- If a locally optimal choice is provably globally optimal $\rightarrow$ Greedy.
- If no structure is apparent and the search space is small $\rightarrow$ Brute Force.
- If the problem is easier in a different representation $\rightarrow$ Transform-and-Conquer.

10.8 Summary

Chapter Summary

1. An algorithm is **not** simply a computer program: it is a precise, finite, correct specification of a computation, independent of any programming language.
2. The same problem can have multiple correct algorithms with very different efficiency profiles (e.g., $O(1)$ vs. $O(n)$ vs. $O(2^n)$ for the same task).
3. Computing problems fall into recurring categories: searching, sorting, graph problems, string processing, geometric problems, numerical problems, and combinatorial problems.
4. Algorithm design is guided by recognised strategies: brute force, divide-and-conquer, decrease-and-conquer, greedy, transform-and-conquer, and iterative improvement.
5. The choice of data structure is inseparable from algorithm design — the right structure can reduce complexity by orders of magnitude.

Chapter 11

Analysis of Algorithms

The previous chapter introduced what an algorithm is and the major design paradigms. Here we turn to the central question of *how efficient* an algorithm is. We measure efficiency in two dimensions—time and space—and develop a mathematical language (asymptotic notation) for comparing algorithms at the level of their growth behaviour, independent of hardware or implementation details.

11.1 Time and Space Complexity

Definition 11.1.1 (Time Complexity). The **time complexity** of an algorithm is the amount of time (measured as a count of primitive operations) used by the algorithm as a function of its input size n .

Definition 11.1.2 (Space Complexity). The **space complexity** of an algorithm is the amount of memory (measured in basic storage units) used by the algorithm as a function of its input size n .

Definition 11.1.3 (Primitive Operation). A **primitive operation** is a basic computational step that executes in constant time, independent of the problem size. Examples include:

- **Declaration:** `int x;`
- **Assignment:** `x = 1;`
- **Arithmetic operations:** `+, -, *, /, %`
- **Logic / comparison operations:** `==, !=, >, <, &&, ||`

The general procedure for time-complexity analysis is:

1. Count the number of primitive operations (focusing on the *important* or *dominant* ones).
2. Express that count as a function of the input size n .

11.1.1 Repetition Structures (Loops)

Example 11.1.4 (Single while loop — factorial). Consider computing $n!$ iteratively:

```
1 j = 1
2 factorial = 1
3 while j <= n:
4     factorial = factorial * j # cost c1 per iteration
5     j = j + 1                # cost c2 per iteration
```

The loop body executes n times, so

$$f(n) = c_0 + c_1 \cdot n + c_2 \cdot n = c_0 + (c_1 + c_2)n.$$

This is a **linear** function of n .

Example 11.1.5 (Nested for loop — matrix sum).

```

1 for j in range(1, m+1):
2     for k in range(1, n+1):
3         total = total + M[j][k] # cost c1

```

The inner loop runs n times for each of the m outer iterations, giving

$$f(n, m) = m \cdot n \cdot c_1.$$

When $m = n$ this becomes $c_1 n^2$ —a **quadratic** function.

Example 11.1.6 (Triple nested loop — cubic example).

```

1 for i in range(1, n+1):
2     M[i] = 0
3     for j in range(i, 0, -1):
4         for k in range(i, 0, -1):
5             M[i] += A[j] * B[k] # 1 multiplication per iteration

```

For a given outer index i , the two inner loops each run i times, so the inner body executes i^2 times. Summing over all i :

$$f(n) = \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} \in \Theta(n^3).$$

11.1.2 Selection Structures (Conditionals)

When an algorithm contains **if/else** or **switch/case** statements, the number of operations executed depends on the input. We therefore distinguish three cases:

Definition 11.1.7 (Best-, Worst-, and Average-Case Complexity). Let $T(n, x)$ be the operation count for an algorithm on input of size n with specific input x .

- **Best-case:** $T_{\text{best}}(n) = \min_x T(n, x)$
- **Worst-case:** $T_{\text{worst}}(n) = \max_x T(n, x)$
- **Average-case:** $T_{\text{avg}}(n) = \sum_x p(x) T(n, x)$, where $p(x)$ is the probability of input x .

Example 11.1.8 (if/else statement).

```

1 if x < a:
2     sum1 += x # cost c1
3 else:
4     sum2 += x # cost c2
5     count += 1 # also part of else branch

```

Let $p = P(x < a)$.

- **Best-case:** c_1 (the if branch)
- **Worst-case:** c_2 (the else branch)
- **Average-case:** $p \cdot c_1 + (1 - p) \cdot c_2$

Example 11.1.9 (switch/case statement). Consider a menu-driven program with four cases having operation counts $T_1 = 5n$, $T_2 = 6 \log_2 n$, $T_3 = 3n$, $T_4 = 4 \log_2 n$, and a constant switching cost C .

- **Best-case:** $C + T_{\min} = C + 4 \log_2 n$

- **Worst-case:** $C + T_{\max} = C + 5n$
- **Average-case:** $C + \sum_{i=1}^4 p(i) T_i$

For example, with probabilities 0.1, 0.4, 0.3, 0.2 the average cost is $C + 1.4n + 3.2 \log_2 n$.

Example 11.1.10 (Sequential search in a linked list).

```

1 pt = head
2 while pt.key != a:
3     pt = pt.next
4     if pt is None:
5         break

```

Let c_1 be the cost of checking the first node and c_2 the cost per iteration. Assuming the key is always present:

- **Best-case:** c_1 (key is the first element)
- **Worst-case:** $c_1 + c_2(n - 1)$ (key is the last element)
- **Average-case:** $c_1 + \frac{c_2(n - 1)}{2}$

Combining success and failure cases with probability $P(\text{succ})$:

$$T_{\text{avg}}(n) = c_1 + nc_2 - P(\text{succ}) \cdot \frac{c_2(n + 1)}{2},$$

which is **linear** in n .

11.1.3 Recursive Functions

For recursive algorithms we establish a **recurrence relation** and solve it using one of two standard methods.

Backward Substitution

Example 11.1.11 (Recursive count of an element in an array).

```

1 def count(array, n, a):
2     if n == 1:
3         return 1 if array[0] == a else 0
4     if array[0] == a:
5         return 1 + count(array[1:], n - 1, a)
6     else:
7         return count(array[1:], n - 1, a)

```

Counting the comparison `array[0] == a` as the important operation:

$$W_1 = 1, \quad W_n = 1 + W_{n-1}.$$

Expanding by backward substitution:

$$W_n = 1 + W_{n-1} = 1 + 1 + W_{n-2} = \cdots = (n - 1) + W_1 = n.$$

The time complexity is $\Theta(n)$.

Example 11.1.12 (Recursive factorial).


```

1 def factorial(n):
2     if n == 1:
3         return 1
4     return n * factorial(n - 1)

```

With cost c_1 per recursive call and c_2 for the base case:

$$f(n) = c_1(n - 1) + c_2.$$

This is a linear function of n .

Forward Substitution

Example 11.1.13 (Pre-order traversal of a complete binary tree).

```

1 def preorder(tree):
2     if tree is not None:
3         tree.item *= 10      # 1 multiplication (cost)
4         preorder(tree.left)
5         preorder(tree.right)

```

Let k be the depth of the tree (number of nodes = $2^k - 1$).

$$W_0 = 0, \quad W_1 = 1, \quad W_2 = 1 + W_1 + W_1 = 3, \quad W_3 = 1 + 2W_2 = 7, \dots$$

By forward substitution:

$$W_k = 1 + 2W_{k-1} = 1 + 2 + 4 + \dots + 2^{k-1} = 2^k - 1.$$

This is a **geometric series**. Since the function visits every node of a tree with $2^k - 1$ nodes, the time complexity is $\Theta(2^k) = \Theta(n)$ in the number of nodes.

11.1.4 Useful Series Formulae

The following closed-form results are frequently needed when summing loop iteration counts.

Key Series Formulae

$$\text{Geometric: } G_n = \frac{a(1 - r^n)}{1 - r}$$

$$\text{Arithmetic: } A_n = \frac{n}{2} [a_0 + a_{n-1}]$$

$$\text{Sum of squares (Faulhaber): } \sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\text{Sum of cubes: } \sum_{k=1}^n k^3 = \frac{n^2(n+1)^2}{4}$$

$$\text{Arithmetico-geometric: } \sum_{t=1}^k t \cdot 2^{t-1} = 2^k(k-1) + 1$$

11.2 Order of Growth

Definition 11.2.1 (Order of Growth). The **order of growth** of a function describes its asymptotic growth behaviour as the input size becomes large. It ignores constant factors and focuses on the dominant term.

When comparing algorithms, constant multipliers and lower-order terms become negligible for large n . For example:

- $13n$ and $130n$ have the *same* order of growth (both linear).
- $13n^2 + 10^2$ and $13n^2$ have the same dominant behaviour; the constant 10^2 can be ignored for large n .
- $n!$ grows far faster than 2^n , which grows faster than any polynomial.

The table below illustrates real running times (in μs) for several complexity classes on various problem sizes:

Size n	$13n$	$13n \log_2 n$	$13n^2$	$130n^2$	2^n	$n!$
10	0.00013	0.00043	0.0013	0.013	0.001024	3.63 s
100	0.0013	0.0086	0.13	1.3	$\approx 4 \times 10^{16} \text{ yr}$	$\approx 3 \times 10^{144} \text{ yr}$
10^4	0.13	1.73	22 min	3.6 hr	—	—
10^6	13	259	150 days	1505 days	—	—

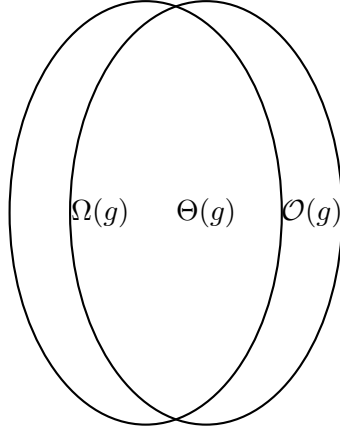
Growth Rate Ordering (slowest to fastest)

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

Remark 11.2.2. A $10\times$ faster computer provides only a modest improvement for exponential or factorial algorithms. For a 2^n algorithm running on $n = 13$ items in one hour, the faster machine raises the limit to about $n = 16$ — a gain of only three additional items. Algorithmic improvement always dominates hardware improvement for hard problems.

11.3 Asymptotic Notation

Three asymptotic notations are used to classify the order of growth of complexity functions: $\mathcal{O}$ (Big-Oh), Ω (Big-Omega), and Θ (Big-Theta). Together they provide upper bounds, lower bounds, and tight bounds respectively.



Functions relative to g

11.3.1 Big-Oh Notation ($\mathcal{O}$)

Definition 11.3.1 (Big-Oh Notation). Let $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$. Then $f(n) \in \mathcal{O}(g(n))$ if $f(n)$ is **bounded above** by a constant multiple of $g(n)$ for all sufficiently large n :

$$\mathcal{O}(g(n)) = \{f(n) : \exists c > 0, n_0 \text{ s.t. } 0 \leq f(n) \leq c g(n) \forall n \geq n_0\}.$$

Equivalently (limit definition):

$$f(n) \in \mathcal{O}(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty.$$

We say $g(n)$ is an **asymptotic upper bound** for $f(n)$.

Example 11.3.2 (Big-Oh examples). Let $f(n) = 4n + 3$.

- $g(n) = n$: choose $c = 5, n_0 = 3$. Then $4n + 3 \leq 5n$ for all $n \geq 3$. Hence $4n + 3 \in \mathcal{O}(n)$.
Verification via limit: $\lim_{n \rightarrow \infty} \frac{4n+3}{n} = 4 < \infty$. ✓
- $g(n) = n^3$: choose $c = 1, n_0 = 3$. Then $4n + 3 \leq n^3$ for all $n \geq 3$. Hence $4n + 3 \in \mathcal{O}(n^3)$.
✓
- $g(n) = e^n$: applying L'Hôpital's Rule, $\lim_{n \rightarrow \infty} \frac{4n+3}{e^n} = \lim_{n \rightarrow \infty} \frac{4}{e^n} = 0 < \infty$. Hence $4n + 3 \in \mathcal{O}(e^n)$. ✓

Warning: Big-Oh does not mean “tight”

$f(n) \in \mathcal{O}(g(n))$ only guarantees an *upper* bound. $4n + 3 \in \mathcal{O}(n^3)$ is true, but it is a loose bound. For the tightest characterisation, use Θ notation.

11.3.2 Big-Omega Notation (Ω)

Definition 11.3.3 (Big-Omega Notation). Let $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$. Then $f(n) \in \Omega(g(n))$ if $f(n)$ is **bounded below** by a constant multiple of $g(n)$ for all sufficiently large n :

$$\Omega(g(n)) = \{f(n) : \exists c > 0, n_0 \text{ s.t. } 0 \leq c g(n) \leq f(n) \forall n \geq n_0\}.$$

Equivalently (limit definition):

$$f(n) \in \Omega(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} > 0 \quad (\text{including } \infty).$$

We say $g(n)$ is an **asymptotic lower bound** for $f(n)$.

Example 11.3.4 (Big-Omega example). Let $f(n) = 4n + 3$ and $g(n) = n$. Choose $c = 1, n_0 = 1$. Then $4n + 3 \geq n$ for all $n \geq 1$, so $4n + 3 \in \Omega(n)$. Via limit: $\lim_{n \rightarrow \infty} \frac{4n+3}{n} = 4 > 0$. ✓

11.3.3 Big-Theta Notation (Θ)

Definition 11.3.5 (Big-Theta Notation). Let $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$. Then $f(n) \in \Theta(g(n))$ if $f(n)$ is **bounded both above and below** by constant multiples of $g(n)$ for all sufficiently large n :

$$\Theta(g(n)) = \{f(n) : \exists c_1, c_2 > 0, n_0 \text{ s.t. } c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0\}.$$

Equivalently (limit definition):

$$f(n) \in \Theta(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c, \quad 0 < c < \infty.$$

We say $g(n)$ is an **asymptotic tight bound** for $f(n)$.

Example 11.3.6 (Big-Theta example). Let $f(n) = 4n + 3$ and $g(n) = n$. $\lim_{n \rightarrow \infty} \frac{4n+3}{n} = 4$, and $0 < 4 < \infty$. Hence $4n + 3 \in \Theta(n)$.

11.3.4 Summary: Limit-Based Classification

Limit Rule for Asymptotic Classification

$L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$			
L	$f \in \mathcal{O}(g)$	$f \in \Omega(g)$	$f \in \Theta(g)$
0	✓		
$0 < c < \infty$	✓	✓	✓
∞		✓	

11.3.5 Simplification Rules

Theorem 11.3.7 (Asymptotic Simplification Rules).

1. **Constant scaling:** $f(n) = \mathcal{O}(c g(n)) \Rightarrow f(n) = \mathcal{O}(g(n))$ for any constant $c > 0$.
2. **Transitivity:** $f = \mathcal{O}(g)$ and $g = \mathcal{O}(h) \Rightarrow f = \mathcal{O}(h)$. e.g. $2n = \mathcal{O}(n^2)$ and $n^2 = \mathcal{O}(n^3)$ implies $2n = \mathcal{O}(n^3)$.
3. **Sum rule:** $f_1 = \mathcal{O}(g_1)$ and $f_2 = \mathcal{O}(g_2) \Rightarrow f_1 + f_2 = \mathcal{O}(\max(g_1, g_2))$. e.g. $5n + 3 \log_2 n = \mathcal{O}(n)$.
4. **Product rule:** $f_1 = \mathcal{O}(g_1)$ and $f_2 = \mathcal{O}(g_2) \Rightarrow f_1 f_2 = \mathcal{O}(g_1 g_2)$. e.g. $3n^2 \cdot \log_2 n = \mathcal{O}(n^2 \log_2 n)$.

Theorem 11.3.8 (Properties of Asymptotic Notation).

- **Reflexivity** ($\mathcal{O}, \Omega, \Theta$): $f(n) = \mathcal{O}(f(n))$, etc.
- **Symmetry** (Θ only): $f = \Theta(g) \Leftrightarrow g = \Theta(f)$.
- **Transitivity** ($\mathcal{O}, \Omega, \Theta$): $f = \mathcal{O}(g)$ and $g = \mathcal{O}(h) \Rightarrow f = \mathcal{O}(h)$, etc.

11.3.6 Asymptotic Notation in Equations

Asymptotic notation may appear inside equations as a shorthand for an unspecified function of that complexity class. For example:

- $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$: the $3n + 1$ term is in $\Theta(n)$.
- $2n^2 + 3n + 1 = 2n^2 + \Theta(n) = \Theta(n^2)$: the dominant term is n^2 .
- $T(n) = 2T(\frac{n}{2}) + \Theta(n)$: a recurrence for merge sort.

11.4 Common Complexity Classes

Standard Complexity Classes (slowest to fastest growth)

Order	Class	Canonical Example
$\mathcal{O}(1)$	Constant	Finding midpoint of an array
$\mathcal{O}(\log n)$	Logarithmic	Binary search
$\mathcal{O}(n)$	Linear	Linear (sequential) search
$\mathcal{O}(n \log n)$	Linearithmic	Merge sort
$\mathcal{O}(n^2)$	Quadratic	Bubble sort, insertion sort
$\mathcal{O}(n^3)$	Cubic	Naïve matrix inversion
$\mathcal{O}(2^n)$	Exponential	Recursive Fibonacci, Tower of Hanoi
$\mathcal{O}(n!)$	Factorial	Travelling Salesman (brute force)

Remark 11.4.1. When the time complexity of algorithm A grows faster than that of algorithm B for the same problem, we say A is **inferior** to B . In practice, $\mathcal{O}$ -notation is the most widely reported bound because it communicates the worst-case guarantee.

Worked derivation — how to determine $\mathcal{O}$ from an algorithm:

1. Count primitive operations to obtain $f(n)$.
2. Discard all constant terms and constant multipliers.
3. Identify the **dominant term** (the fastest-growing term).
4. The dominant term gives the $\mathcal{O}$ -notation.

11.5 Space Complexity

Space complexity counts **basic storage units** (integers, floats, characters) used by an algorithm.

Example 11.5.1 (Array of n integers). An array of n integers requires $\Theta(n)$ space.

Example 11.5.2 (Adjacency matrix for a graph). An $n \times n$ matrix used to store edge information for a graph with n vertices (where $G[x][y] = 1$ iff there is an edge from x to y) requires $\Theta(n^2)$ space.

Space–Time Tradeoff Principle

There is typically an inverse relationship between time complexity and space complexity. Reducing time often requires storing pre-computed or intermediate results (using more memory), and vice versa. Algorithm designers frequently exploit this tradeoff deliberately, e.g. memoisation (caches results for constant-time lookup at the cost of $\Theta(n)$ extra space).

11.6 Summary

Chapter Summary: Analysis of Algorithms

1. **Time complexity** counts primitive operations as a function of input size n ; **space complexity** counts storage units.
2. For *selection structures*, always state best-case, worst-case, and average-case separately.
3. For *recursive functions*, set up a recurrence relation and solve via backward substitution (unfold from W_n) or forward substitution (compute $W_1, W_2, \dots$ and spot the pattern).
4. **Order of growth** ignores constant factors; focus on the dominant term.
5. $\mathcal{O}(g)$: upper bound ($f \leq cg$). $\Omega(g)$: lower bound ($f \geq cg$). $\Theta(g)$: tight bound ($c_1g \leq f \leq c_2g$).
6. Use the limit rule to classify: $L = 0 \Rightarrow \mathcal{O}$; $0 < L < \infty \Rightarrow \Theta$ (and hence also $\mathcal{O}$ and Ω); $L = \infty \Rightarrow \Omega$.

Chapter 12

Searching

This chapter examines three fundamental searching algorithms, classified by their design strategy: sequential search (a brute-force exhaustive approach), binary search (a decrease-and-conquer strategy), and jump search (a compromise between the two). Each algorithm's time complexity is analysed under best-case, average-case, and worst-case assumptions.

12.1 Sequential Search

Sequential search (also called *linear search*) is the simplest searching strategy: traverse the data structure element by element, comparing each key with the target until a match is found or the structure is exhausted.

```
1 def search(head, a):
2     pt = head
3     while pt is not None and pt.key != a:
4         pt = pt.next
5     return pt
```

Because the data is stored *unordered*, every element may need to be inspected. This makes sequential search a **brute-force** (naïve) algorithm.

12.1.1 Time Complexity of Sequential Search

Let c_1 denote the cost of the comparison and return when the target is found immediately, and let c_2 denote the cost of one loop iteration (comparison plus pointer advance).

Best case. The target is the first element. Only the initial comparison executes, giving a cost of c_1 , hence $\Theta(1)$.

Worst case. The target is the last element (or absent). The loop runs $n - 1$ iterations before the final comparison, costing $c_2(n - 1) + c_1 = \Theta(n)$.

Average case (target present). Assume that every element is equally likely to be the target, so each position i has probability $p_i = \frac{1}{n}$. Finding the element at position i costs $c_1 + (i - 1)c_2$. The expected cost is therefore:

$$\begin{aligned} A_s(n) &= \sum_{i=1}^n \frac{1}{n} [c_1 + (i - 1)c_2] = \frac{1}{n} \left[n c_1 + c_2 \sum_{i=1}^n (i - 1) \right] \\ &= \frac{1}{n} \left[n c_1 + c_2 \cdot \frac{n(n - 1)}{2} \right] = c_1 + \frac{c_2(n - 1)}{2} = \Theta(n). \end{aligned} \quad (12.1)$$

Average case (target may be absent). If the target is absent, the full list is scanned at cost $c_1 + n c_2 = \Theta(n)$. Letting $q = P(\text{target in list})$, the overall average is:

$$A(n) = q \left(c_1 + \frac{c_2(n-1)}{2} \right) + (1-q)(c_1 + n c_2),$$

which remains a linear function of n , so the average-case complexity is $\Theta(n)$ regardless of q .

Sequential Search Complexity

Sequential search runs in $\mathcal{O}(n)$ time. Best case is $\Theta(1)$; worst and average cases are both $\Theta(n)$. The data need not be sorted.

12.2 Binary Search

Binary search exploits a **sorted** list by repeatedly halving the search interval — a classic *decrease-and-conquer* strategy.

12.2.1 Recursive Implementation

```
1 def binary_search_recursive(arr, left, right, target):
2     if left > right:
3         return -1
4     mid = left + (right - left) // 2
5     if arr[mid] == target:
6         return mid
7     elif arr[mid] < target:
8         return binary_search_recursive(
9             arr, mid + 1, right, target)
10    else:
11        return binary_search_recursive(
12            arr, left, mid - 1, target)
```

12.2.2 Iterative Implementation

```
1 def binary_search(arr, target):
2     low = 0
3     high = len(arr) - 1
4     while low <= high:
5         mid = (low + high) // 2
6         if arr[mid] == target:
7             return mid
8         elif arr[mid] < target:
9             low = mid + 1
10        else:
11            high = mid - 1
12    return -1
```

12.2.3 Connection to Binary Search Trees

A sorted list can be viewed as the in-order traversal of a **binary search tree** (BST). For example, the sorted list [14, 23, 31, 56, 73, 93, 94] corresponds to the following BST with root 56:

Binary search on the array is equivalent to traversing from the root towards a leaf in the BST, making one comparison per level. An alternative recursive formulation using BST node traversal is:

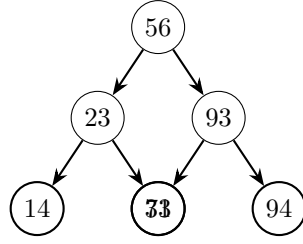


Figure 12.1: BST corresponding to the sorted list [14, 23, 31, 56, 73, 93, 94].

```

1 def binary_search(self, target, current_node):
2     if current_node is None:
3         return False
4     elif target == current_node.data:
5         return True
6     elif target < current_node.data:
7         return self.binary_search(
8             target, current_node.left)
9     else:
10        return self.binary_search(
11            target, current_node.right)

```

12.2.4 Tree Height and Node Count

Definition 12.2.1 (Height and Depth). The **height** of a tree is the number of edges on the longest path from the root to a leaf. The **depth** of a node is the number of edges from that node to the root.

For a *complete* binary tree of height H , the number of nodes n satisfies:

$$2^H \leq n < 2^{H+1}.$$

Taking logarithms gives $H \leq \log_2 n < H + 1$, and since H is an integer we obtain:

$$H = \lfloor \log_2 n \rfloor.$$

12.2.5 Worst-Case Time Complexity

Assume a complete binary tree with n nodes. Let $f(n)$ denote the worst-case number of operations. At each step, a constant amount of work c is done, then the problem is reduced to a subtree of size $\frac{n-1}{2}$. This gives the recurrence:

$$f(n) = f\left(\frac{n-1}{2}\right) + c.$$

Unfolding k times:

$$f(n) = f\left(\frac{n - (2^k - 1)}{2^k}\right) + kc. \quad (12.2)$$

The recursion bottoms out when the argument equals 1, i.e. when $\frac{n - (2^k - 1)}{2^k} = 1$, which gives $2^k = \frac{n+1}{2}$, so $k = \log_2(n+1) - 1 = \lfloor \log_2 n \rfloor$. Substituting back:

$$f(n) = f(1) + kc = c + \lfloor \log_2 n \rfloor \cdot c = (\lfloor \log_2 n \rfloor + 1)c = \Theta(\log_2 n).$$

Deriving Binary Search Complexity

Set up the recurrence $f(n) = f(\frac{n-1}{2}) + c$, unfold k times to get $f(n) = f(\frac{n-(2^k-1)}{2^k}) + kc$, then solve for k by setting the argument to 1. The result is $k = \lfloor \log_2 n \rfloor$ and $f(n) = \Theta(\log_2 n)$.

12.2.6 Average-Case Time Complexity

Let $A_s(n)$ and $A_f(n)$ denote the average number of comparisons for a successful and unsuccessful search respectively. The overall average is $A(n) = q A_s(n) + (1 - q) A_f(n)$, where q is the probability the target is present.

Unsuccessful search. An unsuccessful search always reaches a leaf, traversing the full height of the tree, so $A_f(n) = \Theta(\log_2 n)$.

Successful search. Assume $n = 2^k - 1$ (a complete binary tree). At level t of the tree there are 2^{t-1} nodes, each requiring exactly t comparisons. Under the uniform distribution ($p_i = \frac{1}{n}$):

$$A_s(n) = \frac{1}{n} \sum_{t=1}^k t \cdot 2^{t-1}.$$

To evaluate $\sum_{t=1}^k t \cdot 2^{t-1}$, we use the perturbation method. Let $S = \sum_{t=1}^k t \cdot 2^{t-1}$. Multiplying by 2 and subtracting:

$$\begin{aligned} S &= 1 \cdot 1 + 2 \cdot 2 + 3 \cdot 4 + \cdots + k \cdot 2^{k-1}, \\ 2S &= \quad \quad 1 \cdot 2 + 2 \cdot 4 + \cdots + (k-1) \cdot 2^{k-1} + k \cdot 2^k. \end{aligned}$$

Subtracting gives $(2 - 1)S = k \cdot 2^k - (1 + 2 + 4 + \cdots + 2^{k-1}) = k \cdot 2^k - (2^k - 1)$, so:

$$S = (k - 1) 2^k + 1.$$

Therefore:

$$A_s(n) = \frac{(k - 1) 2^k + 1}{n} = \frac{[\log_2(n + 1) - 1](n + 1) + 1}{n} = \log_2(n + 1) - 1 + \frac{\log_2(n + 1)}{n}.$$

Overall average. Since $q \leq 1$ and $\frac{\log_2(n+1)}{n}$ is negligible for large n :

$$A(n) = q A_s(n) + (1 - q) A_f(n) = \Theta(\log_2 n).$$

Binary Search Complexity

Binary search runs in $\mathcal{O}(\log_2 n)$ time. Best case is $\Theta(1)$ (target at the midpoint); worst and average cases are both $\Theta(\log_2 n)$. **Prerequisite:** the input must be sorted.

Binary Search Requires Sorted Input

Binary search only produces correct results on a sorted array (or equivalently, a valid BST). Applying it to unsorted data silently yields incorrect answers — the algorithm does not detect unsorted input.

Complete/Balanced BST Required for $O(\log n)$

The $O(\log_2 n)$ complexity of binary search on a BST assumes the tree is *complete* or *balanced*. A degenerate BST—where each node has only one child—reduces to a linked list, and binary search degrades to $O(n)$. To guarantee logarithmic search time, use a balanced BST such as an AVL tree (Chapter 9).

12.3 Jump Search

Jump search is a searching algorithm for sorted arrays that reduces the number of comparisons relative to sequential search by “jumping” ahead in fixed-size blocks of $\lfloor \sqrt{n} \rfloor$ elements, then performing a linear scan within the identified block. It is particularly useful when binary search is costly — for example, when searching a very large sorted dataset stored on a slow storage medium such as a disk-based database.

```
1 import math
2
3 def jump_search(arr, target):
4     n = len(arr)
5     step = int(math.sqrt(n))
6     prev = 0
7     while prev < n and arr[min(step, n) - 1] < target:
8         prev = step
9         step += int(math.sqrt(n))
10        if prev >= n:
11            return -1
12    for i in range(prev, min(step, n)):
13        if arr[i] == target:
14            return i
15    return -1
```

12.3.1 Time Complexity of Jump Search

The algorithm operates in two phases: a *jump phase* that skips through at most $\frac{n}{\sqrt{n}} = \sqrt{n}$ blocks, and a *linear scan phase* within a single block of size at most $\sqrt{n}$.

Best case. The target is at the start of the array: $\Theta(1)$.

Worst case. The jump phase inspects up to $\sqrt{n}$ block boundaries, and the linear scan checks up to $\sqrt{n}$ elements within the final block, giving $\Theta(\sqrt{n}) + \Theta(\sqrt{n}) = \Theta(\sqrt{n})$.

Average case. Under the uniform distribution, each element is equally likely, and the expected number of comparisons is:

$$\sum_{i=1}^n \frac{1}{n} \Theta(\sqrt{n}) = \Theta(\sqrt{n}).$$

Jump Search Complexity

Jump search runs in $\mathcal{O}(\sqrt{n})$ time. It sits between sequential search ($\mathcal{O}(n)$) and binary search ($\mathcal{O}(\log n)$) in efficiency. Like binary search, it requires sorted input.

12.4 Summary

The following table compares the three searching algorithms discussed in this chapter.

Algorithm	Best	Average	Worst	Overall
Sequential	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\mathcal{O}(n)$
Binary	$\Theta(1)$	$\Theta(\log_2 n)$	$\Theta(\log_2 n)$	$\mathcal{O}(\log_2 n)$
Jump	$\Theta(1)$	$\Theta(\sqrt{n})$	$\Theta(\sqrt{n})$	$\mathcal{O}(\sqrt{n})$

Sequential search is a brute-force exhaustive algorithm that requires no assumptions about the data ordering. Binary search and jump search are both decrease-and-conquer algorithms that require the input to be sorted. Binary search achieves the best asymptotic complexity of the three at $\mathcal{O}(\log_2 n)$, but jump search can be preferable in practice when random access is expensive (e.g. data on disk), since it performs fewer large jumps followed by a short sequential scan.

Chapter 13

Hash Tables

The search algorithms discussed in Chapter 12 achieve at best $O(\log n)$ time (binary search) for sorted data. *Hash tables* take a fundamentally different approach: they trade space for time, using a **hash function** to map keys directly to array indices, achieving $O(1)$ average-case lookup, insertion, and deletion.

13.1 Direct-Address Tables

Definition 13.1.1 (Direct-Address Table). Suppose the set of actual keys K is drawn from a universe $U = \{0, 1, 2, \dots, m - 1\}$ and all keys are distinct. A *direct-address table* is an array $T[0 \dots m - 1]$ where

$$T[k] = \begin{cases} x & \text{if } k \in K \text{ and } x.\text{key} = k, \\ \text{NIL} & \text{otherwise.} \end{cases}$$

All operations—search, insert, delete—run in $O(1)$ time because the key itself is the index. However, direct-address tables are impractical when the universe U is much larger than the set of actual keys ($|U| \gg |K|$). For example, if keys are 64-bit integers, the table would need $\approx 1.8 \times 10^{19}$ entries regardless of how few keys are actually stored.

13.2 Hashing

To overcome the space problem of direct addressing, we use a *hash function* h that maps the universe U of all possible keys into a smaller range of hash table slots:

$$h: U \rightarrow \{0, 1, 2, \dots, m - 1\}.$$

Definition 13.2.1 (Hash Table and Load Factor). A *hash table* is an array of m slots indexed $0, 1, \dots, m - 1$. A key k is stored at index $h(k)$. If n keys are stored in a table of size m , the *load factor* is $\alpha = n/m$.

Because the domain of h is typically much larger than its range, multiple keys may map to the same slot. This event is called a *collision*.

13.3 Hash Functions

A good hash function should satisfy three properties: it must map every possible key to a valid table index; it should distribute keys as evenly as possible across slots; and it should be fast to compute.

13.3.1 Division Method (Modulo Arithmetic)

The simplest hash function divides the key by the table size and takes the remainder:

$$h(k) = k \bmod m.$$

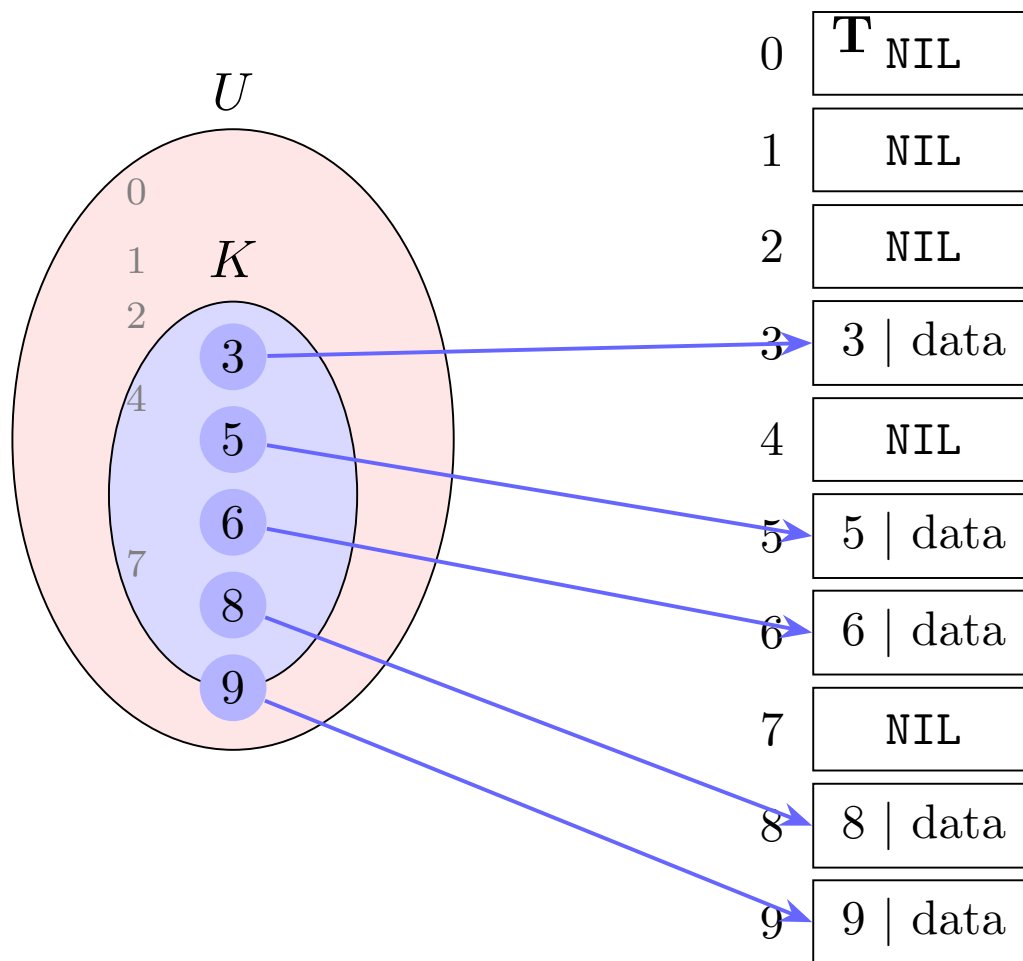


Figure 13.1: Direct-address table: each key maps directly to its index.

Example 13.3.1 (Division Method). With $m = 13$ and $k = 37699$: $h(37699) = 37699 \bmod 13 = 12$.

Choosing m for the Division Method

Avoid powers of 10 (for decimal keys) and powers of 2 (when lower bit patterns are non-uniform). The best choice of m is a **prime number** not too close to any power of 2, as this distributes keys most evenly.

13.3.2 Folding Method

The key is partitioned into several parts, which are then combined (typically by addition) and taken modulo m :

$$h(k) = (k_1 + k_2 + \cdots + k_p) \bmod m,$$

where $k_1, k_2, \dots, k_p$ are fixed-width segments of k .

Example 13.3.2 (Shift Folding). With $m = 13$ and $k = 123456789$, partition into 3-digit groups: $h(123456789) = (123 + 456 + 789) \bmod 13 = 1368 \bmod 13 = 3$. This method is especially useful for very long keys (e.g. 20-digit numbers).

```
1 // Folding hash for a 10-character key
2 int h(char x[10]) {
```

```

3  int i, sum;
4  for (sum = 0, i = 0; i < 10; i++)
5      sum += (int) x[i];
6  return (sum % M);
7  }

```

13.3.3 Mid-Square Method

The key is squared, and the middle r bits (or digits) of the result are extracted as the hash value for a table of 2^r slots.

Example 13.3.3 (Mid-Square). With $k = 3121$: $k^2 = 9,740,641$. Extracting the middle three digits gives $h(k) = 406$.

13.3.4 Multiplicative Congruential Method

A pseudo-random multiplier a is used:

$$h(k) = (a \times k) \bmod m.$$

Example 13.3.4 (Multiplicative Congruential). With $m = 31$ and $a = 8\lfloor m/23 \rfloor + 5 = 8(1) + 5 = 13$, the keys $k = 1, 2, \dots, 15$ map to 13, 26, 8, 21, 3, 16, 29, 11, 24, 6, 19, 1, 14, 27, 9 respectively—a well-spread distribution.

13.4 Collision Resolution

When two keys $k_1 \neq k_2$ satisfy $h(k_1) = h(k_2)$, a *collision* occurs. Two families of strategies handle collisions: *closed addressing* (the key stays at its hashed slot, with overflow stored in a linked list) and *open addressing* (the key probes for an alternative slot within the table).

13.4.1 Closed Addressing: Separate Chaining

Each slot in the hash table points to a linked list (a *chain*). All keys that hash to the same index are appended to that slot's list.

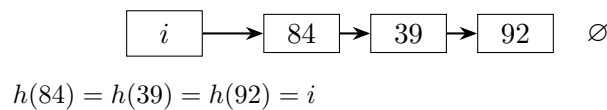


Figure 13.2: Separate chaining: keys 84, 39, 92 all hash to slot i .

13.4.1.1 Python Implementation

The following implementation uses a linked list at each slot, with insert, search, and delete operations:

```

1  class HashTableNode:
2      def __init__(self, key):
3          self.key = key
4          self.next = None
5
6  class HashTable:
7      def __init__(self, num_data):
8          self.load_factor = 3

```

```

9     self.size = max(1, num_data // self.load_factor)
10    self.table = [None] * self.size
11
12    def _hash(self, key):
13        return hash(key) % self.size
14
15    def hash_insert(self, key):
16        index = self._hash(key)
17        head = self.table[index]
18        if head is None:
19            self.table[index] = HashTableNode(key)
20            return True
21        current = head
22        while True:
23            if current.key == key:
24                return False # duplicate
25            if current.next is None:
26                break
27            current = current.next
28        current.next = HashTableNode(key)
29        return True
30
31    def hash_search(self, key):
32        index = self._hash(key)
33        current = self.table[index]
34        while current:
35            if current.key == key:
36                return True
37            current = current.next
38        return False
39
40    def hash_delete(self, key):
41        index = self._hash(key)
42        current = self.table[index]
43        prev = None
44        while current:
45            if current.key == key:
46                if prev is None:
47                    self.table[index] = current.next
48                else:
49                    prev.next = current.next
50                return True
51            prev = current
52            current = current.next
53        return False

```

13.4.1.2 Worst-Case Analysis

The worst case occurs when *all* n keys hash to the same slot, producing a single chain of length n . Searching then degenerates to sequential search (cf. Section 12.1):

- **Unsuccessful search:** n key comparisons, $\Theta(n)$.
- **Successful search:** assuming equal probability $1/n$ for each key, the expected number of comparisons is $\frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2} = \Theta(n)$.

13.4.1.3 Average-Case Analysis

Theorem 13.4.1 (Average-Case Complexity of Chained Hashing). *Under the assumption that each key is equally likely to hash into any of the m slots (simple uniform hashing):*

1. An **unsuccessful search** examines $\alpha = n/m$ keys on average, since it must traverse an entire chain whose expected length is α .
2. A **successful search** examines $\Theta(1 + \alpha)$ keys on average.

Proof. For a successful search, when the i -th key is inserted, the average chain length is $(i-1)/m$. Therefore the expected number of comparisons to find the i -th key later is $1 + (i-1)/m$. Averaging over all n keys:

$$A_s(n) = \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{i-1}{m}\right) = 1 + \frac{1}{nm} \sum_{i=0}^{n-1} i = 1 + \frac{n-1}{2m}.$$

Since $\alpha = n/m$, this is $1 + \frac{\alpha}{2} - \frac{1}{2m} = \Theta(1 + \alpha)$. □

Key Result: Chained Hashing Performance

If n is proportional to m (i.e. $n = O(m)$), then $\alpha = O(1)$ and both successful and unsuccessful searches take $\Theta(1)$ expected time. This is the power of hashing: **constant-time search on average**.

13.4.2 Open Addressing

In open addressing, all keys are stored directly in the hash table (no linked lists). Consequently, the load factor satisfies $\alpha \leq 1$. When a collision occurs at slot $h(k)$, a *probe sequence* is followed to find an empty slot.

13.4.2.1 Linear Probing

The probe function is:

$$H(k, i) = (H'(k) + i) \bmod m, \quad i = 0, 1, 2, \dots, m-1,$$

where $H'(k)$ is the auxiliary hash function (e.g. $H'(k) = k \bmod m$).

Example 13.4.2 (Linear Probing). With $H(k, i) = (k + i) \bmod 7$, $H'(k) = k \bmod 7$, and keys $\{4, 13, 19, 11\}$:

$H(4, 0) = 4$; slot 4 is empty $\rightarrow$ insert 4 at slot 4.

$H(13, 0) = 6$; slot 6 is empty $\rightarrow$ insert 13 at slot 6.

$H(19, 0) = 5$; slot 5 is empty $\rightarrow$ insert 19 at slot 5.

$H(11, 0) = 4$; slot 4 is occupied (by 4). $H(11, 1) = 5$; slot 5 is occupied (by 19). $H(11, 2) = 6$; slot 6 is occupied (by 13). $H(11, 3) = 0$; slot 0 is empty $\rightarrow$ insert 11 at slot 0.

Primary Clustering

Linear probing suffers from *primary clustering*: long runs of occupied slots build up, increasing both search and insertion time. Keys that hash to any slot in a cluster must probe to the end of the cluster, and the cluster grows with each insertion.

13.4.2.2 Quadratic Probing

The probe function introduces a quadratic term:

$$H(k, i) = (H'(k) + c_1 i + c_2 i^2) \bmod m, \quad c_2 \neq 0, \quad i = 0, 1, \dots, m-1.$$

The constants c_1 , c_2 , and m must be chosen carefully so that all slots are visited. For $m = 2^p$, a good choice is $c_1 = c_2 = \frac{1}{2}$.

Example 13.4.3 (Quadratic Probing). With $H(k, i) = (H'(k) + \frac{1}{2}i + \frac{1}{2}i^2) \bmod 8$ and $H'(k) = k \bmod 8$, the probe offsets for $i = 1, 2, \dots, 7$ are 1, 3, 6, 2, 7, 5, 4—covering all slots.

Quadratic probing avoids primary clustering but introduces *secondary clustering*: if two keys have the same initial hash value $H'(k)$, their probe sequences are identical.

13.4.2.3 Double Hashing

A second hash function determines the probe step size, producing a more randomised probe sequence:

$$H(k, i) = (H_1(k) + i \cdot H_2(k)) \bmod m, \quad i = 0, 1, \dots, m - 1.$$

Here H_1 and H_2 are both auxiliary hash functions, and m should be prime to ensure full table coverage.

Example 13.4.4 (Double Hashing). With $m = 7$, $H_1(k) = k \bmod 7$, $H_2(k) = (k \bmod 3) + 1$, and keys $\{0, 4, 13, 19, 11\}$:

$$H(0, 0) = 0 \rightarrow \text{slot } 0.$$

$$H(4, 0) = 4 \rightarrow \text{slot } 4.$$

$$H(13, 0) = 6 \rightarrow \text{slot } 6.$$

$$H(19, 0) = 5 \rightarrow \text{slot } 5.$$

$H(11, 0) = 4$ (collision with 4); $H_2(11) = (11 \bmod 3) + 1 = 3$, so $H(11, 1) = (4 + 3) \bmod 7 = 0$ (collision with 0); $H(11, 2) = (4 + 6) \bmod 7 = 3 \rightarrow \text{slot } 3$.

13.4.2.4 Searching Under Open Addressing

The search algorithm probes slots until the key is found, an empty slot is reached (key absent), or all slots have been checked:

```

1 def hash_search(table, k, hash_fn, probe_fn, m):
2     code = hash_fn(k)
3     loc = code
4     while table[loc] is not None:
5         if table[loc].key == k:
6             return loc          # found
7         loc = probe_fn(loc)
8         if loc == code:
9             break              # full cycle
10    return None                 # not found

```

Three outcomes are possible: the key is found (success), an empty slot is reached (failure), or every slot has been probed without a match (failure).

13.4.3 Time Complexity of Open Addressing

The expected number of probes depends on the probing strategy and the load factor α .

Method	Successful Search	Unsuccessful Search
Linear Probing	$\frac{1}{2} \left(1 + \frac{1}{1 - \alpha} \right)$	$\frac{1}{2} \left(1 + \left(\frac{1}{1 - \alpha} \right)^2 \right)$
Double Hashing	$\frac{1}{\alpha} \ln \frac{1}{1 - \alpha}$	$\frac{1}{1 - \alpha}$

All expressions are functions of $\frac{1}{1 - \alpha}$, which grows rapidly as $\alpha \rightarrow 1$. This means that performance degrades sharply when the table is nearly full.

Why Performance Degrades as $\alpha \rightarrow 1$

As the load factor approaches 1, nearly every slot is occupied and each probe is likely to hit another key, causing long chains of probes. The expressions $\frac{1}{1-\alpha}$ and $\left(\frac{1}{1-\alpha}\right)^2$ diverge as $\alpha \rightarrow 1$, explaining why open-address hash tables must be kept well below full capacity.

13.5 Deletion Under Open Addressing

Deleting a key from an open-addressing hash table is more involved than in chaining. Simply emptying a slot would break the probe chain for any key that was inserted after the deleted one and probed through that slot.

The standard solution is **lazy deletion**: mark the slot as *deleted* (often called a “tombstone”) rather than emptying it. Tombstoned slots are treated as occupied during search (so the probe chain is not broken) but as empty during insertion (so new keys can reuse them).

After a large number of deletions, the table may accumulate many tombstones, degrading search performance. A periodic *garbage collection* pass— re-inserting all live keys into a clean table—restores efficiency.

13.6 Rehashing

As α increases, search and insertion become slower regardless of the probing strategy. *Rehashing* addresses this by allocating a larger table (typically doubling the size to the next prime) and re-inserting every existing key using the new hash function. This resets the load factor to approximately $\alpha/2$ and eliminates any tombstones.

13.7 Summary

Key Result: Hash Table Performance

1. A **direct-address table** provides $O(1)$ operations but requires $|U|$ space, which is impractical for large key universes.
2. **Hashing** compresses the key space via a hash function, achieving $O(1)$ average-case performance with $O(m)$ space.
3. **Separate chaining** (closed addressing) handles collisions with linked lists; average search is $\Theta(1 + \alpha)$.
4. **Open addressing** stores all keys in the table itself ($\alpha \leq 1$); performance depends on the probing strategy and degrades as $\alpha \rightarrow 1$.
5. Good hash function design (prime table sizes, avoiding powers of 2) and keeping α low via rehashing are essential for maintaining constant-time operations.

Chapter 14

Tries

A *trie* (also called a **prefix tree** or **digital tree**) is a tree-based data structure designed for efficient string operations. Unlike the hash tables of Chapter 13, which map individual keys to slots, a trie exploits the *shared prefix structure* of a collection of strings: characters common to multiple keys are stored only once along shared edges from the root.

Definition 14.0.1 (Trie). A *trie* over an alphabet Σ is a rooted tree in which

1. every edge is labelled with exactly one character from Σ ,
2. no node has two outgoing edges with the same label, and
3. a boolean flag `is_end_of_word` marks which nodes correspond to the final character of a stored string.

Searching or inserting a string of length L takes $O(L)$ time, independent of the total number of strings stored.

For example, the strings `bear`, `bell`, `bid`, `buy`, `car`, `care`, and `camp` produce the trie shown in Figure 14.1.

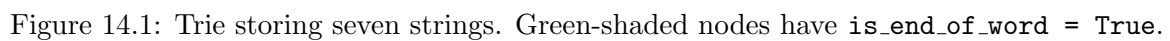
14.1 Applications of Tries

Tries appear in a wide range of practical systems:

- **Autocomplete.** Search engines and mobile keyboards use tries to suggest completions as the user types, because prefix lookup is proportional only to the length of the typed prefix.
- **Spell checking.** A dictionary stored as a trie allows $O(L)$ verification of whether a word exists, and can generate corrections using prefix matching or edit-distance techniques.
- **IP routing.** Longest-prefix matching in routing tables is naturally expressed as a trie traversal on binary representations of IP addresses.
- **Compression.** Algorithms such as LZW leverage trie-based pattern recognition to compress data.

14.2 Trie Implementation Strategies

Each internal node of a trie must map a character to a child node. Three common representations exist for this mapping.



Each node stores a Python dictionary `children` keyed by character. Lookup and insertion at each node are $O(1)$ average-case.

14.2.2 Array Children

```

1 class TrieNode:
2     def __init__(self):
3         self.children = [None] * 26
4         self.is_end_of_word = False

```

14.2.3 Linked List Children (Left-Child Right-Sibling)

Each node stores a single character, a pointer to its *first child*, and a pointer to the *next sibling* at the same level. This is the classic left-child right-sibling (LCRS) representation and is the primary implementation studied in this course.

Definition 14.2.1 (Trie Node (Linked-List Representation)). A `TrieNode` consists of four fields:

1. `char` — the character stored at this node,
2. `is_end_of_word` — boolean flag indicating whether this node completes a stored string,
3. `child` — pointer to the first child node (one level deeper in the trie), and
4. `next` — pointer to the next sibling node (same depth, same parent).

```
1 class TrieNode:
2     def __init__(self, char):
3         self.char = char
4         self.is_end_of_word = False
5         self.child = None
6         self.next = None
```

The `Trie` class itself holds a reference to a dummy root node:

```
1 class Trie:
2     def __init__(self):
3         self.root = TrieNode("")
```

Exam Tip: Navigating the Linked-List Trie

In the LCRS representation, `child` moves *down* a level (to the children of a node), while `next` moves *across* siblings at the same level. Searching for a particular character among a node's children requires traversing the sibling chain via `next`, which takes $O(|\Sigma|)$ in the worst case.

14.3 Core Operations

The linked-list trie supports three primary operations: searching for a word, inserting a word, and traversal. Deletion is rarely implemented in practice because dictionaries seldom change and deletion logic is substantially more complex than insertion.

14.3.1 Helper: Finding a Child

Both search and insert rely on a helper that traverses the sibling linked list of a node's children to locate a given character.

```
1 def _find_child(self, node, char):
2     current = node.child
3     while current:
4         if current.char == char:
5             return current
6         current = current.next
7     return None
```

The helper returns the matching `TrieNode` if found, or `None` otherwise. Its time complexity is $O(k)$ where k is the number of children of `node` (at most $|\Sigma|$).

14.3.2 Searching for a Word

To search for a word, we walk down from the root, consuming one character per level. If at any level the character is absent among the current node's children, the word is not in the trie. If we reach the end of the word, we check `is_end_of_word` to distinguish stored words from mere prefixes.

```
1 def search(self, word):
2     node = self.root
3     for char in word:
4         node = self._find_child(node, char)
5         if not node:
6             return False
7     return node.is_end_of_word
```

Example 14.3.1 (Searching for “bell”). Starting from the root: find ‘b’ among root’s children → found; find ‘e’ among b’s children → found; find ‘l’ among e’s children → found; find ‘l’ among the first l’s children → found. The final node has `is_end_of_word = True`, so the search returns `True`.

Searching for ‘bel’ would follow the same path but stop one level earlier, where `is_end_of_word = False`, so it returns `False`—the string is a prefix of a stored word but is not itself stored.

14.3.3 Inserting a Word

Insertion walks the same path as search, but creates new nodes for any character not yet present. New children are inserted *at the head* of the sibling linked list for $O(1)$ insertion.

```
1 def _add_child(self, node, char):
2     new_node = TrieNode(char)
3     new_node.next = node.child
4     node.child = new_node
5     return new_node
6
7 def insert(self, word):
8     node = self.root
9     for char in word:
10        child = self._find_child(node, char)
11        if not child:
12            child = self._add_child(node, char)
13        node = child
14    node.is_end_of_word = True
```

After processing every character, the last visited node is marked as an end-of-word. If the word (or a prefix of it) already exists, the algorithm simply walks existing nodes and sets the flag at the end.

Warning: Insertion Order Affects Sibling Order

Because `_add_child` prepends to the sibling list, the order of children at any node depends on the order in which words were inserted. A DFS traversal will therefore enumerate words in an order that reflects insertion history, not necessarily alphabetical order.

14.4 Trie Traversal

The binary tree traversal paradigms from Chapter 7 generalise naturally to tries. Since a trie node may have an arbitrary number of children (linked via `child/next`), we iterate over the

sibling chain rather than visiting fixed left and right pointers.

14.4.1 Depth-First Search (Preorder)

Preorder DFS visits a node before recursing into its children. For each child, we recurse, then advance to the next sibling.

```
1 def dfs(self, node):
2     if node is not None:
3         print(node.char, end=" ")
4         child = node.child
5         while child:
6             self.dfs(child)
7             child = child.next
```

For the trie in Figure 14.1, calling `dfs(root)` produces the traversal order:

`root` $\rightarrow$ `b` $\rightarrow$ `e` $\rightarrow$ `a` $\rightarrow$ `r` $\rightarrow$ `l` $\rightarrow$ `l` $\rightarrow$ `i` $\rightarrow$ `d` $\rightarrow$ `u` $\rightarrow$ `y`
 $\rightarrow$ `c` $\rightarrow$ `a` $\rightarrow$ `r` $\rightarrow$ `e` $\rightarrow$ `m` $\rightarrow$ `p`.

14.4.2 Breadth-First Search (Level-by-Level)

BFS uses a queue (cf. Chapter 5) to visit nodes level by level. When a node is dequeued, all its children are enqueued via the sibling chain.

```
1 def bfs(self):
2     queue = Queue()
3     queue.enqueue(self.root)
4     while not queue.is_empty():
5         node = queue.dequeue()
6         print(node.char, end=" ")
7         child = node.child
8         while child:
9             queue.enqueue(child)
10            child = child.next
```

The BFS order for Figure 14.1 is:

`root` $\rightarrow$ `b` $\rightarrow$ `c` $\rightarrow$ `e` $\rightarrow$ `i` $\rightarrow$ `u` $\rightarrow$ `a` $\rightarrow$ `a` $\rightarrow$ `l` $\rightarrow$ `d` $\rightarrow$ `y`
 $\rightarrow$ `r` $\rightarrow$ `m` $\rightarrow$ `r` $\rightarrow$ `l` $\rightarrow$ `e` $\rightarrow$ `p`.

14.5 Application: Printing All Words

A common task is to enumerate every word stored in a trie. Both DFS and BFS approaches carry a `prefix` string that accumulates characters along the path from the root; whenever an end-of-word node is reached, the prefix is a complete stored word.

14.5.1 DFS Approach

```
1 def print_all_words_dfs(self, node, prefix):
2     if node.is_end_of_word:
3         print(prefix)
4     child = node.child
5     while child:
6         self.print_all_words_dfs(child,
7             prefix + child.char)
8         child = child.next
```


Calling `print_all_words_dfs(root, "")` on the example trie prints the words in depth-first order: bear, bell, bid, buy, car, care, camp.

14.5.2 BFS Approach

In the BFS variant, the queue stores *pairs* of (node, prefix).

```
1 def print_all_words_bfs(self):
2     queue = Queue()
3     queue.enqueue((self.root, ""))
4     while not queue.is_empty():
5         node, prefix = queue.dequeue()
6         if node.is_end_of_word:
7             print(prefix)
8         child = node.child
9         while child:
10            queue.enqueue((child,
11                           prefix + child.char))
12            child = child.next
```

Key Result: Trie Traversal Complexity

Both DFS and BFS visit every node in the trie exactly once. If the trie contains N nodes, traversal is $O(N)$. Printing all words additionally incurs $O(L)$ per word for string concatenation, where L is the word's length.

14.6 Application: Autocomplete

Autocomplete suggests possible completions for a given prefix. The algorithm has two phases:

1. **Navigate to the prefix node.** Starting from the root, follow edges matching each character of the prefix using `_find_child`. If the prefix is not present in the trie, there are no suggestions.
2. **Collect all words below.** From the prefix node, perform a DFS or BFS (as in Section 14.5) to enumerate every stored word that extends the prefix.

For instance, searching for the prefix “ca” in the example trie navigates `root → c → a`, then collects the completions `car`, `care`, and `camp`. In practice, results may be ranked by frequency, recency, or other heuristics before being displayed to the user.

14.7 Application: Spell Checking

A trie-based spell checker operates in two stages:

1. **Validity check.** Search the trie for the input word. If `search` returns `True`, the word is valid.
2. **Suggestion generation.** If the word is invalid, generate candidate corrections. Common strategies include:
 - *Prefix matching*—find stored words sharing the longest common prefix with the misspelled word.
 - *Edit distance*—use metrics such as Levenshtein distance to find words within a small number of insertions, deletions, or substitutions.

- *Frequency ranking*—prioritise more commonly used words among the candidates.
- *User history / context*—adjust rankings based on the user’s prior input.

14.8 Summary

Key Result: Trie Operations

1. A **trie** stores a set of strings by sharing common prefixes, achieving $O(L)$ search and insertion for a string of length L .
2. The **linked-list (LCRS) representation** uses **child** and **next** pointers; finding a child costs $O(|\Sigma|)$ per level in the worst case.
3. **DFS** (preorder) and **BFS** (level-by-level) both visit every node once in $O(N)$ time and extend naturally from binary tree traversals to the multi-child trie setting.
4. **Autocomplete** navigates to a prefix node, then collects all words below via traversal.
5. **Spell checking** combines trie search with edit-distance or prefix-matching heuristics to suggest corrections.

Chapter 15

Heaps

A *heap* is a specialised binary tree that maintains a simple ordering invariant between parents and children. Because heaps can be stored compactly in a contiguous array with no pointers, they underpin several fundamental data-structure and algorithm-design patterns, including priority queues (introduced informally in Chapter ??) and an efficient $O(n \log n)$ sorting algorithm.

15.1 Definition and Properties

Definition 15.1.1 (Binary Heap). A **binary heap** is a complete binary tree satisfying the *heap-order property*. A binary tree is *complete* when every level is fully occupied except possibly the last, which is filled from left to right. Two variants exist:

- **Max-heap**: for every node i (other than the root), $A[\text{parent}(i)] \geq A[i]$.
- **Min-heap**: for every node i (other than the root), $A[\text{parent}(i)] \leq A[i]$.

In a max-heap the largest element is always at the root; in a min-heap the smallest element is at the root. Neither variant imposes any ordering between siblings—only the parent–child relationship matters.

Exam Tip: Heap vs. BST

A heap is *not* a binary search tree. In a BST the left child is smaller and the right child is larger; in a heap both children are smaller (max-heap) or both are larger (min-heap). No left–right ordering is guaranteed.

15.2 Array Representation

Because a heap is a *complete* binary tree, it can be stored in a simple array with no explicit child or parent pointers. For the node at index i (0-indexed):

$$\text{left}(i) = 2i + 1, \quad (15.1)$$

$$\text{right}(i) = 2i + 2, \quad (15.2)$$

$$\text{parent}(i) = \left\lfloor \frac{i - 1}{2} \right\rfloor. \quad (15.3)$$

The height of a heap with n elements is $\lfloor \log_2 n \rfloor$, which bounds the cost of any root-to-leaf traversal.

Example 15.2.1 (Max-Heap Array Layout). Consider the max-heap $[25, 20, 16, 9, 8, 5, 2]$. The root is $A[0] = 25$; its children are $A[1] = 20$ and $A[2] = 16$. The children of $A[1] = 20$ are $A[3] = 9$ and $A[4] = 8$, while those of $A[2] = 16$ are $A[5] = 5$ and $A[6] = 2$. Every parent is at least as large as its children, confirming the max-heap property.

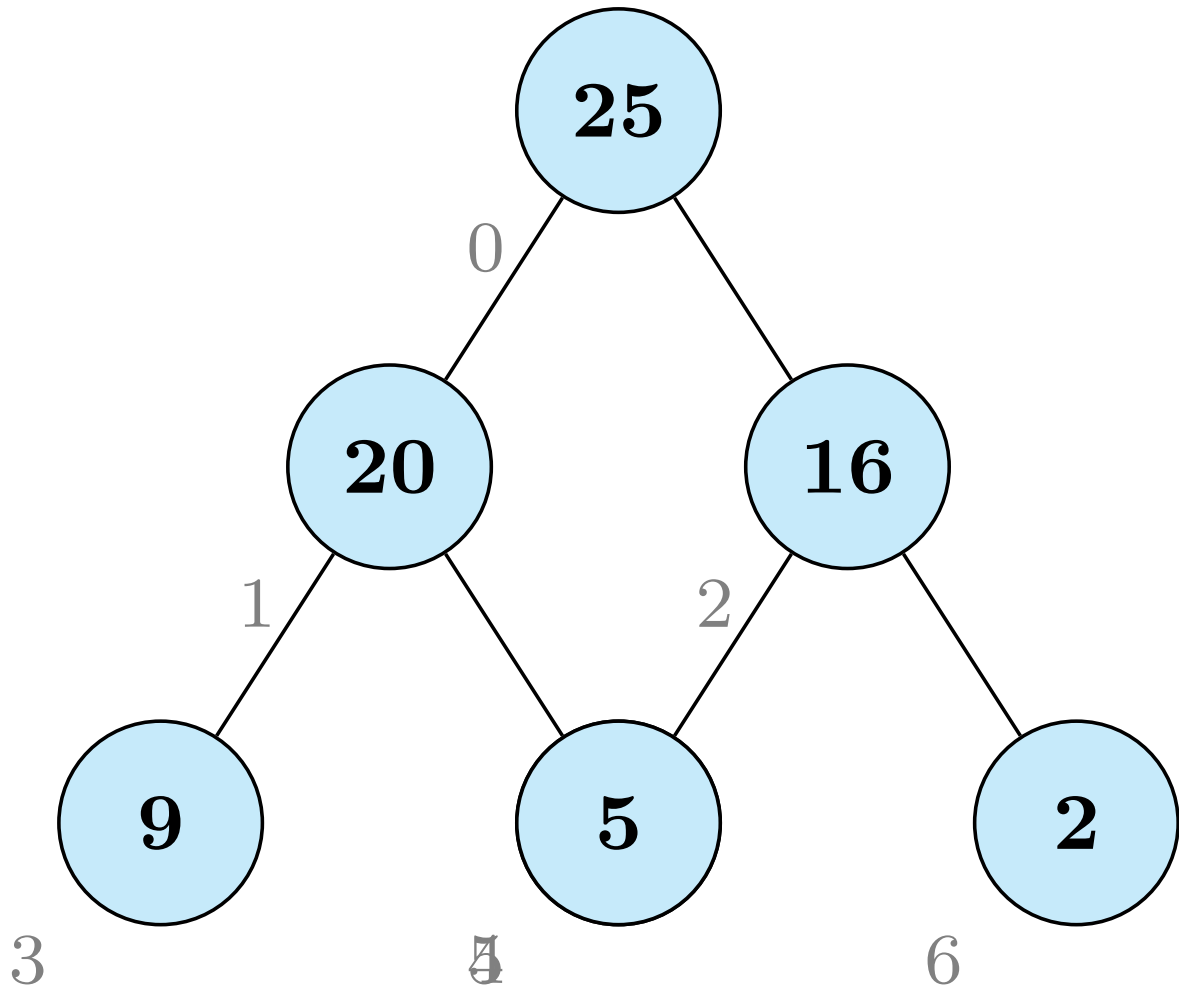


Figure 15.1: A max-heap of seven elements and its array representation $[25, 20, 16, 9, 8, 5, 2]$.

Key Result: Heap Index Formulas

For a 0-indexed array, the children of node i are at $2i + 1$ and $2i + 2$; its parent is at $\lfloor (i - 1)/2 \rfloor$. The height of an n -element heap is $\lfloor \log_2 n \rfloor$.

15.3 Core Operations

15.3.1 Heapify (Sift Down)

The MAX-HEAPIFY procedure restores the max-heap property at a single node i , assuming that the subtrees rooted at the left and right children of i are already valid max-heaps. It compares $A[i]$ with its two children, swaps $A[i]$ with the largest child if necessary, and recurses on the affected subtree.

Example 15.3.1 (Heapify Trace). Starting with the array $[16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$, calling MAX-HEAPIFY($A, 1$) proceeds as follows. Node 1 holds 4, whose children are $A[3] = 14$ and $A[4] = 7$. Since $14 > 4$, we swap 4 and 14, giving $[16, 14, 10, 4, 7, 9, 3, 2, 8, 1]$. Now node 3 holds 4 with children $A[7] = 2$ and $A[8] = 8$. Since $8 > 4$, we swap 4 and 8, yielding the final array $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$. The subtree rooted at index 1 is now a valid max-heap.

Algorithm 6 MAX-HEAPIFY(A, i)

```
1:  $l \leftarrow \text{left}(i)$ 
2:  $r \leftarrow \text{right}(i)$ 
3: if  $l < \text{size}(A)$  and  $A[l] > A[i]$  then
4:    $\text{largest} \leftarrow l$ 
5: else
6:    $\text{largest} \leftarrow i$ 
7: end if
8: if  $r < \text{size}(A)$  and  $A[r] > A[\text{largest}]$  then
9:    $\text{largest} \leftarrow r$ 
10: end if
11: if  $\text{largest} \neq i$  then
12:   exchange  $A[i]$  and  $A[\text{largest}]$ 
13:   MAX-HEAPIFY( $A, \text{largest}$ )
14: end if
```

The element “floats down” at most $h = \lfloor \log_2 n \rfloor$ levels, performing $O(1)$ work per level, so the running time is $O(\log n)$.

15.3.2 Build Heap

To convert an arbitrary array of n elements into a max-heap, we apply MAX-HEAPIFY to every non-leaf node, working from the last internal node up to the root. Leaf nodes (indices $\lfloor n/2 \rfloor$ through $n - 1$) are trivially valid one-element heaps.

Algorithm 7 BUILD-MAX-HEAP(A)

```
1: for  $i = \lfloor (\text{length}(A) - 1)/2 \rfloor$  down to 0 do
2:   MAX-HEAPIFY( $A, i$ )
3: end for
```

Example 15.3.2 (Building a Max-Heap). Given $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$, the last internal node is at index $\lfloor (10 - 1)/2 \rfloor = 4$. Heapifying from index 4 down to 0 proceeds through the following key snapshots:

1. Index 4 ($A[4] = 16$): children are $A[9] = 7$; already valid—no swap.
2. Index 3 ($A[3] = 2$): child $A[7] = 14$ is larger; swap $\Rightarrow A[3] = 14$.
3. Index 2 ($A[2] = 3$): child $A[6] = 10$ is larger; swap $\Rightarrow A[2] = 10$.
4. Index 1 ($A[1] = 1$): child $A[3] = 14$ is larger; swap, then recurse.
5. Index 0 ($A[0] = 4$): child $A[1] = 16$ is larger; swap, then recurse.

The final max-heap is $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$.

Key Result: Build-Heap is $O(n)$

Although there are $O(n)$ calls to MAX-HEAPIFY, each costing $O(\log n)$ in the worst case, a tighter analysis shows that most nodes are near the bottom of the tree and heapify over very few levels. Summing across all levels gives a total cost of $O(n)$, *not* $O(n \log n)$.

15.3.3 Insertion

To insert a new key into a max-heap:

1. Place the new element at the next available position (the end of the array), preserving the complete-tree shape.
2. **Bubble up**: repeatedly compare the new element with its parent and swap if the new element is larger, until the heap-order property is restored or the root is reached.

The new element rises at most h levels, so insertion runs in $O(\log n)$.

Example 15.3.3 (Inserting 15 into a Max-Heap). Starting from the max-heap $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$, we append 15 at index 10. Since $15 > A[\text{parent}(10)] = A[4] = 7$, swap to get $A[4] = 15$. Then $15 > A[\text{parent}(4)] = A[1] = 14$, swap to get $A[1] = 15$. Finally $15 < A[0] = 16$, so the process stops. The resulting heap is $[16, 15, 10, 8, 14, 9, 3, 2, 4, 1, 7]$.

15.3.4 Deletion

To delete an arbitrary node at index i from a max-heap:

1. Replace $A[i]$ with the last element of the heap.
2. Remove the last element (decrease heap size by one).
3. Restore the heap property:
 - If the replacement value is *greater* than its parent, **bubble up**.
 - If it is *smaller* than one of its children, **heapify down**.

Since either direction traverses at most $O(\log n)$ levels, deletion runs in $O(\log n)$.

Example 15.3.4 (Deleting the Root (Extract-Max)). From $[16, 15, 10, 8, 14, 9, 3, 2, 4, 1, 7]$, delete the root (16). Replace $A[0]$ with the last element 7, giving $[7, 15, 10, 8, 14, 9, 3, 2, 4, 1]$. Now heapify down from index 0: swap 7 with 15 (index 1), then swap 7 with 14 (index 4). The resulting heap is $[15, 14, 10, 8, 7, 9, 3, 2, 4, 1]$.

Common Mistake: Forgetting to Check Both Directions

When deleting a node that is not the root, the replacement value could be either larger or smaller than its new neighbours. Always check *both* the parent (for a possible bubble up) *and* the children (for a possible heapify down). Checking only one direction leaves the heap in an invalid state.

15.4 Complexity Summary

Operation	Time	Notes
Build Heap	$O(n)$	Bottom-up construction
Insert	$O(\log n)$	Bubble up
Delete	$O(\log n)$	Replace + heapify/bubble
Heapify	$O(\log n)$	Sift down one path
Find max / min	$O(1)$	Root of max-/min-heap

15.5 Applications

Heaps underlie several important algorithms and data-structure patterns:

1. **Priority queues.** A max- or min-heap provides $O(\log n)$ insertion and $O(\log n)$ extract-max/min, making it the standard implementation of a priority queue. Operating-system task schedulers and print queues are classic examples.
2. **Heap sort.** Build a max-heap from the input array ($O(n)$), then repeatedly extract the maximum and place it at the end of the array. Each extraction costs $O(\log n)$, giving an overall time complexity of $O(n \log n)$ with $O(1)$ extra space.
3. **Graph algorithms.**
 - *Dijkstra's algorithm* uses a min-heap (priority queue) to repeatedly select the unvisited vertex with the smallest tentative distance, achieving $O((V + E) \log V)$.
 - *Prim's algorithm* uses a min-heap to pick the cheapest edge crossing the cut at each step when building a minimum spanning tree.

15.6 Summary

Key Result: Heap Operations

1. A **binary heap** is a complete binary tree stored in an array; the parent-child relationship is computed via index arithmetic ($2i + 1$, $2i + 2$, $\lfloor (i - 1)/2 \rfloor$).
2. MAX-HEAPIFY restores the heap property at a single node in $O(\log n)$ by sifting down; BUILD-MAX-HEAP converts an arbitrary array in $O(n)$.
3. Insertion (bubble up) and deletion (replace + heapify/bubble) both run in $O(\log n)$.
4. Heaps are the standard backing structure for priority queues and enable heap sort ($O(n \log n)$, in-place) and efficient graph algorithms (Dijkstra, Prim).